
FISHBALL7020 · PLUTOSKY · 7020-SDR

Fabric School

A ground-up course in software-defined radio, and in the FPGA inside this one. 46 lessons, from what a radio *is* to your own signal processing running in the AD9361 datapath — by way of spectra, noise, filters, modulation and link budgets.

Assumes no Verilog, no Vivado, no FPGA experience, and no signal processing. Every term is defined the first time it appears.

Xilinx XC7Z020-CLG400 · Analog Devices AD9361, 2R2T
Vivado / Vitis 2022.2 · Ubuntu 22.04 · Verilog-2001
Written against the fishball7020-fpga-devkit block design

Contents

BEFORE ANY CODE

- 0 What a radio does, and what "software-defined" changes
- 1 What an FPGA is, and why it is not a processor
- 2 What is on this board
- 3 The toolchain, and what each tool is for

VERILOG FROM NOTHING

- 4 Your first module
- 5 Clocks, registers and reset
- 6 The two assignments, and why it matters
- 7 Combinational logic, and the latch trap
- 8 Numbers: width, signedness, overflow
- 9 Fixed point: the arithmetic the fabric actually does
- 10 Testbenches, and why you will not skip them

THE RADIO'S DATAPATH

- 11 What an IQ sample actually is
- 12 Sample rate, Nyquist and aliasing
- 13 The block design: the map, and what it quietly assumes
- 14 Clocks, valid strobes, and the 2R2T trap
- 15 The packers, and the filter only channel 0 has
- 16 How samples reach memory, and back

CHANGING THE FABRIC

- 17 The worked example, line by line
- 18 Wiring it in with Tcl
- 19 Packaging your Verilog as an IP, and splicing it into the datapath
- 20 Pins and constraints
- 21 Driving Vivado, and reading what it tells you
- 22 Crossing clock domains
- 23 Registers, and talking to Linux

SIGNALS, BEFORE THE FABRIC

- 24 The frequency domain, and what a spectrum really is
- 25 Noise, decibels, and where the floor comes from

DSP IN THE FABRIC

- 26 Filters, from first principles
- 27 Decimation, and why taps are nearly free
- 28 Mixers, oscillators and CORDIC

BUILDING A RADIO LINK

- 29 What modulation actually is, and where to put it
- 30 Pulse shaping, and why root-raised-cosine
- 31 Synchronisation: finding the signal inside the samples
- 32 Correlation, matched filters, and finding the packet at all
- 33 OFDM, and why this board finds it hard
- 34 Equalisation: undoing a channel that is not flat
- 35 Channel coding: buying decibels with redundancy
- 36 Packets: framing, addressing, and everything above the symbols

MEASURING, AND GETTING ON THE AIR

37 Measuring what you built

38 Link budgets: will it actually work over the air?

39 Antennas and the RF front end: what happens after the SMA

40 IQ files, and the metadata that turns them into measurements

TWO RECEIVERS, AND THE CHIP THAT FEEDS THEM

41 Two coherent receivers: what this board can do that most cannot

42 MIMO and beamforming: what more than one antenna buys

43 The AD9361 itself: the chip in front of your fabric

ONWARD

44 Projects, in order of difficulty

45 The rules worth taping to the wall

APPENDIX

A Where to go for more, and what this course still leaves out

0 What a radio does, and what "software-defined" changes

Start here even if it feels too basic. Half the confusion later comes from fuzzy ideas about what is actually travelling down the cable.

A radio signal is a wave — an electrical voltage that swings up and down very fast. How fast is its **frequency**, measured in hertz (Hz), cycles per second. A million cycles per second is a megahertz (MHz); a billion is a gigahertz (GHz). This board works between 70 MHz and 6 GHz.

To send information you have to **modulate** that wave: change something about it in a pattern the other end can read. You can vary its height (**amplitude**), its timing (**phase**), or its frequency. That is all radio is.

The traditional way, and the software way

A conventional radio does all of this with dedicated circuits. An FM radio has physical parts whose job is specifically to demodulate FM; it cannot become a Wi-Fi receiver, because its behaviour is soldered in.

A **software-defined radio** (SDR) moves the boundary. It keeps only the parts that must be analogue — an antenna connection, amplifiers, filters, and converters — and then turns the signal into *numbers* as early as possible. From that point on, what the radio *is* depends on what you do with the numbers. Same hardware, different software, completely different radio.

THREE WORDS YOU NOW OWN

- ADC** Analogue-to-Digital Converter. Measures a voltage many times a second and produces a number each time. The doorway from the physical world into the numeric one.
- DAC** Digital-to-Analogue Converter. The reverse: turns a stream of numbers back into a voltage. The doorway out.
- Sample** One of those numbers — one measurement of the signal at one instant.

So the shape of every SDR is the same: antenna → analogue bits → ADC → **numbers** → something that processes numbers → and back out the other way. This course is about that "something", and specifically about making it be *your* design.

WHY BOTHER PUTTING YOUR OWN LOGIC IN THERE AT ALL

You could process the numbers on a PC, and often that is the right answer. But the numbers come out fast — up to 61.44 million of them per second on this board, each a pair — and getting them to a PC costs bandwidth and adds delay you cannot control. Some jobs must happen where the samples are: anything that must react in microseconds, anything that must reduce the data before it can be shipped, anything that must be exactly and repeatably timed. That is what the FPGA is for, and it is the subject of the rest of this course.

1 What an FPGA is, and why it is not a processor

The single mental shift that makes everything else make sense.

A **processor** — the CPU in your laptop — is a fixed piece of circuitry that reads instructions one after another and does what each one says. It is fast because it does each step very quickly, but it is fundamentally doing *one thing at a time*.

An **FPGA** — Field-Programmable Gate Array — is not like that. It is a large field of tiny, blank logic elements plus a switching network between them. "Programming" it does not mean giving it instructions to follow. It means **configuring what circuit it physically becomes**.

WHAT THE FIELD IS MADE OF

LUT	Look-Up Table. A tiny memory, typically 6 inputs wide, that can be filled in to produce <i>any</i> logical function of those inputs. This board's chip has 53,200 of them.
Flip-flop	A one-bit memory that captures its input at a precise moment and holds it. Everything that needs to remember a value between clock ticks is built from these.
DSP slice	A hard-wired multiplier-and-adder. Multiplication is expensive to build out of LUTs, so the chip includes 220 ready-made ones. Filters live on these.
Block RAM	Small dedicated memories for buffering data on-chip.

The consequence: everything happens at once

If you describe ten filters, you get ten filters, all running simultaneously, all the time. Not ten filters taking turns. Adding an eleventh does not slow the other ten down — it uses more of the chip. You trade *area* instead of *time*.

This is why FPGAs suit radio. A signal arriving at 61 million samples per second does not wait. An FPGA can have a permanent, physical arrangement of hardware that handles every sample as it goes past, with completely predictable timing.

THE BEGINNER'S MISTAKE

Writing Verilog as if it were software. A line of Verilog is not a step that happens after the previous line — it is a piece of circuitry that exists at the same time as every other piece. Two **always** blocks are two independent lumps of hardware, both live on every clock edge. If you find yourself thinking "and then it does...", stop and ask what the circuit looks like instead.

The price you pay

- **It is slower per operation.** Fabric typically runs at tens to a few hundred MHz, far below a CPU's gigahertz. It wins by doing thousands of things at once, not by being quick.
- **The build is slow.** Turning your description into a configuration means synthesis, placement and routing — twenty minutes to an hour on this board. You cannot iterate the way you do with software, which is why simulation matters so much (lesson 10).

- **Resources are finite and visible.** You will run out of DSP slices or fail timing, and you will have to make real engineering trade-offs.

2 What is on this board

Two computers, one radio chip, and the connection between them.

The main chip is a Xilinx XC7Z020-CLG400, a **Zynq**. A Zynq is unusual: it puts a processor and an FPGA on the same die.

- PS **Processing System** — two ARM Cortex-A9 cores. Fixed silicon. This is where Linux boots, where the IIO daemon runs, where your Python sits. You cannot change its circuitry.
- PL **Programmable Logic** — the FPGA fabric described in lesson 1: 53,200 LUTs, 220 DSP slices. This is what Verilog describes and what a bitstream configures.
- AXI The bus that connects them. When Linux writes to a register and your fabric logic sees it change, AXI carried that write across the boundary.

The radio chip is an Analog Devices **AD9361**: a complete transceiver covering 70 MHz to 6 GHz, with **two receivers and two transmitters**. It contains the amplifiers, the mixers, the filters, and the ADCs and DACs. It is connected to the FPGA fabric by a fast digital interface, and configured by the Linux driver over SPI, a simple serial control bus.

BOARD VOCABULARY

- Bitstream** The file that configures the PL — about 4 MB. It lives inside `BOOT.bin` on the SD card and is loaded at every power-on.
- FSBL** First Stage Boot Loader. The first code that runs; it loads the bitstream and then the bootloader.
- Device tree** A data file describing the hardware to Linux — which chips exist, at which addresses, with which settings. Linux does not discover any of this; it is told.
- 2R2T** "Two receive, two transmit" — the AD9361 mode this board runs in. Remember it; it has consequences in lesson 14.

TWO THINGS THE FABRIC CANNOT REACH

The **USER LED** sits on a PS pin (MIO 0). MIO pins belong to the processor side and are simply not wired into the fabric — no amount of design work will let your HDL blink it. It is a software job.

Most header pins are likewise unknown territory. Four are verified and safe to drive: JP5 pins 7, 9, 11 and 13. Driving an arbitrary pin that turns out to be an input, or tied elsewhere, can damage the board.

3 The toolchain, and what each tool is for

Five programs with confusing names. Here is which one does what.

TOOL	WHAT IT DOES
Vivado	Xilinx's FPGA tool. Takes Verilog plus a wiring diagram, and produces a bitstream. Synthesis, placement, routing and timing analysis all happen here.
Vitis	The software half of the same suite. Here it is used for exactly one thing: building the FSBL.
Icarus Verilog	A free simulator. Runs your Verilog as a program so you can check it does the right thing — in about a second.
Buildroot	Builds the Linux root filesystem that runs on the ARM cores.
<code>./devkit</code>	This repo's wrapper. It drives all of the above in the right order with the right options, and refuses to do dangerous things.

The commands you will actually type

terminal	SHELL
<pre># run from: the repo root ./devkit doctor # can this machine build? answers in a second, not at minute 40 ./devkit setup # fetch upstream source and apply this repo's patches ./devkit sim # simulate the custom HDL (~1 second) ./devkit build # everything (45-90 min) ./devkit build --hdl-only # just the FPGA part (~20 min) ./devkit verify # is the build sane? checks timing, compression, files ./devkit flash # copy onto the running board over the network</pre>	

THE NUMBER THAT SHAPES YOUR WHOLE WORKFLOW

Simulation takes **one second**. A build takes **twenty minutes**. That ratio is 1200:1, and it is the reason the habits in lesson 10 are not optional pedantry. Every question you can answer in the simulator, you answer in the simulator.

4 Your first module

Verilog is a big language. You need about a fifth of it, and this lesson is most of that fifth.

A **module** is the unit of Verilog: a box with wires going in and out, and a description of what is inside. Everything is a module, all the way up.

counter.v

VERILOG

```
// Everything after two slashes is a comment.
module counter (
    input wire    clk,      // one bit, coming in
    input wire    rst,      // one bit, coming in
    output reg [31:0] count // 32 bits, going out
);

    always @(posedge clk) begin
        if (rst)
            count <= 32'd0;
        else
            count <= count + 32'd1;
    end

endmodule
```

Reading it line by line

- `module counter ( ... );` — declares a box called `counter` and lists its wires. `endmodule` closes it.
- `input` / `output` — which way each wire points.
- `[31:0]` — this is not one wire but a bundle of 32, numbered 31 down to 0. Bit 0 is the least significant. Verilog calls a bundle a **vector**.
- `always @(posedge clk)` — "whenever `clk` goes from low to high, do the following". `posedge` is the rising edge.
- `<=` — an assignment. Lesson 6 explains why it is this arrow and not `=`.
- `32'd0` — a literal: 32 bits wide, `d` for decimal, value 0. You will also see `'h` (hex) and `'b` (binary).

Now read it as hardware rather than as instructions: *there is a 32-bit register called `count`. Permanently wired to its input is an adder that computes `count + 1`, and a selector that picks either that or zero depending on `rst`. On every rising clock edge the register captures whatever the selector is presenting.* About 32 flip-flops and a handful of LUTs, existing all at once.

WIRE VERSUS REG

- wire** A connection. Something else drives it continuously. Use for anything you assign with `assign`, and for module inputs.
- reg** Anything you assign inside an `always` block. The name is genuinely misleading: a `reg` only becomes a real flip-flop if you assign it on a clock edge. Assign it in a combinational block and it is just a wire with a confusing keyword.

5 Clocks, registers and reset

Why digital hardware has a heartbeat, and what it is for.

A **clock** is a signal that alternates high, low, high, low, forever, at a fixed rate. It carries no data. Its only job is to say *now*.

Logic gates take time to settle — a signal rippling through an adder arrives at different bits at slightly different moments, and for a few nanoseconds the output is meaningless. The clock solves this by dividing time into slices. Within each slice the combinational logic settles; at the edge, every flip-flop captures the settled value at once.

THIS IS WHAT "TIMING" MEANS

The whole of timing analysis is one question: **does every signal have enough time to settle between one clock edge and the next?** If the longest path through your logic takes longer than a clock period, the flip-flop at the end captures a half-finished value. Vivado measures this and reports it as slack (lesson 21). It is why adding a pipeline register fixes timing: you cut a long path into two shorter ones and give each a full clock period.

Reset

At power-on, flip-flops hold arbitrary values. **Reset** is a signal that forces them to known values so the design starts from a defined state.

Reset only what needs it. Over-resetting costs fabric and can hurt timing, because the reset signal must reach every flip-flop that uses it within one clock period. A counter needs reset; a pipeline register that will be overwritten within two cycles anyway usually does not.

The clock on this board

The datapath clock is called `l_clk`, and it is recovered from the AD9361's own data clock. It is **not a fixed frequency** — it scales with the sample rate. Change the radio's sample rate from software and `l_clk` changes underneath your logic. Do not write anything that assumes a particular number of clock cycles per second.

6 The two assignments, and why it matters

The most common source of designs that simulate correctly and behave wrongly in hardware.

THE RULE, NO EXCEPTIONS

Use **non-blocking** `<=` inside clocked blocks (`always @(posedge clk)`).

Use **blocking** `=` inside combinational blocks (`always @(*)`).

Check yourself: which of these makes three flip-flops, and which makes one?

`b <= a; c <= b;` makes **three** — `a`, `b` and `c` each get a register, and `c` receives the *old* `b`, so a value takes two clock ticks to travel the chain.

`b = a; c = b;` makes **one**. The assignments happen in order, like software, so `c` receives the value that arrived this instant and the intermediate register collapses away.

Why the rule exists

Consider a rank of flip-flops that should shift a value along: `a` into `b`, `b` into `c`. In real hardware all three capture *simultaneously*, so `c` gets the *old* `b`, not the new one.

a shift register, correct

VERILOG

```
always @(posedge clk) begin
    b <= a;
    c <= b;      // gets the OLD b - three separate flip-flops
end
```

the same thing, wrong

VERILOG

```
always @(posedge clk) begin
    b = a;
    c = b;      // gets the NEW b - collapses to a single flip-flop
end
```

Non-blocking assignments all read their right-hand sides *first*, then update every left-hand side together. That is precisely what a rank of flip-flops does on an edge. Blocking assignments happen in written order, like software — correct for describing a chain of gates, wrong for describing registers.

Mix them and you get a design whose simulation and synthesis disagree. Your testbench passes and the board misbehaves — the most expensive bug class there is.

7 Combinational logic, and the latch trap

Logic with no memory: outputs follow inputs continuously.

two forms

VERILOG

```
// continuous assignment - permanent wiring
assign sum    = a + b;
assign isbig  = (a > b);           // a comparator
assign pick   = sel ? a : b;       // a multiplexer ("mux")

// block form, for anything needing if/case
always @(*) begin
    case (sel)
        2'b00: out = a;
        2'b01: out = b;
        2'b10: out = c;
        default: out = 16'd0;      // ALWAYS write a default
    endcase
end
```

`@(*)` means "re-evaluate whenever any input changes" — which is what a lump of gates does naturally.

INFERRED LATCHES

If a combinational block fails to assign a signal on some path, the synthesiser must make the signal *remember* its previous value — so it builds a **latch**, a memory element that is not clocked.

Latches are almost never intended. They are hard to time, they behave differently in simulation and hardware, and they will produce warnings you are tempted to ignore. Avoid them completely by always writing a `default` in every `case`, and an `else` on every `if` — or by assigning every output a safe value at the top of the block and then overriding it.

8 Numbers: width, signedness, overflow

Radio samples are signed and awkwardly sized. Verilog will silently do the wrong thing if you let it.

Width is not checked

Assign a 16-bit expression to a 12-bit target and Verilog quietly discards the top four bits. No error, no warning by default. On this board you move constantly between 12-bit converter samples and 16-bit bus words, so be explicit about widths everywhere.

Signed means saying so

IQ samples are **two's complement** signed numbers: the top bit means negative. A plain `reg [15:0]` is **unsigned** as far as Verilog is concerned, so `a > b` gives wrong answers whenever negatives are involved — and half your samples are negative.

signed arithmetic

VERILOG

```
wire signed [11:0] adc;          // 12 bits from the converter
wire signed [15:0] wide;

// widening a signed number means REPLICATING the sign bit,
// not padding with zeros
assign wide = {{4{adc[11]}}, adc};

// {a, b} concatenates
// {4{x}} repeats x four times
```

Overflow, and what to do about it

Add two 16-bit signed numbers and the result needs 17 bits. Multiply two 16-bit numbers and you need 32. If you keep the result in 16 bits, a large sum *wraps around* — a big positive becomes a big negative, which in a radio sounds like a violent click and looks like broad spectral splatter.

Two honest choices: grow the width to fit, or **saturate** — clamp to the maximum instead of wrapping. Saturation is usually right for signal paths, because a clipped peak is far less damaging than an inverted one.

THE SIZES ON THIS BOARD

Receive The ADC gives 12 bits, **signed**, sign-extended into a 16-bit container. Full scale is ± 2047 , not ± 32767 .

Transmit The DAC takes the **top 12 bits** of your 16-bit word and discards the bottom four. Scale your transmit samples to the full 16 bits — scaling them to ± 2047 transmits 24 dB too quietly.

9 Fixed point: the arithmetic the fabric actually does

There is no floating point in the fabric. Every DSP block you write manipulates integers pretending to be fractions, and getting that pretence right is most of the craft.

What "fixed point" means

A floating-point number carries its own scale — the exponent moves so the same format handles 0.0001 and 10000. That flexibility costs hardware you do not have. So the fabric uses plain integers, and *you* remember where the decimal point is. It never moves, hence **fixed point**.

The usual notation is **Qm.n**: *m* integer bits, *n* fractional bits, plus a sign bit. A signed 16-bit word holding values between -1 and +1 is **Q0.15**: one sign bit and fifteen fractional bits.

FORMAT	RANGE	STEP	USED FOR
Q0.15 (16-bit)	-1 ... +0.99997	1/32768	Normalised samples, filter coefficients
Q1.14	-2 ... +1.99994	1/16384	After a gain of up to 2
Q15.16 (32-bit)	-32768 ... +32767	1/65536	Accumulators

The board's own numbers are fixed point already. Receive samples are 12-bit signed values sign-extended into 16 bits — full scale ± 2047 . Transmit takes the top 12 bits of a 16-bit word, so it is effectively Q0.15 with the bottom four bits discarded.

The number that governs everything: $6.02N + 1.76$

An ideal *N*-bit converter has a best possible signal-to-noise ratio of

the fundamental limit	dB
$\text{SNR} = 6.02 \times N + 1.76 \quad \text{dB}$	

Each extra bit buys about **6 dB**. For the AD9361's 12-bit converters that is **74 dB** — the ceiling, before anything else in the system degrades it.

THIS IS WHERE THE DECIMATION GAIN COMES FROM

That 74 dB is spread across the whole sampled bandwidth. Filter down to a fraction of it and you keep the signal but discard most of the noise — Analog Devices call the correction term **process gain**. It is the same $10 \cdot \log_{10}(D)$ from lesson 26, and dividing it by 6.02 is what turns decibels into "extra bits".

Decimating by 307 gives 24.9 dB, which is 4.1 bits, which takes a 12-bit converter to about 16 bits *in that channel*. The formula above is why the bits are the natural unit.

Scaling: the mistake that costs 40 dB

Fixed point has no automatic gain. If you use only a small part of the available range, you get only a small part of the available dynamic range — permanently.

TWO WAYS TO UNDER-SCALE ON THIS BOARD

On transmit. The DAC takes bits [15:4] of your 16-bit word. Scale your samples to ± 2047 — as if they were receive samples — and you have handed the converter only its bottom bits. Measured on this hardware: amplitude 32767 produces 24 dB more output than 2047, exactly the factor of 16 the alignment predicts. Scale to ± 32767 .

In your own arithmetic. Carry a signal at a tenth of full scale through a filter chain and every stage quantises it against the same fixed step, so you lose about 20 dB of headroom you never get back.

Growth, and the two honest responses

Arithmetic makes numbers bigger. Adding two 16-bit values needs 17 bits; multiplying two needs 32; a 129-tap filter accumulating products needs more still.

- **Grow the width** to fit, then deliberately round back down at the end. Correct, and costs fabric.
- **Saturate** — clamp at the maximum rather than letting the value wrap. Wrapping turns a large positive into a large negative, which in a radio is a violent discontinuity and broad spectral splatter. A clipped peak is far less damaging than an inverted one.

What you must never do is let it wrap silently, which is exactly what Verilog does by default.

saturating add

VERILOG

```
wire signed [16:0] sum = a + b;           // 17 bits: cannot overflow
wire signed [15:0] out =
    (sum > 17'sd32767) ? 16'sd32767 :      // clamp high
    (sum < -17'sd32768) ? -16'sd32768 :     // clamp low
    sum[15:0];
```

Rounding versus truncation

When you discard low bits, truncating (just dropping them) always biases *downwards*. Over a long filter that accumulates into a measurable DC offset. Rounding — add half a least-significant bit before shifting — costs one adder and removes the bias.

DITHER: DELIBERATELY ADDING NOISE TO GET A BETTER ANSWER

Rounding the same way every time creates a *pattern*, and patterned error is not noise — it lands on discrete frequencies as spurs. Adding a tiny random offset before rounding (**dither**) breaks the pattern: the noise floor rises slightly, but the worst spur falls, which is usually the number you actually care about.

A MEASUREMENT TRAP I WALKED INTO ON THIS BOARD

Quantisation error only behaves like random noise if it is *uncorrelated* with the signal. Test with a tone at a simple fraction of the sample rate — $f_s/8$, say — and the error repeats in lockstep with the signal, concentrating into harmonics instead of spreading out. Spur-free dynamic range then measures far worse than the hardware deserves.

My own full-rate sweep used a tone at exactly $f_s/8$ and reported SFDR of 42–48 dB. That number is pessimistic, and the cause is the choice of test tone, not the radio. Use an awkward frequency with no simple relationship to the clock — a prime number of hertz is the usual trick.

Check yourself: your filter output is 32 bits wide and the DAC wants 16. Which 16 do you take?

Not the bottom 16 — those are the fractional part, and you would throw away the entire signal while keeping the noise. Not blindly the top 16 either, unless you know the value genuinely uses that range.

Work out where the binary point sits after the multiply-accumulate, choose the window that covers your actual signal range with headroom for peaks, **round** rather than truncate at that point, and **saturate** rather than wrap. Then verify in simulation with a full-scale input, because that is the case that overflows.

10 Testbenches, and why you will not skip them

One second against twenty minutes. This lesson decides whether learning this board is pleasant or miserable.

A **testbench** is a Verilog module with no inputs or outputs whose job is to wiggle the inputs of your design, watch its outputs, and complain if they are wrong. It is never synthesised — it exists only inside the simulator, so it may use conveniences real hardware cannot.

tb_counter.v

VERILOG

```
module tb;
    reg clk = 0, rst = 1;
    wire [31:0] count;

    counter dut (.clk(clk), .rst(rst), .count(count));    // dut = design under test

    always #5 clk = ~clk;    // flip every 5 time units -> a clock

    initial begin
        $dumpfile("tb.vcd"); $dumpvars(0, tb);    // record every signal

        repeat (2) @(posedge clk);
        rst = 0;
        repeat (10) @(posedge clk);

        if (count !== 32'd10) begin
            $display("FAIL: count=%0d, expected 10", count);
            $fatal;
        end
        $display("PASS");
        $finish;
    end
endmodule
```

terminal

SHELL

```
iverilog -o tb.out tb_counter.v counter.v && vvp tb.out
gtkwave tb.vcd    # when the numbers alone do not explain it
```

SIMULATOR VOCABULARY

- #5** Wait 5 simulation time units. Meaningless in synthesis; essential here.
- initial** A block that runs once at the start. Testbench-only.
- !==** Compares including the unknown state `x`. Prefer it to `!=` in checks — it catches uninitialised signals instead of quietly passing.
- VCD** Value Change Dump: a recording of every signal over time, viewable as waveforms in GTKWave. Indispensable when a test fails and the reason is not obvious.

The mutation check

A test that always passes is worse than no test, because it buys false confidence. This repo guards against that:

terminal

SHELL

```
./devkit sim --mutate  # deliberately corrupt the design; the test MUST now fail
```

If your test still passes with a broken design, your test is not testing. Write the testbench *before* the module, describing what you want to be true; then the first run means something and you have a golden reference to check the hardware against later.

11 What an IQ sample actually is

Every number flowing through this board comes in pairs. Here is why one number would not be enough.

Suppose you sample a radio wave and get the value 0.5. Is the wave rising or falling? You cannot tell. One number gives you amplitude at an instant and nothing about where in its cycle the wave is — and phase is where much of the information lives.

So SDRs take **two** measurements at each instant, from the same signal, using reference oscillators a quarter-cycle (90°) apart:

- I **In-phase** — measured against a cosine reference.
- Q **Quadrature** — measured against a sine reference, 90° later.

Together they pin down both amplitude and phase. If you think of the pair as a point on a plane — I across, Q up — then the distance from the origin is the signal's amplitude, and the angle is its phase. A steady tone traces a circle at constant speed; the speed of rotation is its frequency offset from the tuned centre.

WHY THIS MAKES NEGATIVE FREQUENCY MEANINGFUL

With I and Q you can tell a signal 1 MHz *above* your tuned frequency from one 1 MHz *below* it: one rotates clockwise, the other anticlockwise. With a single real-valued stream those two are indistinguishable. This is exactly why a spectrum on this board runs from negative to positive frequency either side of centre.

How it is carried here

Each sample is two signed integers. On receive both are 12-bit values sign-extended into 16-bit containers; on transmit both are full 16-bit. The stream is interleaved — I, Q, I, Q — so one "sample" costs four bytes. That is why a 5 MSPS stream is 20 MB/s in each direction.

TWO MORE TERMS YOU WILL MEET

- Complex baseband** The name for this I/Q representation, centred on zero. The radio has already removed the carrier frequency; what remains is the signal's shape around it.
- Mixing** Multiplying by an oscillator to shift a signal up or down in frequency. Moving from the antenna's gigahertz down to baseband is mixing, and so is any frequency shift you do yourself in the fabric (lesson 28).

12 Sample rate, Nyquist and aliasing

The one piece of theory you cannot skip, because violating it produces signals that are not there.

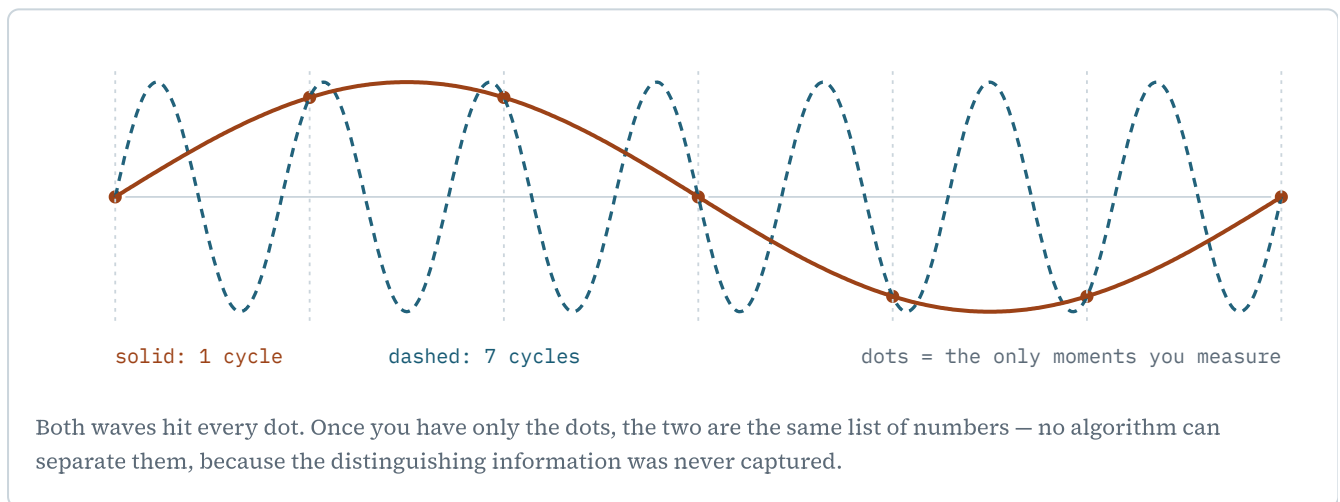
The **sample rate** is how many samples per second you take. This board goes from 2.083 to 61.44 million samples per second (MSPS).

The **Nyquist theorem** says: sampling at rate f can faithfully represent signals occupying a bandwidth of f . For complex I/Q sampling that bandwidth is centred on the tuned frequency and runs from $-f/2$ to $+f/2$. At 5 MSPS you see 5 MHz of spectrum, from 2.5 MHz below your tuned frequency to 2.5 MHz above.

Aliasing: why a signal can appear where it is not

Sampling does not watch the signal. It *glances* at it, at evenly spaced instants, and writes down what it saw. Everything between those glances is simply not recorded.

That creates an ambiguity you cannot argue your way out of. Two different sine waves can pass through **exactly the same points** at exactly those instants:



So when a signal arrives that is too fast for your sample rate, it does not go missing and it does not announce itself. It is **recorded as a slower one**, and from that moment it is indistinguishable from a genuine signal at that lower frequency. This is **aliasing**: the fast signal wears the alias of a slow one.

YOU HAVE ALREADY SEEN THIS HAPPEN

In films, wagon wheels and helicopter rotors sometimes appear to turn slowly backwards. The camera samples 24 times a second. If a spoke moves *slightly less* than one full spoke-spacing between frames, each frame catches it a little short of where it started — and your eye reads that as slow backward rotation. The wheel is not going backwards. The sampling is too slow to tell, so the motion is recorded as a different motion entirely. That is aliasing, exactly, and it is the same arithmetic.

Where a given signal lands

The rule is simple: **add or subtract whole multiples of the sample rate until the frequency falls inside the window**. Wherever it lands is where it will appear.

At 5 MSPS the window runs from -2.5 to $+2.5$ MHz, so:

A REAL SIGNAL AT...	ARITHMETIC	...APPEARS AT
+1.0 MHz	already inside	+1.0 MHz — correct
+3.0 MHz	$3 - 5$	-2.0 MHz
+7.0 MHz	$7 - 5$	+2.0 MHz
-4.0 MHz	$-4 + 5$	+1.0 MHz

Note the third and fourth rows. A signal 7 MHz above where you tuned, and one 4 MHz below it, both land on top of perfectly ordinary places in your spectrum. Nothing about the captured data marks them as impostors.

WHY THE WINDOW IS $\pm FS/2$ HERE

The short version Because this board samples I and Q (lesson 11), it can tell a signal above the tuned frequency from one below it. That buys you the full sample rate as usable width, centred on where you tuned — $-fs/2$ to $+fs/2$. A radio that sampled a single real-valued stream would get only half that, which is the form of Nyquist's rule most textbooks state first.

Check yourself: at 20 MSPS, where does a signal 12 MHz above centre appear?

The window is ± 10 MHz, so 12 MHz is outside it. Subtract the sample rate: $12 - 20 = -8$ MHz. It will appear 8 MHz *below* centre, looking exactly like a genuine signal there.

Try it in the calculator above — then try 28 MHz, which lands in the same place.

The only real defence: filter before you sample

Once a signal has been sampled, the damage is permanent — the alias *is* the data. So the fix has to happen while the signal is still analogue, on the way in. That is what `rf_bandwidth` controls: a filter inside the AD9361, ahead of its converters, that attenuates anything far enough from centre to fold.

terminal

SHELL

```
# run on your HOST - set the rate, then match the filter to it
iio_attr -u ip:192.168.2.1 -i -c ad9361-phy voltage0 sampling_frequency 5000000
iio_attr -u ip:192.168.2.1 -i -c ad9361-phy voltage0 rf_bandwidth 4000000
```

A sensible default is a filter **slightly narrower** than the sample rate — the measurements in this repo use $0.8 \times$ the rate. Set it much *wider* and you are deliberately letting in signals that can only arrive as aliases. Set it much *narrower* and you are throwing away spectrum you paid to sample.

HOW TO CATCH AN ALIAS IN THE FIELD

You suspect a peak in your spectrum is not real. **Change the sample rate and look again.**

A genuine signal sits at a fixed radio frequency, so it stays where it is. An alias is an arithmetic accident of the old rate, so it *jumps* to a different place — or vanishes. That one test settles it in seconds and needs no extra equipment.

What this costs you in practice

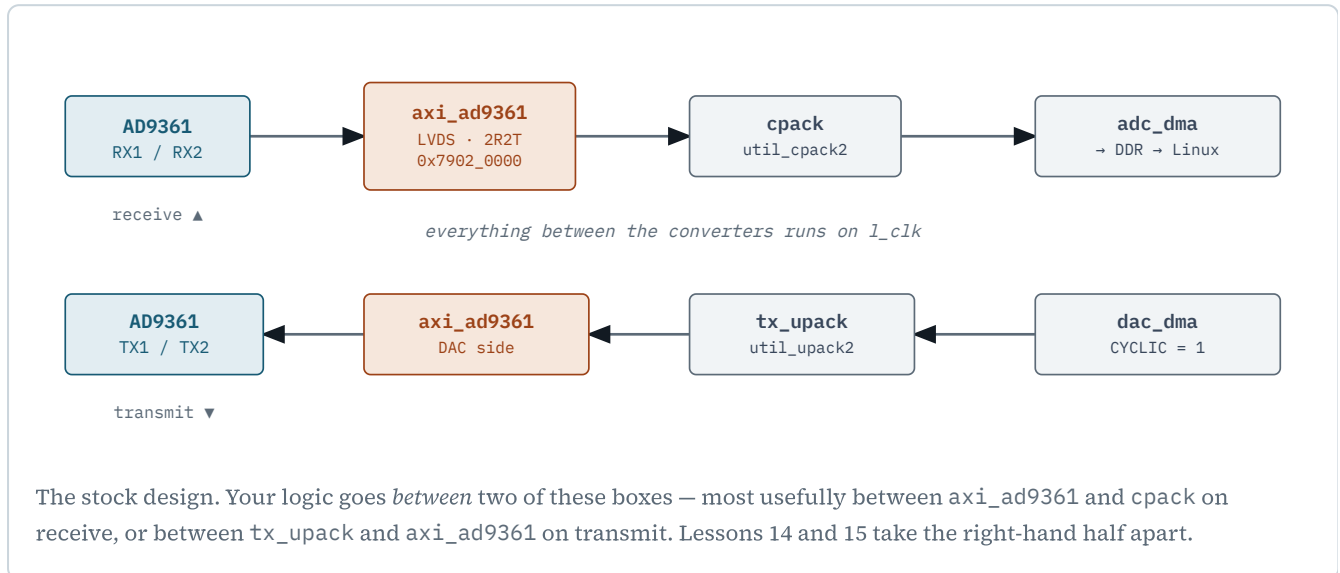
Higher sample rate means more bandwidth, and proportionally more data. At the top of this board's range, 61.44 MSPS, each direction is 245 MB/s.

Measured on this board, capture alone runs at the full 61.44 MSPS over gigabit Ethernet, with only a handful of dropped samples per two million. So getting the data *off* the board is not the wall. The walls are elsewhere: transmit and receive streaming simultaneously starve above about 5 MSPS, and a modest CPU cannot do much useful work on 245 MB/s in real time even once it has it.

Which raises the obvious question — if you only care about a 200 kHz signal, why sample 56 MHz at all? That question has a good answer, and it is [lesson 27](#).

13 The block design: the map, and what it quietly assumes

You are not building a design. You are joining one that already works — so the useful knowledge is not "what does each block do" but "where can I cut in, and what will bite me".



It is a script, not a drawing

Vivado shows the block design as a diagram you can drag boxes around in. On this board that diagram is **generated**, every build, by `firmware/src/hdl/projects/pluto/system_bd.tcl`. Nothing is stored as a picture.

That has three consequences worth internalising:

- **Your change is a diff.** Someone can review it, CI can check it, and git can tell you what moved. A dragged wire in a GUI is none of those things.
- **You cannot accidentally change it.** Clicking in the diagram edits a project that will be regenerated and discarded.
- **The project is a cache, and a treacherous one.** The build reuses an existing project rather than re-running the script — which is exactly the trap in lesson 20.

How software reaches into the fabric at all

Before the table makes sense, one idea has to land: **memory-mapped I/O**.

The processor has a single enormous range of numbered addresses. Most of those numbers are ordinary memory — the DDR chips, where writing a value stores it and reading gets it back. But some of the numbers are not memory at all. They are **wired to hardware**.

Write to one of those addresses and nothing is stored; instead a register inside a block in the fabric changes, and the hardware behaves differently from that instant. Read from it and you are not retrieving something you saved; you are *asking the hardware what it currently is*.

THE PICTURE THAT MAKES IT STICK

Imagine a building where every room has a number. Most rooms are offices with filing cabinets — leave a document, come back, it is still there. But a handful of rooms, using the very same numbering, are control panels for the building's machinery. Walk into room 31,744 and flip a switch and the ventilation changes.

Nothing in the number tells you which kind of room it is. You have to be told — and being told is exactly what the address map and the device tree are for.

This is why the processor and the fabric need no special "send command" mechanism. The processor already knows how to write to an address. The block design's job is to decide *which addresses land on which blocks*.

Reading the addresses

THE NOTATION

0x	Marks a hexadecimal number — base 16, digits 0–9 then A–F. Hardware addresses are written this way because each hex digit is exactly four binary bits, so the digits line up with the chip's real address wires. In decimal they would be unreadable and would line up with nothing.
–	Just a separator for human eyes, like a comma in 1,000,000. It has no meaning. <code>0x7C40_0000</code> and <code>0x7C400000</code> are the same number.
Base address	Where a block's range <i>starts</i> . Each block owns a contiguous run of addresses from there.
Offset	How far into that range a particular register sits. A register at offset <code>0xBC</code> in a block based at <code>0x7902_4000</code> really lives at <code>0x7902_40BC</code> .
IRQ	Interrupt request — a wire from the block back to the processor meaning "stop what you are doing, something happened". <code>ps-13</code> means the thirteenth of the processor's fabric interrupt inputs. Without it a driver would have to keep asking "are you finished yet?", wasting the CPU it was trying to save.

This board's map

BASE ADDRESS	BLOCK IN THE FABRIC	LINUX CALLS IT	INTERRUPT
<code>0x4160_0000</code>	<code>axi_iic_main</code> — an I ² C bus master	<code>axi_iic</code>	ps-15
<code>0x7902_0000</code>	<code>axi_ad9361</code> — the radio interface	<code>cf-ad9361-lpc</code> (receive)	—
<code>0x7902_4000</code>	<i>the same block</i> , 0x4000 further in	<code>cf-ad9361-dds-core-lpc</code> (transmit)	—
<code>0x7C40_0000</code>	receive DMA engine	<code>dma@7c400000</code>	ps-13
<code>0x7C42_0000</code>	transmit DMA engine	<code>dma@7c420000</code>	ps-12
<code>0x7C43_0000</code>	<code>axi_spi</code> — an SPI bus master	<code>axi_quad_spi</code>	ps-11

Two rows there are worth pausing on. **One physical block appears to Linux as two devices**, because its receive registers start at the base and its transmit registers start 0x4000 bytes further in. That single fact is why this board has both `cf-ad9361-lpc` and `cf-ad9361-dds-core-lpc` — they are two windows onto one piece of hardware.

Notice also that the DMA engines have interrupts and the radio interface does not. The DMAs need to announce "a buffer is finished"; the radio interface only ever answers questions.

YOU HAVE ALREADY USED THIS WITHOUT KNOWING

Every register poke you have seen in this course is an address in that table plus an offset:

terminal

SHELL

```
iio_attr -u ip:192.168.2.1 -D cf-ad9361-dds-core-lpc direct_reg_access 0xBC
```

`cf-ad9361-dds-core-lpc` selects the base `0x7902_4000`, and `0xBC` is the offset within it — so that command reads the physical address `0x7902_40BC`. That is the register whose bit 1 switches on the sample-locked GPIO feature. The name is just a friendlier way of saying a number.

Why the map is a contract, not a note

Nothing discovers any of this. There is no scan, no plug-and-play. Linux is **told**, by a file called the **device tree** — `zynq-pluto-sdr-fishball.dts` — which lists every block, its address, its interrupt and its settings.

So the same addresses are written down in *two independent places*: the block design, which decides where the hardware actually answers, and the device tree, which tells the driver where to look. Nothing checks that they agree.

WHAT A DISAGREEMENT LOOKS LIKE

Move a block in `system_bd.tcl` and forget the device tree, and the driver reads an address where nothing lives. On this bus that does not raise an error — it returns zeros, or whatever the bus fabric happens to present.

The symptom is therefore not a crash and not a message. It is a device that *almost* works, or does not appear in `iio_info` at all, with a boot log that says nothing useful. If you ever change an address, change both files in the same commit.

Three things that are not where you would guess

THE RADIO IS NOT CONTROLLED THROUGH THE SPI BLOCK SITTING RIGHT NEXT TO IT

There is an `axi_spi` master in the design. It is **not** on the AD9361's control path. The radio is configured over the *processor's own* SPI0, routed out to fabric pins through EMIO — a completely separate bus on completely separate balls (`R17` , `V18` , `P16` , `V17`).

So if you are tracing how a setting reaches the chip, do not follow `axi_spi` . It goes nowhere near.

SO WHY IS `AXI_SPI` THERE AT ALL? CHECK ITS CHIP SELECT.

Because the block design is inherited. ADI maintain one reference design across a whole family of boards — FMCOMMS cards, ADRV modules, the Pluto — and on some of those, an I²C master and a fabric SPI master drive real peripherals: clock generators, EEPROMs, attenuators. This board keeps the scaffolding whether or not it uses it.

How vestigial is it here? Look at how it is wired up in `system_top.v` :

```
firmware/src/hdl/projects/pluto/system_top.v VERILOG

.spi_clk_o (pl_spi_clk_o),    // -> ball L14
.spi_sdo_o (pl_spi_mosi),    // -> ball N16
.spi_sdi_i (pl_spi_miso),    // <- ball N15
.spi_csn_i (1'b1),          // tied inactive
.spi_csn_o (),              // <-- THE CHIP SELECT GOES NOWHERE
```

Clock, data-out and data-in reach real pins. The **chip-select output is left unconnected** — an empty pair of brackets. It is never brought to a ball and never constrained.

A SPI master with no chip select cannot *address* anything: chip select is how a master says "this message is for you". So as built, this block is a bus that can talk but cannot choose a listener. It is not a peripheral driver on this board; it is leftovers.

WHAT THAT MEANS IF YOU WANT TO USE IT

It is genuinely available — you have an AXI-attached SPI master, its Linux driver (`axi_quad_spi`), its address and its interrupt all already in place. But you cannot just wire up a chip and go. You would first have to **bring the chip select out**: connect `spi_csn_o` to a port, give that port a pin in the constraints file, and add the device to the device tree.

The I²C master is in better shape — `iic_scl` and `iic_sda` reach balls `M14` and `M15` with pull-ups, and I²C needs no chip select because devices are addressed in the protocol itself.

Both are worth knowing about if you ever want to control something *other* than the radio from this board.

TWELVE OF SIXTEEN INTERRUPT LINES ARE FREE

An `xlconcat` gathers sixteen interrupt lines into the processor's `IRQ_F2P` port. Only four are used; the rest are tied to ground. If your logic needs to tell the processor something happened — a correlator fired, a threshold was crossed — the wiring is already there and you are not competing for it.

FOUR OF THE TWENTY-TWO EMIO GPIOS ARE THIS REPOSITORY'S

The processor exports 22 general-purpose pins into the fabric. Eighteen are ADI's stock design; the other four were added here to carry the sample-locked header pins. That is the cheapest way to get a signal between the processor and the fabric — no AXI block, no address, no device-tree node. Worth remembering when you need one bit rather than a bus.

What not to touch, and why

THING	WHAT HAPPENS IF YOU DO
The DDR timing parameters	Board-specific, tuned for this PCB's memory chips. Change them and it will not boot .
<code>axi_ad9361/ID</code>	The Linux driver gates its capture setup on <code>ID == 0</code> . Set it to anything else and capture <i>silently</i> stops working.
<code>CMOS_OR_LVDS_N</code>	The board is physically wired for LVDS. This is not a preference.
An AXI base address, alone	The device tree still points at the old one.
The filters' $\div 8 / \times 8$ rate	The driver offers exactly <code>{1, 8}</code> . A $\div 4$ filter would build perfectly and be unreachable from software — a whole build spent on something you cannot switch on.

READ THE REAL THING

Open `firmware/src/hdl/projects/pluto/system_bd.tcl` and search for `ad_connect`. Every wire in the diagram above is one of those lines. It is far shorter than you expect, and once you have seen that the whole radio datapath is a few dozen readable connections, changing it stops feeling dangerous.

14 Clocks, valid strobes, and the 2R2T trap

Three facts about this specific board that will otherwise cost you a day each. Everything here is defined from scratch — none of it is assumed.

First: what a clock domain is

A **clock domain** is simply *the set of flip-flops driven by one particular clock signal*. Logic inside a single domain is easy to reason about: every register captures at the same instant, so signals move in lockstep.

Trouble starts when a signal crosses *between* domains, because two unrelated clocks have no fixed relationship — their edges drift past each other. This board has three domains, and lesson 20 is entirely about crossing safely between them. For now you only need to know they exist and which one your logic lives in.

CLOCK	WHERE IT COMES FROM	RATE	WHAT IT DRIVES
<code>l_clk</code>	Recovered from the AD9361's own data clock	Scales with sample rate	The whole datapath — filters, packers, the fabric side of both DMAs, and anything you add. This is your clock.
<code>sys_cpu_clk</code>	<code>FCLK_CLK0</code> from the processor	100 MHz, fixed	The control-register bus, and the memory side of both DMAs.
<code>sys_200m_clk</code>	<code>FCLK_CLK1</code> from the processor	200 MHz, fixed	One thing only: a timing reference for the high-speed link to the radio chip. Not for your logic.

TWO TERMS FROM THAT TABLE

- Recovered clock** The AD9361 sends its data along with a clock that marks when each piece is valid. The fabric does not generate this clock — it extracts it from the incoming signal and then runs on it. That is why it changes when you change the sample rate: it is the radio's own timing, not something the FPGA chose.
- LVDS** Low-Voltage Differential Signalling — the electrical standard the chip-to-fabric link uses. Each bit travels as the difference between two wires, which is fast and resistant to noise. It matters here only because it is *fast and serial*: few wires carrying many bits, one after another.

`L_CLK` IS NOT A FIXED FREQUENCY

At the AD9361's slowest rate it is around 4 MHz; at 30.72 MSPS it is 61.44 MHz. Software can change the sample rate at any moment and your clock changes underneath you. Never write logic that assumes a number of clock cycles per second — no "wait 1000 cycles for one millisecond". Design for the top end, and count events, not time.

Second: what a strobe is, and why a clock is not one

The word comes from *stroboscope* — the lamp that flashes for an instant to freeze a spinning object. A **strobe** in digital hardware is the same idea: a brief pulse that says "**now**".

That is worth separating from the other kind of signal you meet:

	A LEVEL	A STROBE
Says	"this is the <i>state</i> of things"	"this <i>event</i> is happening, right now"
Shape	Stays high or low for as long as it applies	High for exactly one clock cycle , then low again
Example here	<code>flag</code> — the bit-map feature is on	<code>fifo_wr_en</code> — a sample word is being written this cycle
You use it to	Choose behaviour	Trigger an action, once

So why not just use the clock?

This is the question worth sitting with, because the clock also pulses, constantly and reliably. Why is it not enough?

Because the clock is always running, and the data is not always new. The clock ticks whether or not anything happened. It is the heartbeat of the circuit, not a statement about the wires beside it.

Data, meanwhile, arrives irregularly. On this board there are at least three reasons a given clock tick might carry nothing you want:

- **The channels take turns.** In 2R2T, half the ticks belong to the other channel (the next section).
- **A filter is decimating.** With the $\div 8$ filter on, seven ticks in eight produce no output at all — the filter is still accumulating.
- **The data ran out.** On transmit, if software could not keep up, the hardware is presenting substituted zeros rather than your samples.

Between real samples, the wires do not go blank. They keep showing *the last value*, because that is what a register does — it holds. So logic that acts on every clock edge will happily process the same sample eight times in a row and never know.

WHAT THAT MISTAKE ACTUALLY PRODUCES

Suppose you are averaging 1000 samples to measure power, and you accumulate on every clock edge instead of on the strobe. In 2R2T you will add each real sample twice, so you reach 1000 additions after only 500 samples. Your average is computed over half the time window you intended — and the number it produces looks perfectly reasonable.

That is the character of strobe bugs: not crashes, not error messages. Plausible, wrong numbers.

So the strobe carries the one piece of information the clock cannot: *is the thing beside me worth looking at this time?* It is the difference between "a moment passed" and "something happened".

the pattern, every time

VERILOG

```
always @(posedge clk) begin           // clk decides WHEN you may act
  if (valid_in)                       // the strobe decides WHETHER you should
    accumulator <= accumulator + sample_in;
end
```

Read those two lines as a pair, because that is the whole convention: **the clock grants permission to act; the strobe supplies the reason**. Every block in this design's datapath is built that way, which is why they can be chained together at all — each one only moves when the one before it says there is something to move.

THE STROBES YOU WILL MEET, AND WHAT EACH IS ANNOUNCING

Valid	"The data on these wires is real this cycle." The most common one, and the default meaning if someone just says "strobe".
Write enable	"Store this, now." <code>cpack/fifo_wr_en</code> is one.
Read enable	"Give me the next one." A <i>request</i> — and therefore not the same as the data arriving, which is a trap you meet at the end of this lesson.
Underflow	"I had nothing to give, so this is a substitute." Announces an event you would otherwise never find out about.

Check yourself: why does a strobe last exactly one cycle, rather than staying high while data is good?

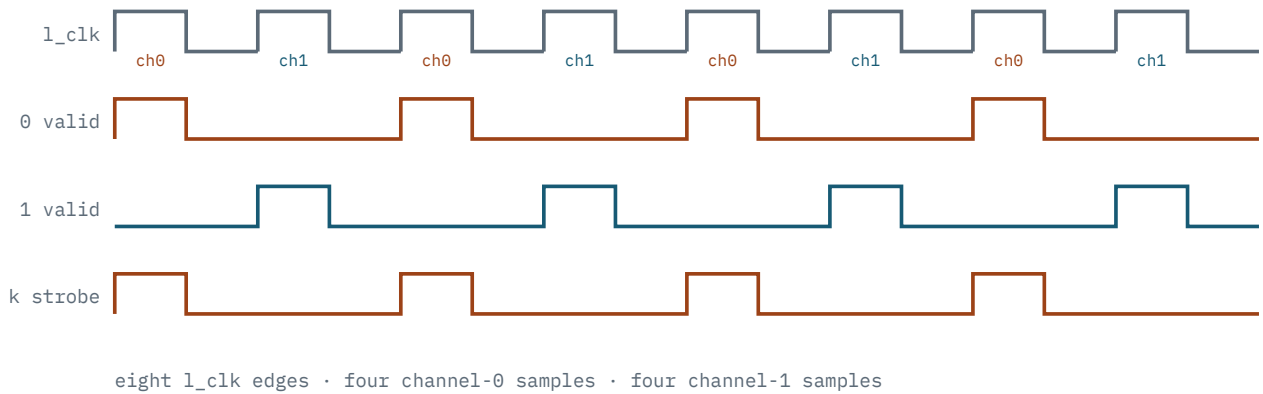
Because it marks an *event*, and events are counted. If a strobe stayed high for three cycles, logic downstream would act three times on one sample — it has no way to tell a long pulse from three short ones.

Anything that genuinely is a lasting condition is a level instead, like the `flag` that switches the bit-map feature on. The two kinds are not interchangeable, and mixing them up is a classic source of counts that are off by a factor.

Third: 2R2T, where clock edges stop meaning samples

This board runs the AD9361 with **two receivers and two transmitters** — "2R2T". But there is only *one* physical link between the radio chip and the fabric. Two channels, one set of wires.

So the two channels take turns on it. That is all **time-multiplexed** means: sharing one path by alternating who gets to use it. One clock period carries a channel-0 sample, the next carries a channel-1 sample, then channel 0 again, forever.



One channel's samples arrive on alternate clock periods, not every period. The strobe that drives cpack follows channel 0, so it too pulses only every second edge.

The consequence: a clock edge is *not* a sample. There are twice as many clock edges as there are samples of any one channel. At 30.72 MSPS, `l_clk` runs at 61.44 MHz.

WHAT GOES WRONG, CONCRETELY

Say you write a counter that increments every clock edge, intending to count samples, and you put its bottom bit on a header pin to use as a scope trigger.

You expect a pulse every sample. You get a pulse every *half* sample — the pin toggles at twice the rate you designed for, and the trigger lines up with nothing. Every pattern you author comes out doubled, and because the waveform still looks plausible it can take a long time to notice.

The fix is one word: gate on the valid strobe.

the 2R2T rule

VERILOG

```
if (valid_in) count <= count + 1'b1; // counts SAMPLES
// count <= count + 1'b1;           // counts CLOCK EDGES - twice as fast
```

Check yourself: at 15 MSPS, how fast does `l_clk` run, and how many edges per channel-0 sample?

`l_clk` runs at 30 MHz — twice the sample rate, because the two channels share one link. There are **two clock edges per channel-0 sample**: one carrying channel 0, the next carrying channel 1.

Which is why counting edges gives you twice the answer you wanted.

The strobes on this board

SIGNAL	WHAT IT MEANS
<code>cpack/fifo_wr_en</code>	Receive. A sample word is being written towards memory. This is your tap point on the receive side.
<code>tx_upack/fifo_rd_valid</code>	Transmit. A real sample is standing at the output <i>this cycle</i> .
<code>tx_upack/fifo_rd_underflow</code>	Transmit. Zeros are being substituted because the data ran out — software could not keep up.
<code>tx_upack/fifo_rd_en</code>	The <i>request</i> for a sample. Not the arrival. See the trap below.

A REQUEST IS NOT AN ARRIVAL

`fifo_rd_en` means "send me a sample". The packer registers its output — it stores the value in a flip-flop before presenting it — so the word you asked for does not appear until **the following clock cycle**.

Capture on `fifo_rd_en` and you will therefore latch the *previous* sample, permanently one behind the transmitter. Nothing breaks, nothing warns you, and the result looks almost right.

Use `fifo_rd_valid | fifo_rd_underflow` instead. That pair goes high *with* the data, and between them they cover both real samples and the zeros substituted on underflow — so there are no exceptions to remember.

15 The packers, and the filter only channel 0 has

Two blocks sit between the radio and the memory system. One of them is wired lopsidedly, and that lopsidedness will quietly ruin your second receiver if nobody warns you.

Why anything sits there at all

Think of the radio as two people talking, and the memory system as a single notepad. The radio produces two separate streams of numbers, one per receiver, arriving whenever the converter feels like producing them. Memory wants one wide, tidy, regular stream.

Something has to sit in the middle and reconcile them. That is all the packers are.

	WHAT THE RADIO GIVES	WHAT MEMORY WANTS
Shape	Separate streams, one per channel	One combined stream
Width	16 bits at a time	64 bits at a time
Timing	Whenever a sample is ready	In bursts, when the bus is free
Channels	One or two, your choice at runtime	Does not know channels exist

The receive one is called `cpack` — short for "channel pack". The transmit one is `tx_unpack`, "unpack", and does the reverse.

THREE WORDS USED BELOW

- FIFO** First In, First Out — a queue. A small on-chip buffer that lets one side push data in at its own pace while the other pulls it out at a different pace.
- Multiplexer** Usually shortened to *mux*. A switch: several inputs, one output, and a control signal choosing which input gets through. The hardware equivalent of `if`.
- Hierarchy** In a block design, a box that contains other boxes — a folder. It appears as one block in the diagram but several blocks exist inside it.

What "packing" actually does

Suppose you asked for one receiver. Each 64-bit word then holds four 16-bit samples of it, back to back. Now ask for two receivers: each word holds two samples of each, interleaved.

The packer rearranges this *on the fly*, because which channels are switched on is decided by software at the moment it opens a buffer. The result is that memory is always densely filled — no padding, no gaps, no bandwidth spent on a channel you did not ask for.

THIS IS WHY YOUR DATA NEEDS NO UNPICKING

When you read from one channel you get `I, Q, I, Q...` and nothing else. Ask for two and you get `I0, Q0, I1, Q1...`. There is no header, no channel tag, nothing to strip out. The packers did that work in hardware, at full sample rate, for free.

Then how does your program know which sample is which?

This is the obvious objection, and it has a satisfying answer: **the layout is decided by something your program already set**. The metadata is in the request, not in the data.

Before opening a buffer, an application *enables* the channels it wants. That enable mask completely determines what comes back — the hardware packs enabled channels in a fixed order, and that order is the channels' **scan index**, always ascending.

On this board's receive core the four scan indices are:

SCAN INDEX	LIBIIO NAME	WHAT IT CARRIES
0	voltage0	Receiver 1, I
1	voltage1	Receiver 1, Q
2	voltage2	Receiver 2, I
3	voltage3	Receiver 2, Q

So the stream you get is a direct consequence of what you asked for:

YOU ENABLE	YOU RECEIVE, REPEATING	BYTES PER SAMPLE INSTANT
voltage0, voltage1	I1 Q1 I1 Q1 ...	4
voltage2, voltage3	I2 Q2 I2 Q2 ...	4
all four	I1 Q1 I2 Q2 I1 Q1 I2 Q2 ...	8

Nothing in the buffer identifies a sample. Its identity *is its position*, and position is knowable because you chose the mask.

YOU HAVE BEEN RELYING ON THIS ALL ALONG

Every capture command in this course names its channels explicitly:

```
terminal SHELL

# receiver 1 only - 4 bytes per sample instant
iio_readdev -u ip:192.168.2.1 -b 65536 -s 1048576 cf-ad9361-lpc voltage0 voltage1

# both receivers - 8 bytes per sample instant
iio_readdev -u ip:192.168.2.1 -b 65536 -s 1048576 cf-ad9361-lpc voltage0 voltage1 voltage2
voltage3
```

The channel names at the end are not decoration. They *are* the mask, and they are why you can interpret the bytes that come back.

ENABLE ORDER DOES NOT CHANGE PACK ORDER

Ask for `voltage3` before `voltage0` and the data still arrives in scan index order — `voltage0` first. The hardware packs by index, not by the order you happened to mention them.

So do not infer the layout from your own argument order. If you are unsure, ask the library:

`iio_buffer_first()` returns the address of a given channel's first sample in the buffer, and `iio_buffer_step()` the distance to its next one. Using those two, your de-interleaving code stays correct even if the mask changes.

GETTING THE MASK WRONG IS SILENT

If you enable four channels but de-interleave as though there were two, every other sample pair lands in the wrong stream. You do not get an error — you get two signals that look like noise, or a constellation that seems to have twice as many points as it should.

At the wire-protocol level the mask is a fixed-width hex field: `00000003` enables channels 0 and 1, `0000000F` all four. Writing `3` instead of `00000003` fails with `-22 EINVAL` and no explanation.

Check yourself: you enable only channel 1. What does a 64-bit word contain?

Four 16-bit samples of channel 1 — two complete I/Q pairs — and nothing belonging to channel 0. "Enabled" decides what gets packed, so a disabled channel costs no memory and no bandwidth.

The strobe that moves each word, and the lopsided bit

Each packer has one signal that says "move a word now". Where that signal comes from is the whole story of this lesson:

DIRECTION	THE STROBE	COMES FROM
Receive	<code>cpack/fifo_wr_en</code>	Channel 0's valid, alone.
Transmit	<code>tx_upack/fifo_rd_en</code>	Channel 0's valid OR channel 1's.

On receive, channel 1's own valid signal — `adc_valid_i1` — is **connected to nothing at all**. Every channel is captured on channel 0's timing. Remember that for two paragraphs.

The filter that exists on one channel only

Between the radio interface and `cpack`, channel 0 passes through a block called `rx_fir_decimator`. Channel 1 does not — it goes straight past.

That block is a hierarchy containing:

- **Two filters** — one for I, one for Q, since they must be filtered identically.
- **A clock-domain crossing** for the on/off signal (lesson 21's problem, already solved for you here).
- **Bypass multiplexers** — the switch that decides whether samples go through the filters or straight past them.

What is it *for*? The chip has a floor

A fair question, given the trouble it causes: why did ADI put a decimating filter in the fabric at all, when the AD9361 already has perfectly good filters of its own?

Because the AD9361 cannot go slow enough. Ask it:

```
terminal SHELL

iio_attr -u ip:192.168.2.1 -i -c ad9361-phy voltage0 sampling_frequency_available
# [2083333 1 61440000] <- minimum, step, maximum
```

2,083,333 samples per second is a hard floor in the silicon. The chip will not go below it. So without help, the narrowest slice of spectrum this radio can deliver is about 2.1 MHz wide — and an enormous number of interesting signals are far narrower than that:

SIGNAL	ROUGHLY HOW WIDE	FITS IN 2.08 MHZ?
Amateur SSB voice	3 kHz	700× too much spectrum
Narrowband IoT	180 kHz	11× too much
FM broadcast channel	200 kHz	10× too much
A GSM carrier	200 kHz	10× too much

For any of those you would be forced to capture 2.08 MSPS, ship it all to the host, and throw away 90% of it there — paying full bandwidth and full CPU for a sliver of signal.

The fabric decimator removes that floor. Measured on this board, with the converter set as slow as it will go:

	WITHOUT THE FILTER	WITH IT ENGAGED
Converter rate	2,083,333 SPS	2,083,333 SPS
Rate delivered to you	2,083,333 SPS	260,416 SPS
Data rate	8.33 MB/s	1.04 MB/s
Processing gain	—	9 dB (~1.5 bits)

260 kSPS is comfortably below anything the chip can reach alone, and it is exactly the region where FM, GSM and narrowband IoT live. That is what the block is for: **it extends the radio's usable range downward, past a limit built into the silicon.**

WHICH ALSO EXPLAINS THREE THINGS THAT OTHERWISE LOOK ARBITRARY

- **Why the factor is fixed at eight**, and why the driver offers only `{1, 8}`. It is not a general-purpose resampler. It is one specific extension of the rate range, and eight is enough to clear the gap between the chip's floor and the narrowband world.
- **Why there is a matching $\times 8$ interpolator on transmit**. The floor applies in both directions — you cannot *send* a 200 kHz-wide signal at 200 kSPS either.
- **Why both share one coefficient file**. They are the same filter doing the same job in opposite directions: one generic anti-alias design for a factor of eight.

AND WHY THE CHANNEL-1 PROBLEM WAS INVISIBLE TO WHOEVER DESIGNED IT

This same block design is used across a family of boards, and many of them run **1R1T** — one receiver, one transmitter. On those, there *is* no channel 1. A filter on "channel 0 only" is a filter on the only channel there is, and the asymmetry described below simply does not exist.

It becomes a defect only on a 2R2T board like this one, where a second receiver is sitting there quietly being sampled on the first one's timing with no filter of its own. Inherited designs carry inherited assumptions, and this is what one looks like.

Why it is a filter *and* a $\div 8$, never just a $\div 8$

"Decimate by eight" sounds like it should be trivial: keep every eighth sample, throw the other seven away. Why drag a 129-tap filter and a coefficient file into it?

Because throwing samples away is **undersampling**, and you already know what undersampling does (lesson 12). Take 61.44 MSPS and keep one sample in eight and you now have a 7.68 MSPS stream — whose window is only ± 3.84 MHz. Everything that was outside that window does not disappear. It *folds in*.

THE PICTURE: EIGHT STACKED COPIES

Discarding seven of every eight samples takes the entire 61.44 MHz of captured spectrum and folds it into 7.68 MHz — eight slices piled on top of one another, summed together, with no way to tell them apart afterwards.

Any transmitter anywhere in those other seven slices lands directly on top of your signal. And it is not only interference: the *noise* from all eight slices adds up too, so the result is roughly eight times the noise power in the same bandwidth.

The FIR filter's job is to empty those seven slices out **before** the samples are discarded. It passes what is inside ± 3.84 MHz and attenuates everything beyond it, so that when the folding happens there is almost nothing left to fold. That is why this kind of filter is called an **anti-alias filter** — it does not "improve" the signal, it removes what would otherwise arrive uninvited.

The order is not negotiable, and it is what the word means:

OPERATION	WHAT IT IS	RESULT
Downsampling	Keep one sample in D. Nothing else.	Aliases. Almost never what you want alone.
Decimation	Filter <i>first</i> , then keep one sample in D.	A clean, narrower stream.

AND THIS IS WHERE THE FREE DYNAMIC RANGE ACTUALLY COMES FROM

Lesson 26 claims decimating by D buys you $10 \cdot \log_{10}(D)$ decibels of signal-to-noise. Now you can see *why*, and why it is not magic.

The converter's noise is spread across the whole captured bandwidth. The filter discards the noise living in the seven slices you are not keeping, and the discard step then removes those slices without letting their noise back in. Signal preserved, seven eighths of the noise gone.

Remove the filter and the gain vanishes completely — all that noise folds straight back on top of you. The processing gain and the anti-alias filter are not two features. They are the same fact seen from two directions.

So what is in the .coe file?

Just numbers — one tap value per line, 129 of them. That list *is* the filter: it decides exactly where the passband ends, how steeply the response falls, and how far down the stopband sits.

That last figure is the one that matters here. If the stopband is 80 dB down, whatever folds in arrives 80 dB weaker than it was. If a sloppy design only reaches 40 dB, a strong neighbouring transmitter can still land on your signal at a level you will notice. **The coefficients set how much aliasing survives**, which is why they are a designed artefact and not an afterthought.

Check yourself: why can the AD9361's own analogue filter not do this job for you?

It does, for its own rate — that is exactly what `rf_bandwidth` sets. But the analogue filter protects the *converter's* sample rate, and here the converter is still running at full speed. The fabric is creating a *second, much narrower* rate downstream of it, and nothing in the chip knows about that.

Every point in a chain where the rate drops needs its own anti-alias filter at that point.

The one line of Tcl that builds it

The whole hierarchy comes from a single call in the block-design script:

```
firmware/src/hdl/projects/pluto/system_bd.tcl
```

TCL

```
ad_add_decimation_filter "rx_fir_decimator" 8 2 1 {61.44} {61.44} <coe>
```

Taken piece by piece, left to right:

PIECE	WHAT IT IS	WHAT HAPPENS IF YOU CHANGE IT
<code>ad_add_decimation_filter</code>	Not a block — a <i>helper procedure</i> , defined in <code>projects/common/xilinx/adi_fir_filter_bd.tcl</code> . Calling it builds the whole hierarchy: both filters, the crossing, and the bypass muxes.	There is a matching <code>ad_add_interpolation_filter</code> for the transmit side.
<code>"rx_fir_decimator"</code>	The name the hierarchy gets. This is the label you see in the Vivado diagram and the prefix on every signal inside it.	Rename it and every <code>ad_connect</code> referring to it must change too.
<code>8</code>	The decimation factor. Keep one sample in every eight.	Do not. The Linux driver offers exactly <code>{1, 8}</code> . A $\div 4$ filter would build perfectly and be unreachable from software.
<code>2</code>	Number of channels — two, because I and Q each need their own filter.	This is why the hierarchy contains two filters rather than one.
<code>1</code>	Parallel paths — how many samples arrive per clock tick. One, here.	Higher values are for designs where several samples arrive at once.
<code>{61.44}</code>	The clock rate in MHz the filter is told to expect, so the tool can budget how much hardware reuse is possible.	Getting this wrong makes the tool build the wrong amount of hardware.
<code>{61.44}</code>	The sample rate in MHz. Together with the clock rate it tells the tool how many clock ticks are available per sample — which is what makes taps cheap (lesson 26).	—
<code><coe></code>	The coefficient file : a plain text list of the filter's tap values, which is the filter's design.	Change these and you change what the filter does — but see the trap below.

How you switch it on

Each hierarchy has one pin called `active`. Low means "pass samples straight through"; high means "send them through the filters". That pin is driven from a single bit of a general-purpose register, pulled out by a small block called `decim_slice`.

You never set that bit by hand. You ask for the slower *rate*, and Linux engages the filter for you:

terminal

SHELL

```
# run on your HOST. Ask what rates the receive path will give you:
iio_attr -u ip:192.168.2.1 -i -c cf-ad9361-lpc voltage0 sampling_frequency_available

# You get exactly TWO numbers, and they are not fixed - they follow
# whatever the AD9361 is currently converting at:
# converter at 30.72 MSPS -> "30720000 3840000"
# converter at 61.44 MSPS -> "61440000 7680000"
# In both cases: the full rate, and the full rate divided by eight.

# Asking for the lower one turns the filter on:
iio_attr -u ip:192.168.2.1 -i -c cf-ad9361-lpc voltage0 sampling_frequency 3840000
```

The AD9361 keeps running at its full rate; the fabric now hands you one eighth of it. Ask for the full rate again and the mux flips back. **The filter is bypassed unless something asks for the slow rate.**

WHY THERE ARE EXACTLY TWO, AND WHY THEY MOVE

The list is not a menu of everything the board can do — it is a menu of what *this block* can do, and this block can only bypass or divide by eight. So it always offers precisely two options: pass-through, and pass-through ÷ 8.

Both numbers shift whenever you retune the AD9361, because they are derived from its rate rather than stored. If you see a pair you did not expect, check what the converter is set to — that is the number doing the deciding:

terminal

SHELL

```
iio_attr -u ip:192.168.2.1 -i -c ad9361-phy voltage0 sampling_frequency
```

Check yourself: you edit the coefficients, rebuild, flash — and the response is unchanged. Why?

Almost certainly because the filter is bypassed. Nothing asked for the decimated rate, so samples are taking the pass-through path and your new coefficients sit in a block nothing is routed through.

Set `sampling_frequency` to one eighth of the converter rate and measure again. (And check you deleted the Vivado project before rebuilding — lesson 20.)

ONE COEFFICIENT FILE FEEDS BOTH DIRECTIONS

The stock taps are 129 values from `library/util_fir_int/coefile_int.coe` — the **same file** for receive and transmit. Edit it and you have changed both, which is rarely what you meant. Point one side at a different file instead.

Also ignore the `.v` files sitting in `library/util_fir_int/` and `util_fir_dec/`. They look like the filter's source code. They are dead code in this project — never packaged, never built. Only the `.coe` matters.

THE REAL TRAP: TURNING THE FILTER ON QUIETLY RUINS CHANNEL 1

Put the two halves of this lesson together. `cpack` captures *every* enabled channel on channel 0's timing — and channel 1 has no filter of its own.

So the moment you engage the decimator, channel 1 is being sampled at one eighth rate **with no anti-alias filter in front of it**. Everything outside $\pm F_s/16$ folds straight onto it (lesson 12), and it is additionally shifted in time relative to channel 0 by the filter's **group delay** — the delay a filter imposes while it gathers enough samples to compute an output.

In the stock design, channel 1 is only a trustworthy receiver while the filter is bypassed. This is ADI's wiring, not something this repository introduced.

It is also fixable, and this repo fixes it. See the next section.

Fixing it: both receivers, in lockstep

The helper already loops over its channel count, so the repair is to ask for four channels rather than two and feed the second pair through as well. Channel 1 then gets its own pair of filters, identical to channel 0's:

patches/optional/0004-filter-both-receive-channels.patchTCL

```
// 4 channels: ch0 I/Q and ch1 I/Q, rather than 2
ad_add_decimation_filter "rx_fir_decimator" 8 4 1 {61.44} {61.44} <coe>

// feed channel 1 in ...
ad_connect axi_ad9361/adc_data_i1 rx_fir_decimator/data_in_2
ad_connect axi_ad9361/adc_data_q1 rx_fir_decimator/data_in_3

// ... and take cpack's inputs from the filter, not from axi_ad9361
ad_connect cpack/fifo_wr_data_2 rx_fir_decimator/data_out_2
ad_connect cpack/fifo_wr_data_3 rx_fir_decimator/data_out_3
```

The word that matters is **lockstep**. Both channels now share one `active` bit, one coefficient set and therefore one group delay. They are not two filters that happen to be similar — they are the same filter instantiated twice, switching on together and delaying their signals by exactly the same amount. That is what keeps the two receivers sample-aligned, which is the whole point of having two.

`cpack/fifo_wr_en` still takes `valid_out_0`, and that is now correct rather than merely tolerable: all four paths produce output on the same schedule.

CHANNEL 1, DECIMATOR ENGAGED	STOCK	PATCHED
A tone 10 MHz out of band appears at	+2.320 MHz (an alias)	not at all
at a level of	70.1 dB	below the noise
DSP48 slices used	72 / 220	94 / 220
Worst negative slack	+0.205 ns	+0.215 ns

Two extra filter instances, about 11 DSP slices each, and over 70 dB of alias suppression on a channel that previously had none. Full write-up: [docs/both-receive-channels.md](#) in the repository.

WHY ONE FILTER DESIGN IS CORRECT FOR BOTH RECEIVERS

A reasonable worry: if the two receivers were tuned to different frequencies, would a single shared filter design still protect both?

On this chip the question cannot arise. The AD9361 has exactly **two** local oscillators — one for receive, one for transmit — and both receivers share the receive one. Ask the board and only `RX_LO` and `TX_LO` exist; there is no `RX2_LO`. The analogue `rf_bandwidth` is shared too: set it on one channel and the other follows. Only *gain* is genuinely per-channel.

So both receivers always observe the same band, at the same width, at the same moment. That is not a limitation so much as the point — it is what makes the pair useful for MIMO, direction finding and any measurement where the two must be comparable. And it is exactly why one filter design is the right filter for both.

IF YOU DO WANT TWO DIFFERENT CENTRE FREQUENCIES

Put a digital mixer on each channel in the fabric, ahead of the decimator (lesson 27). Each channel's oscillator shifts a different part of the captured band down to zero, and each channel's decimating filter then protects its own stream in its own frame. No aliasing, two independent tuned channels, one shared radio front end.

That is a channelizer, and it is the thing the AD9361's single LO cannot do for you but the fabric can.

AND THE TRANSMIT INTERPOLATOR DOES NOT WORK AT ALL

The same lopsidedness, mirrored, with a worse ending. The transmit strobe is channel 0's valid *OR* channel 1's — and channel 1 has no interpolator, so on this 2R2T board its valid keeps firing at full rate and drags the packer along with it.

Measured: `tx_upack` was read at **twelve times** the intended rate, and a tone sent through the interpolator did not come out at all — the received spectrum matched a muted transmitter to within 1.2 dB, while the same data sent the normal way arrived clean.

You only reach this by setting the transmit core's `sampling_frequency` to one eighth of the AD9361's rate. Do not.

16 How samples reach memory, and back

The full journey in both directions: from the packer, through a DMA engine, across a dedicated port into the DDR chips, and up into your program's buffer. Every address and every setting here is this board's.

The problem being solved

At 61.44 MSPS each direction is 245 MB/s. A 666 MHz ARM core cannot copy that, byte by byte, and do anything else. It would not even come close.

So it does not. A separate piece of hardware moves the data while the processor gets on with its life, and only tells the processor when a whole buffer is done. That hardware is a **DMA engine** — Direct Memory Access.

THE PIECES, NAMED

DDR	The board's main memory: two Micron chips totalling 1 GB. It is attached to the processor side, not the fabric — so anything the fabric wants to put there has to cross over.
AXI	The on-chip bus standard everything uses to talk to everything else. Two flavours matter here: AXI-Stream , a simple "here is a word, is it valid, are you ready" flow, and AXI-MM (memory-mapped), which carries addresses and does reads and writes at them.
HP port	A High-Performance slave port on the processor side. These are wide, fast doors into the memory controller that the fabric can use directly, without troubling the CPU. This board wires two of them.
Descriptor	A small instruction telling the DMA engine "move this many bytes, to or from this address". Software writes descriptors; the engine executes them.

The two engines on this board

Both are ADI's `axi_dmac`, one per direction, 64 bits wide on the fabric side — which is exactly the width `cpack` produces and `tx_upack` consumes. That is not a coincidence; the packers exist partly to make this width match.

	RECEIVE	TRANSMIT
Block	<code>axi_ad9361_adc_dma</code>	<code>axi_ad9361_dac_dma</code>
Register address	<code>0x7C40_0000</code>	<code>0x7C42_0000</code>
Interrupt	ps-13	ps-12
Door to memory	<code>S_AXI_HP1</code>	<code>S_AXI_HP2</code>
Fabric side	<code>DMA_TYPE_SRC 2</code> — a FIFO fed by <code>cpack</code>	<code>DMA_TYPE_DEST 1</code> — AXI-Stream into <code>tx_upack</code>
Memory side	AXI-MM <i>writing</i> to DDR	AXI-MM <i>reading</i> from DDR
Notable setting	<code>SYNC_TRANSFER_START</code>	<code>CYCLIC 1</code>
Linux device	<code>cf-ad9361-lpc</code>	<code>cf-ad9361-dds-core-lpc</code>

Those addresses are **mirrored in the device tree** (`zynq-pluto-sdr-fishball.dts`). Move a block in the block design and the driver probes the wrong place; add one and it needs a device-tree node before Linux can see it at all.

EACH DMA STRADDLES TWO CLOCK DOMAINS

Look back at lesson 14. The *fabric* side of each engine runs on `l_clk`, which moves with the sample rate. The *memory* side runs on `sys_cpu_clk`, a fixed 100 MHz.

So the DMA is itself a clock-domain crossing — and a well-tested one you get for free. It absorbs the difference with internal buffering, which is also why a brief stall on the memory side does not immediately corrupt the sample stream: there is slack in between.

Receive, step by step

1. The AD9361 sends samples over the LVDS link. `axi_ad9361` recovers the clock, deserialises, sign-extends 12 bits to 16, applies DC-offset correction.
2. Channel 0 passes through `rx_fir_decimator` (lesson 15); channel 1 goes straight past it.
3. `cpack` interleaves the enabled channels into 64-bit words, one word per pulse of `fifo_wr_en`.
4. Those words land in the receive DMA's input FIFO — a small on-chip queue absorbing the difference between the steady sample rate and the bursty memory bus.
5. The engine bursts them across `S_AXI_HP1` into DDR, at addresses software gave it in a descriptor.
6. When a buffer is full, the engine raises interrupt ps-13. The driver marks that buffer ready and hands the next one to the hardware.
7. Your program's `read()` — or `iio_readdev`, or pyadi-iio — returns the completed buffer. **No sample was ever copied by the CPU.**

`SYNC_TRANSFER_START` is why a capture begins on a clean packer boundary rather than halfway through an interleaved word. Without it, your first samples could be the tail of a word and the channel order would be shifted.

Transmit, step by step

1. Your program writes samples into a buffer. The driver hands the DMA a descriptor pointing at it in DDR.
2. The engine reads from DDR across `S_AXI_HP2`, filling an internal FIFO ahead of demand.
3. It presents 64-bit words to `tx_upack` as an AXI-Stream, one per pulse of `fifo_rd_en`.
4. `tx_upack` splits each word back into per-channel 16-bit samples.
5. `axi_ad9361` takes bits `[15:4]` of each — the DAC is 12 bits — and sends them over the LVDS link. Bits `[3:0]` are discarded, which is exactly what the sample-locked GPIO feature reuses.
6. Interrupt ps-12 tells the driver a buffer has been consumed, so it can queue the next.

UNDERFLOW, AND WHY IT SOUNDS LIKE A BROKEN RADIO

If software does not refill fast enough, the transmit engine runs dry. It does not stall and wait — the radio cannot pause. It **substitutes zeros** and raises `fifo_rd_underflow`.

The result is a transmitted waveform that is not the one you generated: chunks of your signal replaced by silence, at unpredictable moments. Measured on this board, that is exactly what happens above about 5 MSPS when transmit and receive stream continuously at the same time — and it looks like terrible modulation quality rather than like a missing-data problem, which makes it easy to misdiagnose.

The receive equivalent is **overflow**: the engine has nowhere to put a sample because software has not drained the previous buffer, so samples are lost.

`CYCLIC 1`, AND WHY A CRASHED PROGRAM CAN KEEP TRANSMITTING

The transmit engine can be told to **repeat a buffer forever** without any further software involvement. That is enormously useful — it is how you transmit at the full 61.44 MSPS without the host feeding anything at all.

It is also a safety trap. Once started, the repetition lives in *hardware*. Your program exiting, crashing, or being killed does not stop it; the DMA keeps reading the same DDR buffer and the transmitter keeps radiating. On this board a locally-killed transmit client has been measured leaving the radio live indefinitely.

If you use cyclic mode, make stopping it explicit and verified — not something you assume happens when your process ends.

Buffer sizing, from Linux

Buffers are allocated by the kernel's IIO DMA layer, and there is a ceiling on how large one block may be. This board sets it at boot:

```
buildroot/board/pluto/S21misc
```

```
SHELL
```

```
MAX_BS=`fw_printenv -n iio_max_block_size 2> /dev/null || echo 67108864`  
echo ${MAX_BS} > /sys/module/industrialio_buffer_dma/parameters/max_block_size
```

64 MB by default — about 16 million samples — overridable from the U-Boot environment with `fw_setenv iio_max_block_size`.

The trade is the usual one. **Large buffers** mean fewer interrupts and far more tolerance of a host that pauses, at the cost of latency: nothing is delivered until the whole buffer is full. **Small buffers** react quickly but demand that software keeps up, every time, or you underflow.

WATCH IT HAPPEN

The DMA engines report their own overflow and underflow, and there is a tool that just prints them:

terminal

SHELL

```
# run on your HOST - raise the sample rate until this starts complaining  
iio_adi_xflow_check -u ip:192.168.2.1 cf-ad9361-lpc
```

Then find where *your* link gives up. It is a genuinely useful number to know before you design around a rate.

17 The worked example, line by line

A real module from this repo. Twenty lines of logic that teach four habits at once.

What it does. The AD9361's DAC is 12 bits, but you hand it 16-bit samples. It takes the top 12 and throws the bottom four away — literally, in ADI's HDL: `dac_data_out <= dma_data[15:4]`. Those four bits reach the fabric and stop there. This module catches them and puts them on four header pins, in lockstep with the sample that carried them. It costs nothing in signal quality, because the bits never reached the DAC.

```
tx_gpio_bitmap.v VERILOG

module tx_gpio_bitmap (
    input wire      clk,          // l_clk
    input wire      rst,
    input wire [3:0] sample_in,   // the discarded low nibble
    input wire      valid_in,     // fifo_rd_valid | fifo_rd_underflow
    input wire      flag,         // enable, from an AXI register
    input wire [3:0] gpio_in,     // what the PS wants when disabled
    output wire [3:0] pin_o
);

    // (1) 'flag' is written in the AXI clock domain and read here in
    //      l_clk. Two flip-flops make that safe. See lesson 22.
    reg flag_meta, flag_sync;
    always @(posedge clk) begin
        flag_meta <= flag;
        flag_sync <= flag_meta;
    end

    // (2) capture on the VALID strobe, never on the clock: in 2R2T a
    //      sample arrives only every second l_clk edge.
    reg [3:0] held;
    always @(posedge clk) begin
        if (rst) held <= 4'd0;
        else if (valid_in) held <= sample_in;
    end

    // (3) our nibble, or the processor's GPIO
    assign pin_o = flag_sync ? held : gpio_in;

endmodule
```

The four habits

1. Synchronise anything arriving from another clock. `flag` is written by Linux. Sampling it directly risks metastability. Two flip-flops cost almost nothing.
2. Gate on valid, never on the clock. The 2R2T trap from lesson 14.
3. Reset only what needs it. `held` does; the synchroniser flops do not. Over-resetting wastes fabric and strains timing.

4. **Keep the bypass clean.** With `flag` clear the pins behave exactly as before. A feature you cannot fully switch off is one people will not switch on.

TRY IT

Change the module so a second flag bit puts a free-running counter on the pins instead of the sample nibble. You now have a divided sample clock on a header pin — a scope trigger with a fixed, known relationship to the transmitted RF, which no software timing can give you.

18 Wiring it in with Tcl

This board's block design is a script, not a drawing — which means your change is a reviewable diff rather than a mouse gesture nobody can check.

Tcl is the scripting language Vivado is driven by. You do not need to learn it as a language; you need four kinds of line, all in `firmware/src/hdl/projects/pluto/system_bd.tcl`.

system_bd.tcl

TCL

```
# 1. make the source file part of the project
add_files -norecurse $ad_hdl_dir/projects/pluto/tx_gpio_bitmap.v

# 2. instantiate it as a cell in the block design
create_bd_cell -type module -reference tx_gpio_bitmap tx_bitmap

# 3. slice the bits you want out of a wide bus
ad_ip_instance xlslice nibble_slice [list \
    DIN_WIDTH 16 DIN_FROM 3 DIN_TO 0 DOUT_WIDTH 4]

# 4. connect things up
ad_connect axi_ad9361/l_clk      tx_bitmap/clk
ad_connect axi_ad9361/rst      tx_bitmap/rst
ad_connect tx_upack/fifo_rd_data_0 nibble_slice/Din
ad_connect nibble_slice/Dout    tx_bitmap/sample_in
ad_connect tx_upack/fifo_rd_valid bitmap_valid_or/Op1
ad_connect tx_upack/fifo_rd_underflow bitmap_valid_or/Op2
ad_connect bitmap_valid_or/Res  tx_bitmap/valid_in
```

The helper blocks you will reach for

BLOCK	USE
<code>xlslice</code>	Take bits <code>[3:0]</code> out of a 16-bit bus. Configured by parameters, no HDL required.
<code>xlconcat</code>	Glue narrow signals into a wide one.
<code>util_vector_logic</code>	AND / OR / NOT on buses — this is how the valid-or-underflow OR above is built.
<code>xlconstant</code>	Tie a signal permanently high or low.
<code>create_bd_port</code>	Bring a signal out to a physical pin — then constrain it, lesson 20.

WHERE IN THE FILE MATTERS

`add_files` must run before the module is referenced, and this script is sourced from inside `adi_project_create`. Put the `add_files` line next to your `create_bd_cell`, not in `system_project.tcl` — by the time that runs the fileset is already built, and your module will simply be "not found".

19 Packaging your Verilog as an IP, and splicing it into the datapath

Lesson 18 dropped a module into the block design in three lines. That is the right answer for a small change and the wrong one for anything you intend to keep. This lesson turns the same Verilog into a real Vivado IP — one that appears in the catalog, carries parameters and interfaces, and can be cut into the receive or transmit path the way ADI's own blocks are.

Module or IP? Decide once, up front

	CREATE_BD_CELL -TYPE MODULE	A PACKAGED IP
Effort	three lines of Tcl	a directory, a Tcl script and a build step
Reusable in another project	no — the path is hard-coded	yes — it is in the catalog
Parameters	Verilog parameters, no GUI, no validation	named, typed, with a GUI and enablement rules
Interfaces	loose wires, connected one at a time	buses — one <code>ad_connect</code> moves the whole bundle
Versioning	none	1.0, 1.1, ... and the design records which it used
Out-of-context synthesis	no — resynthesised with the whole design	yes, and cached, so rebuilds are faster

The honest rule: **prototype as a module, package when it works**. Packaging a design you are still changing is a tax you pay on every edit, because every change means re-running the packaging step before the project sees it.

Step 1 — write the RTL with the packager in mind

Vivado infers interfaces from *signal names*. Name your ports the way it expects and a 36-signal AXI4-Stream collapses into one bus you connect with a single line; name them creatively and you get 36 loose pins forever. Here is a receive-path block — a scaler, one multiply per sample — written for this board's conventions:

```

module util_rxscale #(
  parameter DATA_WIDTH = 16
) (
  input wire          clk,          // 1_clk - the recovered clock
  input wire          resetn,

  // ---- the ADI datapath convention, in ----
  input wire          valid_in,     // one pulse per sample
  input wire          enable_in,    // channel is switched on (a level, not a strobe)
  input wire [DATA_WIDTH-1:0] data_in,

  // ---- and out ----
  output reg          valid_out,
  output wire         enable_out,
  output reg [DATA_WIDTH-1:0] data_out,

  input wire [7:0]     shift        // runtime control, see step 6
);
assign enable_out = enable_in;      // pass the level straight through

wire signed [DATA_WIDTH-1:0] s = data_in;

always @(posedge clk) begin
  if (!resetn) begin
    valid_out <= 1'b0;
    data_out  <= {DATA_WIDTH{1'b0}};
  end else begin
    valid_out <= valid_in;          // SAME latency as the data - one cycle
    data_out  <= s >>> shift;      // arithmetic shift: keeps the sign
  end
end
endmodule

```

THE CONTRACT YOU ARE SIGNING WHEN YOU CUT INTO THIS DATAPATH

1. Delay **valid** by exactly as much as **data**. They are a pair. One extra register on the data and every sample downstream is labelled with its neighbour's strobe.
2. **enable** is a level, not a strobe. It says "this channel is switched on", set by software long before any samples flow. Do not register it into your pipeline and do not treat it as a qualifier per sample.
3. Do not change the rate unless you mean to. One **valid_in** should produce one **valid_out**. Producing fewer is a decimator, and that has consequences two paragraphs down.
4. Everything is on **1_clk**. Not **sys_cpu_clk**. If you need a value from the processor side, it crosses domains, and lesson 22 is about how.

Step 2 — package it

The GUI route is *Tools* → *Create and Package New IP* → *Package a specified directory*, and it works. It is also a sequence of mouse clicks that nobody can review and you cannot repeat, so use it to explore and then write the script. This repository's library is entirely scripted, and your IP should match:

<code>library/util_rxscale/util_rxscale_ip.tcl</code>	TCL
---	-----

```

source ../../scripts/adi_env.tcl
source $ad_hdl_dir/library/scripts/adi_ip_xilinx.tcl

adi_ip_create util_rxscale           # make the packaging project
adi_ip_files  util_rxscale [list "util_rxscale.v"] # the sources it contains
adi_ip_properties_lite util_rxscale    # name, vendor, taxonomy, clocks

ipx::save_core [ipx::current_core]
```

`adi_ip_properties_lite` Fills in the vendor, library, version and taxonomy and infers clock and reset. Use it for a datapath block like this one.

`adi_ip_properties` The same, *plus* it infers an AXI4-Lite slave from ports named `s_axi_awvalid`, `s_axi_wdata` and the rest. Use it when your IP has its own registers — step 6.

`adi_add_bus` Declares a custom interface by mapping your port names onto an abstraction. It is how `util_cpack2` turns five loose signals into the single `packed_fifo_wr` bus you see in the block design. Worth copying when your block has more than a handful of ports.

Build it the way every other IP here is built:

<code>packaging the IP</code>	RUN FROM: FIRMWARE/SRC/HDL/LIBRARY/UTIL_RXSCALE/
-------------------------------	--

```

cp ../util_pack/util_cpack2/Makefile . # then edit LIBRARY_NAME and the DEPS
make                                   # runs vivado -mode batch on the _ip.tcl
ls component.xml                       # this file IS the IP
```

WHY DROPPING IT IN `library/` IS ENOUGH

`adi_project_create` sets the project's `ip_repo_paths` to `$ad_hdl_dir/library` and then calls `update_ip_catalog`. So any directory under `library/` containing a `component.xml` is in the catalog of *every* project in the tree, with no per-project registration at all. Put your IP anywhere else and you have to add the path yourself — which is a line somebody will forget.

Step 3 — instantiate it

Once it is in the catalog it instantiates like any ADI block, from `projects/pluto/system_bd.tcl`:

<code>system_bd.tcl</code>	TCL
----------------------------	-----

```

ad_ip_instance util_rxscale rx_scale_i [list DATA_WIDTH 16]
ad_ip_instance util_rxscale rx_scale_q [list DATA_WIDTH 16]
```

Note what changed from lesson 18: no `add_files`, because the IP carries its own sources, and parameters are passed by name rather than hoped for.

Step 4 — cut it into the receive path

Insertion is always the same two moves: **delete the existing connection, then make two new ones**. On receive, the cut is between `axi_ad9361` and whatever currently consumes its samples — `rx_fir_decimator` in this design, `cpack` in a design without the filter.

system_bd.tcl — channel 0, I and Q

TCL

```
# 1. break what ADI wired
ad_disconnect axi_ad9361/adc_data_i0 rx_fir_decimator/data_in_0
ad_disconnect axi_ad9361/adc_valid_i0 rx_fir_decimator/valid_in_0

# 2. clock and reset your block from the same domain as the data
ad_connect axi_ad9361/l_clk rx_scale_i/clk
ad_connect sys_rstgen/peripheral_aresetn rx_scale_i/resetn

# 3. route through it
ad_connect axi_ad9361/adc_data_i0 rx_scale_i/data_in
ad_connect axi_ad9361/adc_valid_i0 rx_scale_i/valid_in
ad_connect axi_ad9361/adc_enable_i0 rx_scale_i/enable_in

ad_connect rx_scale_i/data_out rx_fir_decimator/data_in_0
ad_connect rx_scale_i/valid_out rx_fir_decimator/valid_in_0
ad_connect rx_scale_i/enable_out rx_fir_decimator/enable_in_0
```

Repeat for `_q0`, and — this is the part people skip — for `_i1` and `_q1` as well.

DO ALL FOUR CHANNELS, OR YOU WILL REDISCOVER THIS BOARD'S KNOWN DEFECT

`cpack/fifo_wr_en` is driven by **one** channel's valid, and it captures every enabled channel on that single strobe. So if your block adds a cycle of latency to channel 0 and you leave channel 1 untouched, the two channels are permanently one sample apart in the buffer — and nothing anywhere reports an error.

This is not hypothetical: it is exactly the asymmetry documented in `both-receive-channels.md` and measured in lesson 15. Whatever you insert, insert it in all four paths, with identical latency.

Step 5 — and the transmit path, which is fussier

Transmit runs the other way: `tx_upack` produces samples, `axi_ad9361` consumes them. The cut is symmetrical:

system_bd.tcl — transmit, channel 0 I

TCL

```
ad_disconnect tx_upack/fifo_rd_data_0 axi_ad9361/dac_data_i0

ad_connect tx_upack/fifo_rd_data_0 tx_scale_i/data_in
ad_connect tx_upack/fifo_rd_valid tx_scale_i/valid_in
ad_connect axi_ad9361/dac_enable_i0 tx_scale_i/enable_in
ad_connect tx_scale_i/data_out axi_ad9361/dac_data_i0
```

TWO TRANSMIT-SIDE FACTS THAT HAVE EACH COST A BUILD HERE

The read enable is an OR, and it bites

`tx_upack/fifo_rd_en` is the OR of the interpolator's valid *and* channel 1's DAC valid. On a 2R2T board like this one, channel 1 therefore drags the packer along at full rate regardless of what channel 0's path is doing. A same-rate block is fine. A *rate-changing* block on transmit is not — engage the fabric ÷8 interpolator and, measured, TX1 emits nothing at all.

The DAC keeps the top 12 bits

You hand it 16 and it discards the low nibble. So a scaler that shifts right is throwing away bits the converter was never going to use anyway — and those four discarded bits are the ones `tx-gpio-bitmap.md` routes to the header pins.

Step 6 — giving it a knob software can turn

The `shift` input has to come from somewhere. Two routes, in increasing order of effort:

Borrow a GPIO bit

What ADI did for the decimator. `axi_ad9361` exposes `up_adc_gpio_out`, a register of spare bits written from userspace; a `xlslice` picks out the ones you want. It is a few lines, needs no new address, and is how the filter's `active` bit is controlled today. Cheap, and limited to a handful of bits.

Add your own AXI4-Lite slave

Name the ports `s_axi_*`, call `adi_ip_properties` instead of the `_lite` variant so the bus is inferred, then give it an address: `ad_cpu_interconnect 0x7C44_0000 rx_scale_i`. You now have real registers at a real address, reachable with `devmem` or from a driver, at the cost of writing the register file. Lesson 23 is about that side.

Whichever you pick, the value crosses from `sys_cpu_clk` to `l_clk`, so it needs the treatment in lesson 22 — and a constraint, or Vivado will quietly drop the path.

Step 7 — build it, and the one step everybody forgets

the whole loop

RUN FROM: THE REPO ROOT

```
# 1. package (only when the IP's own sources changed)
cd firmware/src/hdl/library/util_rxscale && make && cd -

# 2. DELETE THE PROJECT. build_hdl.tcl reuses an existing pluto.xpr and
#    will silently ignore your block-design change if you skip this.
rm -rf firmware/src/hdl/projects/pluto/pluto.{xpr,cache,gen,hw,ip_user_files,runs,sim,srcs,sdk}

# 3. build, check, flash
./devkit build --hdl-only
./devkit verify
./devkit flash --boot-only
```

Step 2 is the single most expensive mistake in this whole course. The build succeeds, the bitstream is valid, the board boots — and it is running your *previous* design. Delete the project.

PROVE THE INSERTION BEFORE YOU TRUST IT

Give your block a deliberately obvious effect first — a shift of 3, which is a clean -18 dB — and check that a capture really drops by 18 dB on *both* channels and *both* I and Q. Then set the shift to 0 and check the capture is bit-identical to a capture taken before your block existed.

Those two tests together catch a mis-wired channel, a swapped I/Q, an off-by-one on valid, and a block that is not actually in the path at all — which is more failure modes than any amount of staring at the block diagram will.

Check yourself: your block adds one cycle of latency. What else in the design has to know?

On receive: **nothing downstream cares about absolute latency**, because `cpack` captures on a strobe and the DMA just stores what it is given. But everything cares about latency *differences* — so the answer is that the other three channels have to have the same one cycle, or the four streams in the buffer no longer line up.

And if you are doing anything that compares the two receivers — the direction-finding of lesson 42, or simply trusting that sample n on channel 0 and channel 1 happened at the same instant — then a one-cycle difference is a phase error that scales with frequency, and it will look like a real measurement.

20 Pins and constraints

A signal that reaches the edge of the fabric still has to be told which physical ball to leave by, and at what voltage.

Constraints live in an **XDC** file (Xilinx Design Constraints) — `system_constr.xdc`. One line per pin:

system_constr.xdc

XDC

```
set_property -dict {PACKAGE_PIN V10 IOSTANDARD LVCMOS33 PULLTYPE PULLDOWN} \  
[get_ports sample_gpio_o[0]]
```

PACKAGE_PIN Which physical ball on the chip package. `V10` is a grid reference.

IOSTANDARD The electrical standard, which must match the voltage supplied to that bank of pins. `LVCMOS33` means 3.3 V logic.

PULLTYPE An optional weak resistor holding the pin to a known level when nothing is driving it.

get_ports Names the signal in your top-level design that this applies to.

The four pins verified on this board

JP5 PIN	BALL	BANK	STANDARD
7	V10	13	LVCMOS33
9	U9	13	LVCMOS33
11	U10	13	LVCMOS33
13	T9	13	LVCMOS33

Bank 13 is supplied from the 3.3 V rail, which is why the standard is `LVCMOS33`. Declaring an **IOSTANDARD** that does not match the bank's supply is a build error at best.

DO NOT INVENT PINS FROM THE SCHEMATIC

Driving a ball that turns out to be an input, or tied elsewhere on the board, can damage the hardware. The four above are known good because they were traced and then measured. Anything else needs the schematic *and* care.

21 Driving Vivado, and reading what it tells you

You will barely click anything. But you must be able to read the report.

THE TRAP THAT COSTS EVERYONE ONE BUILD

`build_hdl.tcl` reuses an existing Vivado project rather than re-running `system_bd.tcl`. Change the block design or a coefficient file without deleting the project and your change is **silently ignored** — you wait twenty minutes, flash, and then debug logic that was never built.

before EVERY HDL or coefficient change

SHELL

```
# run from: firmware/src
```

```
rm -rf hdl/projects/pluto/pluto.{xpr,cache,gen,hw,ip_user_files,runs,sim,srcs,sdk}
```

Check yourself: you change a coefficient file, rebuild, flash — and measure the old filter. What happened?

The build reused the existing Vivado project instead of re-running the block-design script, so your new coefficients were never read. You flashed a bitstream built from the previous ones.

Delete the project directory first. This applies to coefficient files exactly as it does to Verilog — they are both inputs to the block design.

What the build actually does

STAGE	MEANING
Synthesis	Verilog → a netlist of gates and flip-flops. Catches syntax errors and infers latches.
Placement	Decides which physical LUT and flip-flop on the die each piece of that netlist becomes.
Routing	Chooses the wires between them. This is where timing is won or lost, because wire length is delay.
Bitstream	Writes the configuration file.

Reading timing

The number that matters is **WNS** — Worst Negative Slack, in nanoseconds. **Slack** is spare time: how much earlier than its deadline the slowest signal arrived.

- **WNS positive** — timing met, with that much margin. The stock build on this board lands near `+0.2 ns`.
- **WNS negative** — some path is too slow. The bitstream still builds, and may even appear to work, but it is unreliable and temperature-dependent. Do not ship it.

Fixes, in the order worth trying: add a pipeline register to cut a long combinational path in two; narrow an arithmetic operation; move a multiply onto a DSP slice; and only then start adjusting constraints.

Your budget on this chip

RESOURCE	STOCK BUILD	AVAILABLE	SPARE
DSP slices	72	220	148
LUTs	11,896	53,200	41,304

Two-thirds of the multipliers and three-quarters of the logic are free. A 300-tap filter fits comfortably.

VERIFY BEFORE YOU FLASH

`./devkit verify` checks the five output files, confirms timing was met, and confirms the bitstream is *compressed*. That last one matters more than it sounds: an uncompressed bitstream overflows the FSBL's on-chip memory and the board fails to boot **with no message at all**.

22 Crossing clock domains

The richest source of bugs that pass simulation, pass timing, and then fail on hardware occasionally.

Your enable bit is written by Linux in the AXI clock domain. Your datapath runs on `l_clk`, whose frequency changes with the sample rate. These two clocks have no fixed relationship whatsoever.

A flip-flop needs its input to be stable for a short window either side of the clock edge. If the input changes exactly then, the flip-flop can enter **metastability** — an unstable in-between state, neither 0 nor 1, that resolves after an unpredictable delay. Anything downstream sees garbage, and worse, different downstream logic may resolve it differently.

two-flop synchroniser

VERILOG

```
reg meta, sync;
always @(posedge dest_clk) begin
    meta <= src_signal;    // this one may go metastable
    sync <= meta;         // by now it has had a full cycle to settle
end
// use 'sync' everywhere. Never use 'meta'.
```

ONE BIT ONLY

This works for slowly-changing control bits. For a multi-bit value it is *wrong*: each bit settles independently, so different bits can land on different cycles and you read a number that never existed. Multi-bit crossings need a handshake or an asynchronous FIFO.

Telling the tools it is deliberate

Synthesis does not know the crossing is intentional, so you say so in the XDC:

system_constr.xdc

XDC

```
set_max_delay -datapath_only \
    -from [get_cells ../flag_reg] \
    -to   [get_cells ../flag_meta_reg] 4.000
```

A REAL BUG FROM THIS REPO'S HISTORY

`set_max_delay -datapath_only` **requires** `-from`. Written without it, Vivado does not raise an error — it silently drops the constraint, and nothing in the build log mentions it. On this board that left the bit-map flag's crossing entirely unconstrained through several releases. It happened to work.

Patch `0009` fixed it, and CI now parses the XDC and fails the build if any `-datapath_only` line is missing its `-from`. That is the right response to a silent failure: make it loud, permanently.

23 Registers, and talking to Linux

Logic you cannot control from software is a demo. Here is where the two halves meet.

ADI's cores expose a block of AXI registers — memory addresses that, when written from Linux, change signals in the fabric. One of them, `GP_CONTROL` at offset `0xBC` on the transmit core, is a general-purpose output whose bits you can slice and use. That is how the bit-map feature gets its enable without adding a whole new AXI device:

BIT	MEANING
0	Interpolator bypass (ADI's own use)
1	Sample-nibble GPIO enable (added by patch <code>0006</code>)

Because two independent features share one register, every write must be **read-modify-write**: read the current value, change only your bit, write it back. Assigning the whole register clears the other feature.

Poking it by hand

terminal

SHELL

run on your HOST

```
iio_attr -u ip:192.168.2.1 -D cf-ad9361-dds-core-lpc direct_reg_access 0xBC
iio_attr -u ip:192.168.2.1 -D cf-ad9361-dds-core-lpc direct_reg_access
```

BIT 31 DECIDES WHERE THE ADDRESS GOES

On `cf-ad9361-lpc` (the receive core) a plain address like `0xB8` is passed to the AD9361 over SPI, not to the FPGA core. Only `0x800000B8` reaches the core's own register. The transmit core happens to run standalone and maps plain addresses to itself — so identical code "works" on one core and silently reads and writes *radio chip* registers on the other.

Giving it a proper name

Raw register pokes are fine for bring-up and wrong for a shipped feature. Add a **sysfs attribute** in the Linux driver and your feature becomes a named file, readable and writable from anywhere including over the network:

cf_axi_dds.c

C

```
static ssize_t axidds_tx_sample_gpio_store(struct device *dev,
      struct device_attribute *attr, const char *buf, size_t len)
{
    /* read-modify-write: bit 0 belongs to someone else */
    cf_axi_dds_lock(st);
    reg = dds_read(st, ADI_REG_DAC_GP_CONTROL);
    if (enable) reg |= ADI_TX_SAMPLE_GPIO_EN;
    else        reg &= ~ADI_TX_SAMPLE_GPIO_EN;
    dds_write(st, ADI_REG_DAC_GP_CONTROL, reg);
    cf_axi_dds_unlock(st);
    return len;
}
```

terminal

SHELL

```
# run on your HOST - no ssh needed, it is a device attribute
iio_attr -u ip:192.168.2.1 -d cf-ad9361-dds-core-lpc tx_sample_gpio_en 1
```

24 The frequency domain, and what a spectrum really is

You have been reading spectra since lesson 12 without ever being told what one is. Here is the whole idea, and the three settings that decide whether the picture can be trusted.

One signal, two descriptions

A signal can be described by **what it does over time** — a list of samples, one after another — or by **which frequencies it is made of**. Neither description is more true than the other. They are two complete accounts of the same thing, and you can convert between them without losing anything at all.

The conversion is the **Fourier transform**. Its practical form on a computer is the **DFT** (Discrete Fourier Transform), and the clever algorithm everyone actually runs is the **FFT** (Fast Fourier Transform). When a tool says "4096-point FFT", 4096 is how many samples it swallowed in one go.

THE INTUITION, WITHOUT THE INTEGRAL

To ask "how much 1 MHz is in this signal?", multiply the signal by a 1 MHz reference wave and add up the result. If the signal really contains 1 MHz, the products keep the same sign and the sum grows. If it does not, the products land half positive and half negative, and the sum stays near zero.

Do that for every frequency and you have a spectrum. The FFT is nothing more than an efficient way of doing all of those sums at once, reusing the arithmetic they share.

What the picture is actually showing

An FFT of N samples gives you N numbers back, called **bins**. Each bin is one narrow slice of frequency, and its value is how much energy landed in that slice. Plot the bins left to right, in decibels, and that is the spectrum you have been looking at.

Because this board samples I and Q (lesson 11), its bins run from $-f_s/2$ to $+f_s/2$ around the tuned frequency — negative offsets are real and mean "below the local oscillator".

The three settings that decide what you see

SETTING	WHAT IT CONTROLS	ON THIS BOARD
Sample rate	How wide a span you see: $-f_s/2$ to $+f_s/2$	2.083 to 61.44 MSPS, so up to 61.44 MHz of span
FFT length	How finely you can separate two nearby signals: resolution = f_s / N	4096 points at 61.44 MSPS gives 15 kHz bins
Window	How much a strong signal smears over its neighbours	A Hann window is the usual default

WHY A WINDOW IS NEEDED AT ALL

- The problem** The FFT quietly assumes your chunk of samples repeats forever. Unless the signal happens to fit a whole number of cycles into the chunk, the end does not line up with the start — and that sudden step is broadband energy which was never in the signal. It appears as **spectral leakage**: skirts spreading out either side of a tone, burying anything small that was sitting there.
- The fix** Multiply the chunk by a shape that tapers smoothly to zero at both ends, so there is no step left to smear. That shape is the **window**. Hann is the sensible default; Blackman-Harris trades more width for even less leakage; a *rectangular* window means no window at all.
- The cost** Tapering widens the tone by roughly 1.5×. You trade a little resolution for a very large reduction in leakage, and on real measurements it is almost always worth it.

The trade you cannot escape

Resolution is f_s / N . To see finer frequency detail you need a longer FFT, which means collecting samples for longer, which means less ability to see something that changes quickly. Frequency resolution and time resolution trade directly against one another. There is no setting that gives you both, and no amount of processing invents the difference.

At 61.44 MSPS a 4096-point FFT covers 66.7 μ s of time and 15 kHz per bin. Drop to 2.083 MSPS and the same 4096 points cover 1.97 ms and 509 Hz per bin. Same transform, entirely different instrument.

PROCESSING GAIN, AND WHY A NOISE FLOOR IS NOT A NUMBER

Noise spreads itself across every bin; a tone lands in one. Double the FFT length and each bin collects half as much noise, so the tone appears to rise 3 dB above the floor — while absolutely nothing about the signal changed.

A quoted noise floor is therefore meaningless without the FFT length beside it. Two measurements of the same board, taken minutes apart, can differ by 10 dB through this alone. It is also why lesson 37 insists you state the transform length whenever you quote one — and why the figures in `docs/measured-performance.md` always do.

THE SAME IDEA, WEARING THREE HATS

Lengthening an FFT, decimating by 8, and narrowing the receive filter all improve signal-to-noise by exactly the same mechanism: *noise is spread out and signal is not, so narrowing your view always favours the signal*. You will meet this again in lesson 25 as noise bandwidth and in lesson 27 as process gain. It is one fact with three names.

Check yourself: at 61.44 MSPS with a 4096-point FFT, can you separate two signals 10 kHz apart?

No. Resolution is $61.44 \text{ MHz} / 4096 = 15 \text{ kHz}$ per bin, so two signals 10 kHz apart fall in the same bin and appear as one.

Two ways out: a longer FFT (65 536 points gives 937 Hz), or — usually the better move on this board — drop the sample rate so you are not spending resolution on spectrum you do not care about. At 2.083 MSPS the same 4096-point FFT gives 509 Hz bins, and the capture is thirty times smaller.

25 Noise, decibels, and where the floor comes from

Every measurement in this course is quoted in dB against a noise floor. Both halves of that sentence deserve explaining properly, because almost every wrong RF number traces back to one of them.

Decibels, once and for all

A **decibel** is a ratio written on a logarithmic scale. Radio needs it because one page has to hold both a transmitter and the thermal noise it is competing with, and those differ by a factor of a hundred billion.

the two formulas

dB

power ratio $\text{dB} = 10 \times \log_{10}(P_1 / P_2)$

amplitude ratio $\text{dB} = 20 \times \log_{10}(A_1 / A_2)$

Two formulas, because power goes as amplitude squared — the 20 is not a different unit, it is the same formula with the square pulled out front. Worth memorising outright: **3 dB** is double the power, **6 dB** is double the amplitude, **10 dB** is ten times the power, **20 dB** is ten times the amplitude.

And because they are logarithms, *they add*. A +19 dBm transmitter through a 20 dB attenuator arrives at -1 dBm. No multiplication anywhere. That is the whole reason the unit survives.

UNIT	RATIO AGAINST	WHERE YOU MEET IT HERE
dB	nothing — a bare ratio	"20 dB of attenuation", "70 dB of suppression"
dBm	1 milliwatt	"+19 dBm output" — an absolute power you could meter
dBFS	the converter's full scale	"-20 dBFS" — always negative, meaningless off this converter
dBc	the carrier	"71 dBc image rejection" — bigger is cleaner
dBm/Hz	1 mW, in one hertz of bandwidth	"-174 dBm/Hz" — a <i>density</i> , not a power

Mixing these up is the most common unit error in radio. dBm is a power. dBFS only means something relative to one particular converter. dBc only means something relative to one particular signal. A density in dBm/Hz is not a power until you multiply it by a bandwidth.

Where noise actually comes from

Three sources stack up, and on this board they arrive in this order of importance:

1. **Thermal noise.** Charge carriers jiggle at any temperature above absolute zero, and that jiggle is a voltage. It sets an absolute floor no design can beat: **-174 dBm per hertz** at room temperature. Over 5 MHz of bandwidth that is $-174 + 10 \cdot \log_{10}(5 \times 10^6) = -107 \text{ dBm}$.
2. **The receiver's own noise**, quantified as **noise figure** (NF) — how many dB worse than that ideal floor the hardware actually is. Every amplifier and mixer adds some. The AD9361 is roughly 3–6 dB depending on gain setting and band.
3. **Quantisation noise** from the converter — the $6.02N + 1.76 \text{ dB}$ limit from lesson 9. On this board it is usually the smallest of the three, which is the point of having 12 bits.

THE CONSEQUENCE THAT SURPRISES PEOPLE

The noise floor depends on **bandwidth**. Halve the bandwidth you are listening to and you halve the noise power — 3 dB better signal-to-noise, for free, having changed nothing whatever about the signal.

That is the same fact as decimation's process gain (lesson 27) and as the FFT-length effect in lesson 24. All three are one principle wearing different clothes: *noise is spread out, signal is not, so narrowing your view favours the signal*. It is also why "listen to less spectrum" is almost always the first thing to try when a weak signal will not come out of the mud.

SNR is not one number — it is a number and a bandwidth

"The SNR is 30 dB" is an incomplete statement, in exactly the way "it is 20 degrees" is incomplete without saying Celsius. The honest form names the bandwidth it was measured in. A tone measured in a 15 kHz FFT bin and the same tone measured across 5 MHz of receiver differ by $10 \cdot \log_{10}(5\,000\,000 / 15\,000) = 25\text{ dB}$, with nothing about the radio having changed.

THREE NOISE NUMBERS THAT ARE NOT COMPARABLE, AND GET COMPARED ANYWAY

Tone SNR in an FFT bin

What `sdr_selftest.py` reports. Flattering, because the bin is narrow. Always quoted with the transform length.

SNR across the occupied bandwidth

What a demodulator actually experiences. This is the number that predicts whether a link works.

Noise figure

A property of the hardware alone, independent of bandwidth and of what you are receiving. It does not become an SNR until you supply both.

Quoting the first where the second was meant is how a link that "has 70 dB of SNR" fails to decode.

Reading your own board

A measured example from this hardware — cable loopback at 900 MHz, 5 MSPS, TX2A through a 20 dB pad into RX2A:

QUANTITY	VALUE	MEANING
Signal	−16.7 dBFS	Comfortably below clipping, comfortably above the floor
Noise floor	−53 dBFS	In a 4096-point FFT — the length is part of the figure
Carrier SNR	71 dB	The usable dynamic range in <i>that</i> measurement
Thermal limit, 5 MHz	−107 dBm	What no receiver of any price could beat

Check yourself: you drop from 20 MSPS to 200 kSPS. What happens to your SNR, and why?

It improves by about **20 dB**. You narrowed the bandwidth by a factor of 100, so you are collecting one hundredth of the noise power — $10 \cdot \log_{10}(100) = 20 \text{ dB}$ — while the signal inside that band is unchanged.

The catch: this only holds if the narrowing is done with a *proper filter*. Simply throwing samples away folds all the out-of-band noise back in and you gain precisely nothing — which is the defect lesson 15 documents on this board's channel 1.

26 Filters, from first principles

Almost everything you will build in the fabric is a filter or contains one. The idea is simpler than the notation suggests.

A **filter** passes some frequencies and rejects others. In the digital world you do that with arithmetic: each output sample is a weighted sum of the most recent input samples.

a 4-tap FIR, conceptually

VERILOG

```
out[n] = c0*in[n] + c1*in[n-1] + c2*in[n-2] + c3*in[n-3];
```

That is a **FIR** filter — Finite Impulse Response, so called because if you feed it a single spike, the output stops after a finite number of samples (four, here). The weights `c0..c3` are **taps** or coefficients, and they are the entire design: choosing them chooses which frequencies survive.

WHY THIS FILTERS ANYTHING AT ALL

Averaging neighbouring samples smooths a signal — rapid wiggles cancel, slow trends survive. That is a low-pass filter. Subtracting neighbours does the reverse, emphasising change: a high-pass filter. Every other response is a more carefully chosen set of weights between those extremes. Nothing more mysterious is happening.

What it costs in fabric

A filter with N taps needs N multiplications per output. Multiplication is expensive in LUTs, which is why the chip has 220 dedicated **DSP slices**. A naive 100-tap filter would need 100 of them — nearly half your budget for one filter. The next lesson is about why that estimate is usually far too pessimistic.

FIXED POINT, AND THE TWO WAYS IT GOES WRONG

The fabric has no floating point. Coefficients are stored as scaled integers, which means two failure modes to keep in mind. **Overflow**: sums grow, so a 16×16 multiply needs 32 bits and an accumulator needs more; truncate carelessly and a loud signal wraps around into noise.

Quantisation: rounding the coefficients changes the response, usually by filling in the stopband — a filter designed for 80 dB of rejection may deliver 50 with 12-bit coefficients.

27 Decimation, and why taps are nearly free

The single most useful trick in fabric DSP, and the reason the channelizer in this repo is affordable.

Decimation means producing one output for every D inputs — lowering the sample rate by a factor of D. You must filter first, or the discarded bandwidth aliases back in (lesson 12). So decimation is always a filter plus a throw-away.

Here is the trick. If you only produce one output every D input samples, the hardware has **D input-sample periods to compute each output**. One multiplier can therefore be reused D times. A 128-tap filter decimating by 8 needs roughly 16 multipliers, not 128.

WHAT THAT CHANGES ABOUT YOUR DESIGN CHOICES

Taps become close to free in a decimating filter, and that should change what you optimise for. This repo's coefficient generator designs a Kaiser-windowed sinc — the simple, textbook approach that needs no toolbox — rather than chasing the roughly 30% tap saving an equiripple design would give. Thirty percent more taps costs almost nothing here; the simplicity is worth more than the saving.

Why sample at 61 MSPS only to throw most of it away?

It looks wasteful. Capture 56 MHz of spectrum, decimate by 307, and keep a 200 kHz channel — you discarded 99.7% of what you collected. Why not just tell the AD9361 to sample at 200 kSPS in the first place? It *can*; its analogue filter goes down to 200 kHz.

Sometimes that is exactly the right answer. But there are four reasons the wide-then-decimate route wins, and the first one is worth real money.

1. You gain dynamic range — about four extra bits

The converter's own quantisation noise is roughly fixed in total power, and it is spread evenly across the whole sampled bandwidth. Filter down to a narrow slice and you keep all of your signal but only a *fraction* of that noise. The improvement is $10 \cdot \log_{10}(D)$ decibels, where D is the decimation factor.

DECIMATION	RATE OUT	PROCESSING GAIN	WORTH
4	15.36 MSPS	6 dB	1 bit
32	1.92 MSPS	15 dB	2.5 bits
307	200 kSPS	24.9 dB	~4 bits

The AD9361's converter is 12 bits. Decimating by 307 gets you the effective dynamic range of a 16-bit converter in that channel, for free, in fabric you have already paid for. This is **oversampling**, and it is why high-end converters sample far faster than the signal needs.

2. A digital filter is far sharper than the analogue one

The AD9361's analogue filter is a handful of poles: a gentle slope, a corner that moves with temperature, and a response you cannot know exactly. A 321-tap FIR gives you 80 dB of stopband rejection, a transition band a few percent wide, and *exactly* linear phase — a response you designed and can verify. If a strong transmitter sits 300 kHz from the signal you want, only the digital filter will save you.

3. You can retune without touching the radio

Changing the AD9361's local oscillator takes time, disturbs its calibration, and breaks phase continuity. Changing the frequency of a digital mixer in the fabric (lesson 28) is a single register write, effective on the next sample, with the phase relationship preserved. Capture wide once, then move around inside that capture instantly and as often as you like.

For anything phase-sensitive — direction finding, coherent multi-channel work, interferometry — this is not a convenience, it is the only workable approach.

4. One capture, many channels

The local oscillator gives you one place at a time. A wide capture can be split into as many narrow channels as you have fabric for, all simultaneously, all phase-coherent with one another. That is what a **channelizer** is, and it cannot be done by tuning.

WHEN TO JUST SAMPLE NARROW INSTEAD

If you know exactly where your signal is, will never need to move, do not need the extra dynamic range, and have no strong neighbours to reject — set the AD9361 to a low rate and a narrow bandwidth and be done. It is simpler, it uses no fabric, and it is the right engineering answer for a fixed single-channel receiver.

The wide-then-decimate route earns its complexity when you need **dynamic range, selectivity, agility, or several channels at once**. If none of those apply, do not build it.

Check yourself: why does decimating by 8 not need eight times the hardware?

Because you only produce one output for every eight inputs, so the hardware has eight clock periods to compute each one — and a single multiplier can be used eight times over in that window.

A 128-tap filter decimating by 8 therefore needs roughly 16 multipliers, not 128. That reuse is what makes taps nearly free, and it is why the generator in this repo favours a simple design with more taps over a clever one with fewer.

The worked example in this repo

FILE	ROLE
<code>firmware/scripts/gen_fir_coe.py</code>	Designs and verifies the coefficients. Standard library only — no MATLAB, no numpy.
<code>coefile_wbfm_102100.coe</code>	Its output: 321 taps.
<code>system_bd.tcl</code>	Wires the FIR IP in and points it at that file.
<code>patches/optional/0003-*.patch</code>	The whole change, opt-in.

It channelises wideband FM: filter and decimate in the fabric so the host receives a narrow, already-clean stream instead of the full firehose.

COEFFICIENTS COUNT AS AN HDL CHANGE

Edit the `.coe` and you must delete the Vivado project, exactly as for a Verilog edit. Otherwise the FIR IP is regenerated from cache with the *old* taps, and you will spend an afternoon measuring a filter you did not design.

DO NOT ENGAGE THE FPGA'S ÷8 TRANSMIT INTERPOLATOR

It exists in the stock design and it does not work on this board: upstream's `tx_upack` read-enable ORs in channel 1's DAC valid, and this board runs 2R2T. Measured result — TX1 emits nothing at all, indistinguishable from a muted transmitter to within 1.2 dB.

When checking "is it transmitting", always compare against a *muted reference in absolute dBFS*. A normalised plot once made that silence look like a spray of components.

28 Mixers, oscillators and CORDIC

How to move a signal in frequency without touching the radio's tuning.

Multiplying a signal by a complex exponential shifts it in frequency. That is all a **mixer** is. Multiply your baseband by $e^{j2\pi ft}$ and the whole spectrum slides up by f ; use a negative f and it slides down.

To do that you need a source of sine and cosine at an arbitrary frequency — a **numerically-controlled oscillator** (NCO). The classic implementation is a **phase accumulator**: a counter that adds a fixed step every sample, so its value ramps and wraps, representing an angle. Feed that angle into a lookup table of sine values and you have an oscillator whose frequency is set by one number.

phase accumulator

VERILOG

```
reg [31:0] phase;
always @(posedge clk)
    if (valid) phase <= phase + step;    // step sets the frequency

// frequency = step / 2^32 * sample_rate
// the accumulator wraps naturally, which is exactly what an angle does
```

A lookup table costs block RAM and its size grows with the precision you want. **CORDIC** is the alternative: an algorithm that computes sine, cosine and rotation using only shifts and adds, no multipliers and no table. It takes one iteration per bit of precision, which pipelines beautifully in fabric. For frequency shifting, rectangular-to-polar conversion, or computing magnitude and phase, CORDIC is usually the right answer on an FPGA.

YOU ALREADY USED ONE TODAY

The tone generator inside this board's transmit core is exactly this — an NCO in the fabric. It is why a full-rate transmit test costs no host bandwidth at all: the waveform is computed on the chip, sample by sample, instead of being shipped there.

29 What modulation actually is, and where to put it

The lesson that decides your whole architecture: stream the information, not the samples.

To send data you map bits onto something measurable. The standard scheme is to choose, for each **symbol** period, one point from a fixed set — a **constellation** — and transmit a pulse of that amplitude and phase.

SCHEME	POINTS	BITS/SYMBOL	TRADE
BPSK	2	1	Most robust, slowest.
QPSK	4	2	The workhorse.
16-QAM	16	4	Twice the data, needs ~7 dB more SNR.
OFDM	dozens of subcarriers at once		Handles echoes well; high peak-to-average ratio.

Between symbols you cannot simply jump — an abrupt step spreads energy across the whole spectrum. So each symbol is shaped by a **pulse-shaping filter**, usually a root-raised-cosine, which confines the signal to its allotted bandwidth while still letting the receiver recover each symbol cleanly.

The architecture question

So: should sophisticated modulation at high rates always live in the fabric? Not quite. The useful question is **what you send down the wire to the board**.

STREAM BITS, NOT SAMPLES

A QPSK signal at 61.44 MSPS with 4 samples per symbol carries 15.36 million symbols per second — about **3.8 MB/s of actual information**. The same signal as I/Q samples is **245 MB/s**. Identical content, sixty-four times the bandwidth, because you are shipping the *shape* of the wave instead of its *meaning*.

Put the modulator in the fabric and you send it the 3.8 MB/s. The fabric does the pulse shaping and the rate conversion — the part that is simple, repetitive, and fast. That is the whole argument, and it is why the boundary usually falls exactly there.

The three honest options

APPROACH	GOOD FOR	LIMIT
Stream samples from the host	Arbitrary, unique, one-off waveforms. Easiest by far — write a file, send it.	The pipe. On this board roughly 30 MB/s total, so about 5 MSPS if you need transmit and receive simultaneously.
Cyclic buffer	Anything <i>periodic</i> : test tones, beacons, radar chirps, calibration signals. Load once, hardware repeats forever, full rate, no HDL needed.	The waveform repeats. No good for unique data.
Modulate in the fabric	Unique data at rates the pipe cannot carry; anything needing deterministic timing or a reaction in microseconds.	You have to write and verify HDL, and a build is twenty minutes.

Note that "sophisticated" and "fast" are separate axes. A complex scheme at a modest rate streams perfectly well from a PC, where it is far easier to write and debug. A simple scheme at a very high rate belongs in fabric. The genuinely hard case is complex *and* fast — and there the answer is to split it: the clever, slow, irregular parts (coding, framing, adaptation) stay on the processor; the dumb, fast, regular parts (pulse shaping, interpolation, mixing) go in the fabric. That partition is what almost every real radio does.

TRY IT

Build a QPSK modulator in the fabric: take two bits per symbol from an AXI register, map them to one of four points, pulse-shape with an interpolating FIR, and feed the result into the transmit path. You will have used lessons 8, 13, 19, 20 and 21 at once — and you will have moved the bandwidth bottleneck by a factor of sixty.

30 Pulse shaping, and why root-raised-cosine

A symbol is a number. A radio has to turn it into a shape. Which shape you pick decides how much spectrum you occupy and whether your symbols smear into each other — and the standard answer is strange enough to be worth deriving rather than copying.

The obvious idea, and why it fails

Say you want to send the QPSK symbols from lesson 29 at 1 Msym/s. The obvious thing is to hold each symbol steady for a microsecond and then jump to the next: a train of rectangles.

It fails for a reason you can see in lesson 24's terms. A rectangle in time is a **sinc** in frequency — skirts that fall off as $1/f$ and never truly end. Your 1 Msym/s signal, which ought to need about 1 MHz, is still radiating measurably 20 MHz away, into somebody else's band. Sharp edges in time are wide in frequency; there is no way around it.

THE TWO DEMANDS THAT FIGHT EACH OTHER

Be narrow in frequency

so you do not splatter over the neighbours, and so the receiver can filter away everything that is not you.

Be clean in time

so that when the receiver samples at the middle of symbol n , the tails of symbols $n-1$ and $n+1$ contribute exactly nothing.

Smearing between symbols has a name — **ISI**, intersymbol interference — and it closes the eye of the constellation just as effectively as noise does. Sharpen the pulse in frequency and you lengthen it in time, which creates ISI. Shorten it in time and you widen it in frequency. Pulse shaping is the negotiated settlement.

Nyquist's trick: zero at the sampling instants

The insight is that a pulse does not have to be short. It only has to be **zero at every other symbol's sampling instant**. Between those instants it can do whatever it likes, because nobody looks there.

The family of pulses with that property is called **Nyquist pulses**, and the one everybody uses is the **raised cosine**. It has a single knob, the **roll-off** β , between 0 and 1:

B	BANDWIDTH NEEDED	TIME-DOMAIN TAILS	IN PRACTICE
0	exactly R_s — the theoretical minimum	ring on forever, decay as $1/t$	unbuildable; brutally sensitive to timing error
0.22	$1.22 \times R_s$	modest	what 3G/UMTS chose
0.35	$1.35 \times R_s$	short, well behaved	the common default — and what this repository's test waveforms use
1	$2 \times R_s$	very short	wasteful of spectrum, very forgiving of timing

So β buys timing tolerance with bandwidth. **Occupied bandwidth** = $(1 + \beta) \times \text{symbol rate}$ is the one formula to carry out of this lesson.

Why *root* raised cosine, and why it is split in half

Here is the part that confuses everyone the first time. The pulse that must satisfy the Nyquist condition is the one measured **end to end** — transmitter, channel and receiver combined. So you do not put a raised cosine in the transmitter. You put a **root raised cosine** (RRC) in the transmitter and an identical RRC in the receiver. Two square roots multiply back to the raised cosine you wanted, and you get it only after the receiver has done its half.

SPLITTING IT IS NOT A COMPROMISE — IT IS STRICTLY BETTER

The receiver's copy of the RRC is also the **matched filter** for the transmitted pulse: the filter shaped exactly like the signal you are looking for. Lesson 32 shows why that shape, and no other, maximises signal-to-noise at the decision instant.

So the split gives you two things at once — zero ISI end to end, *and* the best possible SNR at the receiver. That is why every digital standard you have heard of does it this way.

How you actually build one

Mechanically, pulse shaping is **upsampling followed by an FIR filter** — lesson 27's decimator run backwards:

1. Take one complex symbol per symbol period.
2. Insert $\text{sps}-1$ zeros after each one. This is the **upsample** step; sps is samples per symbol, and 4 is the usual choice.
3. Run the result through the RRC filter. Its taps fill in the zeros with the pulse shape, so what comes out is a smooth waveform at $\text{sps} \times$ the symbol rate.

The filter is defined over a **span** of symbols — how many symbol periods of tail you keep before truncating. Span 10 at 4 samples per symbol is **41 taps**, which is what the test waveforms in this repository use.

scratchpad/rf/signals.py — the whole shaper

PYTHON

```
up = np.zeros(nsym*sps, dtype=complex)
up[::sps] = syms           # one symbol, then sps-1 zeros
h = rrc(beta=0.35, sps=4, span=10) # 41 taps
x = np.convolve(up, h)      # the shaped waveform
```

Three lines. In the fabric it is the same three ideas — a strobe that fires once per symbol, a multiply-accumulate chain, and the polyphase decomposition of lesson 27 so the multipliers are not wasted on the zeros you just inserted.

THE BANDWIDTH YOU CALCULATE IS NOT THE BANDWIDTH YOU MEASURE

The QPSK waveform in `docs/modulation-and-throughput.md` runs at 61.44 MSPS with 4 samples per symbol, so $R_s = 15.36$ Msym/s, and $(1 + 0.35) \times 15.36$ predicts **20.7 MHz**. The measurement says **17.96 MHz**.

Neither is wrong. The formula gives the bandwidth at which the raised-cosine skirt reaches zero; the measurement is **99 % occupied bandwidth**, the span containing 99 % of the power, and the last 1 % lives out in those skirts. Any bandwidth figure is a number *and* a threshold. Compare two that used different thresholds and you will conclude something false.

SEE THE ISI FOR YOURSELF

Regenerate the QPSK test waveform with `beta=0.35` and again with the RRC replaced by a plain rectangle, and run both through `analyse.py`. The rectangle's constellation points smear into short radial streaks instead of tight clusters — that streak *is* ISI, and no amount of transmit power removes it.

Check yourself: you need 2 Msym/s of 16-QAM inside a 2.5 MHz channel. What roll-off can you afford?

$(1 + \beta) \times 2 \leq 2.5$, so $\beta \leq 0.25$.

That is tight but ordinary — 0.22 is a real standard's choice. The price is paid in timing sensitivity: the tails are longer, so the receiver's sampling instant has to be more accurate, and lesson 31's timing recovery has to work harder. Note also that the constellation choice does not enter the calculation at all. Bandwidth is set by the *symbol* rate; 16-QAM simply carries four bits in each of those symbols instead of two.

31 Synchronisation: finding the signal inside the samples

A receiver does not get symbols. It gets a stream of numbers with the symbols hidden at unknown times, unknown phase and unknown orientation. Recovering them is a whole discipline — here is enough of it to avoid the classic traps.

Three unknowns, in order

UNKNOWN	WHY IT EXISTS	WHAT IT DOES IF IGNORED
Timing — when is a symbol?	Your sample clock and the transmitter's are unrelated, and the cable adds delay.	You sample between symbols and get a smeared, unusable constellation.
Carrier phase — which way is up?	The transmit and receive oscillators have no agreed phase reference.	The constellation is rotated by an arbitrary angle.
Carrier frequency — is it drifting?	Two independent oscillators are never exactly equal.	The constellation spins.

ON A LOOPBACK, ONE OF THE THREE IS FREE

Transmit and receive here share a single 40 MHz reference, so their synthesisers produce genuinely equal frequencies. Measured on this board with the standard trick — raise a QPSK signal to the fourth power and look for a tone at four times the offset — the frequency error came out at **0.0 Hz**.

That is a property of a loopback, not of radio. Over the air with two separate boxes you would have a real offset to track, and that is what carrier recovery loops are for.

Timing recovery

Matched-filter the incoming stream, then decide which sample within each symbol period is the right one. The classic detectors are **Gardner** and **Müller & Mueller**, both feedback loops that nudge the sampling instant until an error measure settles.

For an offline capture you can do something simpler and completely robust: try every candidate phase and pick the best by a quality measure. For a constant-envelope modulation like QPSK, the correct instant is the one where the recovered symbols have the *least* spread in magnitude:

brute-force timing search

PYTHON

```
# z is the matched-filtered stream, sps samples per symbol
best = None
for phase in range(sps * OVERSAMPLE):
    s = z[phase::sps * OVERSAMPLE]
    # 1/kurtosis: 1.0 means perfectly constant magnitude
    q = mean(abs(s)**2)**2 / mean(abs(s)**4)
    if q > best: best, best_phase = q, phase
```

That quality number is diagnostic in itself. A clean lock reads **1.000**; a value near **0.5** means the symbols look Gaussian — which is what you get when the data is genuinely corrupted rather than merely mistimed. It is how the DMA-starvation problem on this board was first identified.

Phase ambiguity, and the bug it causes

QPSK's four points are symmetric under 90° rotation. So the standard way to remove an unknown phase is to raise the symbols to the fourth power — which cancels the data — take the angle, and divide by four.

It works, and it leaves you with a constellation that could still be rotated by any multiple of 90° , because the fourth power threw that information away. That residue is **phase ambiguity**, and real systems resolve it with a known preamble or by encoding data in phase *changes* rather than absolute phase.

THE 45° VERSION, WHICH COST REAL TIME HERE — TWICE

Estimating phase as `angle(mean(s4))/4` leaves the constellation sitting at $0^\circ, 90^\circ, 180^\circ, 270^\circ$. But a QPSK reference lattice conventionally sits at $45^\circ, 135^\circ, 225^\circ, 315^\circ$. Compare the two and every symbol is a half-quadrant from where it should be.

The resulting error vector magnitude is about **76%** — which looks exactly like a broken transmitter, not like a maths error. On this board it was hit twice, on two different estimators, and both times the radio was blamed first.

The check that catches it instantly: run your demodulator against the clean file you transmitted. If it does not read close to 0%, the bug is yours. Make that reflexive.

Check yourself: your EVM is 76% and the spectrum looks perfect. What do you suspect?

The analysis, not the radio. A spectrum that looks right means the signal reached the receiver with its shape intact; 76% is close to the exact value you get from a 45° lattice mismatch ($|e^{j45^\circ} - 1| \approx 0.765$).

Genuine corruption degrades the spectrum too. Clean spectrum plus terrible constellation is almost always a synchronisation or convention error.

32 Correlation, matched filters, and finding the packet at all

Lesson 31 recovered timing and phase once you knew a signal was there. This is the step before it: deciding that something is there, and where it starts, in a stream of numbers that mostly is not. It is also the single operation this board's fabric is best at.

Correlation, in one sentence

Slide a known template along the received samples, and at each offset multiply the two together point by point and add up the result. Where the template lines up with a real copy of itself, every product is positive and the sum shoots up. Everywhere else the products are a random mix and the sum stays small.

That is all **correlation** is — and lesson 24's intuition for the Fourier transform was the same operation with a sine wave as the template. Detection, demodulation and spectrum analysis are three uses of one idea.

WHY THE ANSWER IS A FILTER, NOT A SEARCH

Correlating against a template is *identical* to running an FIR filter whose taps are that template, time-reversed and complex-conjugated. So you do not write a search loop. You write the filter from lesson 26, load different coefficients, and read its output.

That filter has a name — the **matched filter** — and a property no other filter has: among every possible filter, it produces the highest signal-to-noise ratio at the moment the template lines up. This is why lesson 30's receiver-side RRC is not an arbitrary choice.

Where the gain comes from, and how much

Add up N samples of signal that are all lined up and the amplitudes add directly: N times bigger. Add up N samples of noise and they partly cancel, growing only as $\sqrt{N}$. The ratio improves by $N / \sqrt{N} = \sqrt{N}$ in amplitude — which is $10 \cdot \log_{10}(N)$ dB in power.

A 64-sample preamble therefore buys **18 dB**. A 1024-sample one buys **30 dB**. That is how a signal sitting below the noise floor is found at all: you do not see it, you accumulate it.

It is the *same* trade as every other one in this half of the book — longer look, narrower effective bandwidth, less noise. Lesson 24 called it FFT length, lesson 25 called it noise bandwidth, lesson 27 called it process gain. Here it is called correlation gain.

What makes a good preamble

The template you correlate against is usually a **preamble** — a known sequence sent at the front of every packet. It needs one property: it must look like itself at zero offset and like nothing at all at every other offset. That property is its **autocorrelation**, and the ideal shape is a single sharp spike, which is why engineers call it a thumbtack.

SEQUENCE	WHAT IT IS	WHY YOU WOULD PICK IT
Barker	Short ± 1 codes, lengths up to 13	Off-peak never exceeds $1/N$. Used by 802.11b at length 11.
m-sequence / PN	Output of a shift register with feedback	Any length you like, near-ideal thumbtack, and free in fabric — it is a shift register.
Zadoff-Chu	A complex sequence of constant amplitude	Perfect autocorrelation and a flat spectrum. LTE's synchronisation signal.
A repeated half	The same block sent twice	Lets the receiver correlate the signal <i>against itself</i> — no template needed. See lesson 33.

FREQUENCY OFFSET EATS YOUR CORRELATION LENGTH

Coherent accumulation assumes the phase stays put across the window. A frequency offset rotates it: over N samples at rate f_s , the phase turns by $2\pi \cdot \Delta f \cdot N / f_s$. Once that reaches half a turn the second half of your window cancels the first, and a longer preamble makes detection *worse*.

The working rule is to keep the rotation under about a tenth of a turn: $N < f_s / (10 \cdot \Delta f)$. Two untuned AD9361s can sit tens of kilohertz apart, so at 5 MSPS and 10 kHz of offset you have roughly **50 samples** of coherent window — not 1024, however much gain you wanted. Past that you must correlate in shorter blocks and add the magnitudes, which is more robust and costs several dB of the gain you were after.

A CORRELATION PEAK IS NOT A DETECTION

A raw peak grows with the input level, so a strong nearby interferer produces a bigger peak than your actual packet and trips any fixed threshold. Divide the correlator output by the energy in the same window before comparing. The normalised value runs 0 to 1, means "how much of what I am seeing looks like my template", and lets a threshold survive a 40 dB change in signal level.

Then pick the threshold from the false-alarm rate you can live with, not from one capture that worked.

Why this belongs in the fabric

Lesson 29 argued for pushing work into the fabric when the input rate is huge and the output rate is tiny. Detection is the textbook case: 61.44 million samples per second go in, and a handful of "packet starts here" events come out. Better still, the coefficients never change, so the multipliers can be specialised:

TEMPLATE	COST PER TAP	A 64-TAP COMPLEX CORRELATOR
Arbitrary complex taps	4 real multiplies	~256 DSP48s — will not fit on this board's 220
± 1 taps (BPSK preamble)	an add or a subtract	0 DSP48s — an adder tree in LUTs
$\pm 1, \pm j$ taps (QPSK)	an add, a subtract and a swap	0 DSP48s

So choosing a ± 1 preamble is not a theoretical nicety. It is the difference between a correlator that fits in this XC7Z020 and one that does not.

THE SAME FILTER IS A RADAR

Matched filtering a chirp against its own template is called **pulse compression**, and it is how every modern radar gets both long range and fine resolution: send a long chirp for energy, compress it to a short spike for timing. The chirp waveform measured in `docs/modulation-and-throughput.md` is already the transmit half. Correlating the capture against the transmitted chirp is a one-line change to `analyse.py` — and it is project 4 in lesson 44 for a reason.

Check yourself: your link has -10 dB SNR and needs $+8$ dB to decode. How long a preamble, and what does that assume?

You need 18 dB of gain, so $N = 10^{1.8} \approx 64$ samples.

It assumes the phase holds still across all 64. At 5 MSPS that means the frequency offset must be under about $f_s/(10 \cdot N) = 7.8$ kHz. Two free-running AD9361s will not reliably be that close, so a real design either corrects frequency coarsely first, or splits the 64 into blocks no longer than that and adds the magnitudes, which costs a few dB of gain to buy immunity from the rotation.

33 OFDM, and why this board finds it hard

Wi-Fi, LTE, 5G, DVB-T and DAB all use it, so it is worth understanding properly rather than as a black box. It is also the one waveform this board measurably struggles with — and the reason why is the most useful thing in this lesson.

The problem it solves

A signal arriving by two paths — direct, and bounced off a wall — arrives twice, slightly apart. That is **multipath**, and the delay between the first and last meaningful echo is the **delay spread**. Indoors it is tens to hundreds of nanoseconds; outdoors it can be microseconds.

Two echoes add, and at some frequencies they add in phase while at others they cancel. The channel is no longer a flat attenuation — it is a comb, with notches. Engineers call this a **frequency-selective** channel.

A single-carrier receiver fixes this with an equaliser long enough to span the delay spread. At 20 Msym/s and 1 μ s of spread that is a 20-tap adaptive filter that must be continuously retrained, and the cost grows with the square of the bandwidth. It works, and it is horrible.

THE OFDM BARGAIN

Instead of one carrier at 20 Msym/s, send **64 carriers at 312.5 ksym/s each**. Each one is now so narrow that the channel across it is a single complex number — an amplitude and a phase — rather than a shape.

Equalisation collapses from an adaptive filter to **one complex multiply per subcarrier**. That is the entire trade, and everything else in OFDM is machinery to make it legal.

The FFT *is* the modulator

Here is the part that feels like a trick. You do not build 64 oscillators. You write your 64 symbols into the 64 bins of a frequency-domain array and take an **inverse FFT**. What comes out is the time-domain waveform with all 64 carriers already on it, correctly spaced and correctly summed. The receiver takes a forward FFT and reads the symbols straight out of the bins.

Lesson 24's transform, run backwards, is the transmitter. This is why OFDM became practical exactly when the FFT became cheap, and why it is a good fit for an FPGA.

scratchpad/rf/signals.py — one OFDM symbol

PYTHON

```
X      = np.zeros(64, dtype=complex)
X[idx] = qpsk_symbols          # 52 used bins; DC and the edges left empty
x      = np.fft.ifft(X) * np.sqrt(64)
out    = np.r_[x[-16:], x]     # cyclic prefix: the last 16 copied to the front
```

Four lines, and the parameters are 802.11a's: a 64-point FFT, 52 subcarriers carrying data, a 16-sample cyclic prefix. The unused bins are not waste — the edge ones are a guard band so the signal does not splatter, and the DC bin is left empty because a direct-conversion receiver like this one has its own LO

leakage sitting exactly there (lesson 11).

Orthogonality, and why the spacing is not negotiable

The subcarriers overlap. They do not interfere anyway, because each one fits a *whole number of cycles* into the symbol period, so when the receiver's FFT sums over that period every other subcarrier integrates to exactly zero. That is **orthogonality**, and it forces the spacing: $\Delta f = 1 / T_{\text{symbol}}$.

It is lesson 24's leakage argument in reverse. Leakage happens when a signal does *not* fit a whole number of cycles into the FFT window. OFDM arranges for every signal to fit exactly, and gets zero leakage as a result. Break the arrangement — a timing error, a frequency offset, a sampling-clock mismatch — and the subcarriers leak into each other. That failure has its own name, **ICI** (inter-carrier interference), and OFDM is notoriously more sensitive to frequency offset than single-carrier ever was.

The cyclic prefix: the bit that looks like waste

Copy the last 16 samples of the symbol to the front and send 80 samples instead of 64. You have just thrown away 20 % of your capacity. It buys two things, and both are essential:

1. **Echoes land in the guard, not in the next symbol.** As long as the delay spread is shorter than the prefix, the smeared tail of symbol n falls inside the prefix of symbol $n+1$, which the receiver discards. ISI between symbols disappears.
2. **The channel becomes a multiply.** Because the prefix makes the symbol look periodic, the channel's linear convolution behaves like a *circular* convolution over the FFT window — and a circular convolution in time is a plain multiplication in frequency. That is the theorem that makes the one-tap equaliser correct rather than approximate.

So the prefix length is a direct statement about the environment you expect: **CP duration > delay spread**, and anything longer is capacity you are giving away.

Equalisation, in the only place it is easy

Scatter a few **pilot** subcarriers of known value through the grid, or send one known symbol at the start of the packet. The receiver divides what it got by what it knows was sent, and that quotient is the channel at that subcarrier: $H = Y / X$. Interpolate across the pilots for the rest, then recover every data subcarrier with $\hat{X} = Y / H$.

One complex division per subcarrier per symbol. That is the whole equaliser — the thing that costs an adaptive filter and a training sequence in a single-carrier receiver. Lesson 34 is about what that single-carrier receiver has to do instead, and it is a great deal more work.

WHY OFDM ALWAYS ARRIVES WITH ERROR-CORRECTING CODE ATTACHED

The flip side of narrow subcarriers is that a channel notch does not degrade everything a little — it destroys a few subcarriers completely, and dividing by a near-zero H amplifies noise enormously. No amount of transmit power helps.

The fix is forward error correction plus **interleaving**: scramble the coded bits across subcarriers so that a wiped-out group becomes a scatter of single-bit errors spread thinly through the codeword, which the code then repairs. This is why the standards call it **COFDM** — coded OFDM — and why an uncoded OFDM link like the test waveform here is not what any real system ships. Coding typically buys 4–6 dB, which is why a link budget (lesson 38) is allowed to count on it.

Measured on this board, and the honest explanation

At 61.44 MSPS over the 20 dB cable loopback, the seven test waveforms in `docs/modulation-and-throughput.md` give QPSK 2.17 % EVM, 16-QAM 2.24 % — and OFDM between 13 % and 51 %, run to run. That looks like a broken implementation. It is not, and the measurement that proves it is worth copying.

TRANSMIT BACKOFF	RX RMS	OFDM EVM
0 dB	−29.2 dBFS	13.4 %
−6 dB	−35.1 dBFS	19.6 %
−12 dB	−40.4 dBFS	28.4 %
−18 dB	−44.3 dBFS	38.0 %

It gets monotonically **worse** as the signal weakens. That is the signature of a noise-limited link. Had it been clipping, backing off would have *improved* it. So the cause is not the transmitter compressing; it is that OFDM simply delivers less average power.

PAPR: THE PRICE OF ADDING 52 THINGS TOGETHER

Sum 52 independent subcarriers and, by the central limit theorem, the result is very nearly Gaussian — which means occasional large peaks. Measured on this board the OFDM waveform's **PAPR** (peak-to-average power ratio) is **11.06 dB** against **4.14 dB** for QPSK.

A transmitter is limited by its *peak*. At the same peak, OFDM therefore puts about **7 dB less average power** on the link than QPSK does. Same noise floor, less signal, worse EVM — entirely as measured, and nothing to do with the modulation being implemented badly.

WHAT WAS RULED OUT, AND HOW

- **The analysis itself.** Run against the clean transmit file the demodulator reads 0.00 %. Always do this first.
- **Band-edge filter roll-off.** Per-subcarrier EVM is nearly uniform — only 1.2× worse at the edges than the centre. A filter problem would be dramatically edge-weighted.
- **Cyclic prefix too short.** Sweeping it from 16 to 128 samples barely moved the number. On a cable there is no delay spread to guard against.
- **The AD9361's tracking loops.** Turning DC and quadrature tracking off made it worse, not better.

Four hypotheses, four measurements, one survivor. This is the shape a real investigation takes, and it is worth more than the answer.

If you wanted to build it in the fabric

Xilinx ships an FFT core; a 64-point pipelined-streaming complex FFT runs comfortably at 61.44 MSPS and costs a handful of DSP48s and one BRAM. The transmit chain is: symbol mapper → write bins → IFFT → prepend prefix → the packer of lesson 15. The receive chain is that reversed, plus the synchroniser.

And that synchroniser is where the work actually is. The standard trick is Schmidl & Cox: send a symbol whose two halves are identical, and have the receiver correlate the incoming stream *against a delayed copy of itself*. The correlation peaks when the two halves line up, giving both the symbol boundary and — from the phase of the correlation — the frequency offset. It needs no template at all, which makes it exactly lesson 32's machinery pointed at the signal's own structure.

Check yourself: indoors the delay spread is 200 ns. At 61.44 MSPS, is a 16-sample cyclic prefix enough?

16 samples at 61.44 MSPS is $16 / 61.44 \text{ MHz} = 260 \text{ ns}$. So yes — just, with 60 ns of margin, and no margin at all if the room is larger than you assumed.

Note how the answer depends on the *sample rate*, not on the FFT size. Halve the rate to 30.72 MSPS and the same 16 samples become 521 ns of protection, because every sample now lasts twice as long. It is one of the few places in OFDM where slowing down genuinely buys robustness rather than just costing throughput.

34 Equalisation: undoing a channel that is not flat

OFDM sidestepped the problem by making every subcarrier narrow enough that the channel is one number. A single-carrier receiver cannot do that, and has to undo the channel directly. This is the filter that does it — and, unusually, it designs itself.

What a channel does to your signal

Multipath (lesson 33) means the receiver gets several delayed copies added together. Written as samples, that is a **convolution** — the same operation as the FIR filter of lesson 26, except that nobody chose the coefficients:

what arrives

THE CHANNEL, AS A FILTER

$$y[n] = x[n]*h[0] + x[n-1]*h[1] + x[n-2]*h[2] + \dots + \text{noise}$$

| |
what you sent the echo, half as strong, one sample late

The channel is an unwanted FIR filter sitting in your signal path. So the fix is another FIR filter that undoes it. The whole subject is that sentence plus the three complications that follow from it.

THREE WORDS BEFORE WE GO ON

Tap	One coefficient of a filter. A "7-tap equaliser" remembers seven samples.
Pre-cursor / post-cursor	Energy from a symbol that lands <i>before</i> its own sampling instant, and after it. A symbol-spaced equaliser needs taps on both sides.
Convergence	An adaptive filter starts wrong and improves. Convergence is how long that takes, measured in symbols.

Complication one: you cannot simply invert it

The obvious equaliser is $1/H$ — divide by the channel and everything cancels. It is called the **zero-forcing** equaliser, and it works beautifully on paper.

The trouble is the notches. Where two echoes cancel, H is nearly zero, and $1/H$ is enormous. Your equaliser faithfully removes the ISI there and multiplies the noise by a hundred while doing it. Zero-forcing has zero residual ISI and can have appalling signal-to-noise.

The standard answer is **MMSE** — minimum mean-square error — which minimises *total* error instead, and so deliberately leaves a little ISI in the deep notches rather than amplifying the noise there. Every practical equaliser is an MMSE equaliser.

Complication two: you do not know the channel, and it changes

So the filter has to find its own coefficients while running. The workhorse is **LMS** — least mean squares — and it is three lines:

LMS, per received sample	THE WHOLE ALGORITHM
<pre> y = dot(w, x) # filter: w is the taps, x the last N samples e = d - y # error: d is what the symbol should have been w = w + mu * e * conj(x) # nudge every tap toward reducing that error </pre>	

Each tap is pushed in whichever direction would have reduced this sample's error, by an amount proportional to the error. Run it for a few hundred symbols and the taps settle into the MMSE solution on their own. Nobody solved anything; the filter descended a hill.

μ , the step size	Too large and the taps overshoot and diverge into nonsense. Too small and the equaliser is still converging when the packet has ended. Normalised LMS divides μ by the input power, which makes a single choice work across a 40 dB range of signal levels and is almost always what you want.
RLS	Recursive least squares: converges in tens of symbols rather than hundreds, at roughly N^2 arithmetic per sample instead of N . Worth it when the packet is short; rarely worth it in an FPGA.

Complication three: where does *d* come from?

The error needs to know what the symbol should have been. Three answers, used in this order:

MODE	WHERE <i>D</i> COMES FROM	WHEN
Training	A known sequence sent at the start of the packet	First. Reliable, but costs airtime.
Decision-directed	The nearest constellation point to what came out	After training, for the rest of the packet. Free, but only works once the eye is already open — feed it garbage and it confidently converges to garbage.
Blind (CMA)	Nothing. The constant-modulus algorithm just pushes every output toward a fixed magnitude	When there is no training sequence and the modulation has constant amplitude (PSK). Slow, and blind to phase.

THE DECISION-FEEDBACK EQUALISER, AND WHY IT IS TEMPTING

A **DFE** splits the job in two: a normal feed-forward filter cleans up the pre-cursor ISI, and a second filter subtracts the post-cursor ISI using the symbols already *decided* — which are clean, noiseless constellation points.

Because the feedback path carries decisions rather than noisy samples, it cancels ISI without amplifying noise at all. That is a real advantage in a deep-notch channel. The price is **error propagation**: one wrong decision is fed back and helps produce the next wrong decision. DFEs are excellent at high SNR and can fall apart at low SNR, which is exactly the opposite of what you want under stress.

Where it goes on this board

This is the architecture point, and it is the same one lesson 29 made. An adaptive equaliser costs **two** complex multiply-accumulate chains per tap — one to filter the samples, one to update the taps — so a modest 11-tap complex equaliser is **88 real multiplies per input sample**. Where you put it decides what that costs:

RUNNING AT	MULTIPLIES PER SECOND	DSP SLICES, TIME-SHARED ON THE 100 MHZ CLOCK
61.44 MSPS, in front of the decimator	5.4 G	~55 of the 148 free
2 Msym/s, behind it	176 M	~2

Both fit. But one of them spends a third of the device's arithmetic on a filter that did not need to run that fast, and leaves nothing for the rest of your design. Equalisation belongs **after** the decimation and the timing recovery, at one or two samples per *symbol* — at which point it is two DSP slices, or a comfortable software job on the ARM cores. Put the wideband work in the fabric and the clever, low-rate, iterative work behind it.

TWO WAYS AN EQUALISER QUIETLY DESTROYS A WORKING LINK

It adapts on noise

Between packets there is no signal, only noise, and LMS will happily converge to whatever minimises the error on noise — which is nonsense. Gate adaptation on the detector from lesson 32: no correlation peak, no updating.

It fights the carrier recovery

A slowly rotating phase (lesson 31) looks to the equaliser exactly like a channel that is slowly changing, so both loops chase the same error and can oscillate against each other. The usual fix is to make the carrier loop much faster than the equaliser, so the equaliser sees a phase that has already stopped moving.

MAKE YOURSELF A CHANNEL, BECAUSE THE CABLE HAS NOT GOT ONE

A 20 dB pad and half a metre of coax is very nearly a perfect channel — there is nothing to equalise, which makes it useless for testing an equaliser. So synthesise one: convolve the QPSK test waveform with `[1, 0.5]` before transmitting it and the EVM in `analyse.py` will jump dramatically. Then run a 5-tap LMS over the received symbols and watch it come back down. You will have built a channel, broken your link with it, and repaired the link in software — which is the entire subject in one afternoon.

Check yourself: why does an OFDM receiver need only one tap per subcarrier when a single-carrier receiver needs eleven?

Because the number of taps you need is set by **delay spread measured in symbol periods**. Split 20 Msym/s into 64 subcarriers and each symbol lasts 64 times longer, so the same 500 ns of echo is now a small fraction of one symbol rather than ten of them. There is nothing left to span.

You paid for it, though — with the cyclic prefix, with tight frequency-offset tolerance, and with 11 dB of PAPR. OFDM did not abolish the work; it moved it somewhere cheaper.

35 Channel coding: buying decibels with redundancy

Lesson 33 said coding is worth 4 to 6 dB and lesson 38 spends them. This is where they come from, why they are real, and what they cost — because a decibel bought with arithmetic is a decibel you did not have to buy with a bigger amplifier.

The idea, and the exchange rate

Send more bits than the message needs, arranged so that the extra ones let the receiver work out which of the received bits are wrong and repair them. The ratio is the **code rate** $R = k/n$: rate 1/2 means every 1 message bit goes out as 2 transmitted bits.

What you get back is **coding gain** — the reduction in signal-to-noise needed for the same error rate. It is measured in decibels and it is entirely real: a rate-1/2, constraint-length-7 convolutional code with soft decisions buys about 5 dB. Against lesson 38's budget that is the difference between a link that works and one that does not, and it costs a few thousand LUTs rather than a bigger amplifier and a licence.

WHY THERE IS A LIMIT, AND ROUGHLY WHERE IT IS

Shannon's capacity formula says a channel of bandwidth B and signal-to-noise ratio S/N can carry at most $C = B \cdot \log_2(1 + S/N)$ bits per second, error-free, with a good enough code. Two things follow.

First, error-free is achievable at all — which was a shock in 1948 and is why anybody looked for these codes. Second, there is a floor: below about **-1.6 dB** of energy per bit against noise density, no code of any kind works. Modern turbo and LDPC codes operate within roughly 1 dB of that floor, so this is a nearly finished subject, and "invent a better code" is not the answer to your link problem.

Soft decisions, and the free 2 dB most people throw away

Your demodulator produces a point on the constellation. Rounding it to the nearest symbol and handing the decoder a bit is a **hard decision**. Handing it the distance as well — "this is probably a 1, but only just" — is a **soft decision**, usually expressed as a **log-likelihood ratio** (LLR).

Soft decisions are worth about 2 dB over hard ones, for free, because the decoder can weigh a confident bit against an uncertain one instead of treating them alike. Two design consequences, both easy to get wrong:

- Your demodulator must output LLRs, *not* bits. A slicer in the middle of the chain throws the information away permanently.
- LLRs need the noise level to be scaled correctly, so a receiver that measures its own SNR (lesson 37) decodes better than one that does not.

The families, and which one to reach for

CODE	WHAT IT IS	TYPICAL GAIN	WHERE YOU MEET IT
CRC	Detects errors; corrects nothing	—	Every packet ever. It tells the receiver to <i>discard</i> , which is what makes a retry protocol possible (lesson 36).
Hamming, BCH	Algebraic block codes over bits	2–3 dB	Headers, and anywhere short and cheap beats strong.
Reed–Solomon	Algebraic, over <i>bytes</i>	3–5 dB	Bursts. RS(255,223) repairs any 16 corrupted bytes wherever they fall. CDs, QR codes, DVB.
Convolutional + Viterbi	A shift register and an optimal search back through its states	~5 dB soft	The workhorse. K=7, rate 1/2 is 802.11a, GPS and a hundred others.
Turbo, LDPC	Iterative: two decoders passing probabilities back and forth until they agree	within ~1 dB of Shannon	LTE, 5G, Wi-Fi, DVB-S2. Heavy, and worth it when every decibel counts.
Polar	Provably capacity-achieving, good at short lengths	near Shannon	5G control channels.

For a first link on this board the honest answer is **convolutional, rate 1/2, K=7, soft Viterbi**. It is the best gain-per-unit-effort in the table, it is what the textbooks work through, and Xilinx ships the decoder as IP.

INTERLEAVING, OR THE CODE DOES NOTHING

Every code in that table is designed against *scattered* errors. Real channels produce *bursts* — a fade, an interferer, a wiped-out group of OFDM subcarriers in a notch (lesson 33). A burst of 40 consecutive bad bits defeats a code that could have repaired 40 scattered ones easily.

An **interleaver** fixes this by writing the coded bits into a block in one order and reading them out in another, so that bits which were adjacent in the codeword are far apart on the air. The burst is spread thinly across many codewords, each of which sees a handful of errors it can repair. Coding and interleaving are not two options; below about 1–2 dB of fade they are one mechanism, and the code without the interleaver is close to useless.

THE WATERFALL, AND WHY A CODE CAN BE WORTH NOTHING AT ALL

Error rate against SNR for a coded link is not a gentle slope. It is flat and terrible, then falls off a cliff over about 1 dB, then is essentially perfect. That cliff is the **waterfall**.

Above it the code is worth its full 5 dB. Below it the code is worth *nothing* — in fact slightly less than nothing, because you spent half your throughput on it. So "add coding" does not rescue a link that is 10 dB short. It rescues one that is 3 dB short, decisively.

In the fabric

The encoder is trivial — a shift register and two XOR trees, a handful of LUTs, no DSP slices at all. The decoder is where the work is.

Viterbi, $K=7$	64 states, and for each one an <i>add-compare-select</i> per bit plus a traceback memory. A few thousand LUTs and a BRAM or two, easily fast enough at symbol rate on this part. This is one of the classic reasons FPGAs exist.
Reed–Solomon	Finite-field arithmetic: awkward in software, natural in LUTs. Also available as IP.
LDPC	Hundreds of parallel messages iterating five to fifty times. It fits on this part only in small, slow configurations; it is what the big Zynqs and the ASICs are for.

And as always (lesson 29), put it at symbol rate behind the decimation, not at 61.44 MSPS in front of it.

WHAT THIS MEANS FOR THE OFDM MEASUREMENT IN LESSON 33

The OFDM test waveform in this repository is **uncoded**, and it measures 13 % to 51 % EVM over the cable. No real OFDM system transmits that way — 802.11a, LTE and DVB-T all pair OFDM with convolutional or LDPC coding *and* an interleaver, precisely because narrow subcarriers in a notch fail completely rather than gracefully.

So read that measurement as what it is: a clean characterisation of the *physical layer*, with the layer that normally rescues it deliberately absent.

Check yourself: your link is 3 dB short. Rate-1/2 coding gives 5 dB of gain — do you take it?

Usually yes, but check what you are paying with. Rate 1/2 doubles the transmitted bits, so you either **halve the data rate** at the same bandwidth, or **double the bandwidth** at the same data rate — and doubling the bandwidth costs you 3 dB of noise floor (lesson 25), eating more than half the gain you just bought.

So: keep the bandwidth and accept half the throughput, and you are 2 dB ahead. Keep the throughput and widen the signal, and you are only about 2 dB ahead as well but now occupy twice the spectrum. A weaker code — rate 3/4, about 3.5 dB — is often the better trade, which is exactly why every standard ships several rates and switches between them.

36 Packets: framing, addressing, and everything above the symbols

Every lesson so far has ended at "and now you have symbols". Nothing so far has said who the symbols are for, where they start, whether they arrived intact, or what to do when they did not. That is this lesson, and on this board one number in it decides your whole architecture.

The layers, named once

Physical layer (PHY)	Symbols on and off the air. Lessons 29 to 35 are all PHY.
Media access (MAC)	Whose turn it is to transmit, framing, addressing, acknowledgements, retries. This lesson.
Everything above	Routing, ordering, congestion, applications. A different book, and usually somebody else's code.

The boundary that matters here is the one between MAC and PHY, because it is also the boundary between "can live on the host" and "cannot".

What a frame is made of

a serviceable frame, in order of appearance				ON THE AIR
[preamble]	[sync word]	[header]	[payload]	[CRC]
AGC settles timing and carrier recovery	"packet starts HERE"	length, type, address	the actual bytes, scrambled and coded	did any of this arrive intact?

Preamble	A known pattern long enough for the receiver's gain control, timing recovery and carrier recovery (lesson 31) to converge before anything important arrives.
Sync word	The correlation template from lesson 32. The preamble gets the loops locked; the sync word says exactly which sample the frame begins on.
Header	How long the payload is, what it contains, who it is for.
CRC	A checksum. It corrects nothing — it tells the receiver whether to believe the frame, which is what makes every retry scheme possible.

THE HEADER NEEDS ITS OWN PROTECTION, AND ITS OWN CRC

Corrupt one bit of the length field and the receiver reads the wrong number of symbols: it either truncates a good frame or runs off the end into noise, and then mis-frames everything that follows. A single bit error becomes a lost *sequence* of frames.

So every real standard codes the header more strongly than the payload and gives it a separate short CRC, checked before anything is done with the values in it. 802.11's SIGNAL field is exactly this. It is the cheapest robustness you will ever add.

Scrambling: the step that looks pointless and is not

Payloads contain long runs of zeros. Sent literally, a run of identical symbols means a waveform with no transitions — and lesson 31's timing recovery feeds on transitions, so it drifts and loses lock in the middle of your own data. A constant symbol also puts a spike in the spectrum where you wanted a flat one, and a DC offset where this receiver already has LO leakage (lesson 11).

The fix is a **scrambler**: XOR the data with a pseudo-random sequence from a shift register (the same object as lesson 32's m-sequence) before transmitting, and XOR with the same sequence at the far end. Nothing is hidden — the sequence is public — but the transmitted stream now looks random whatever the payload was. Timing recovery gets its transitions, and the spectrum stays flat.

Whose turn is it? Media access on a shared medium

SCHEME	THE RULE	WHAT IT NEEDS
ALOHA	Transmit whenever. Retry after a random delay if unacknowledged.	Almost nothing. Collapses above ~18 % utilisation.
CSMA	Listen first; transmit only if the channel is idle.	A receiver that can report "busy" in <i>microseconds</i> .
TDMA	Everyone gets a numbered slot.	Shared time. Much easier than CSMA for one designer with two radios.
FDMA	Everyone gets their own frequency.	Spectrum. Trivial here — this board's transmit and receive local oscillators are <i>independent</i> , so it can transmit on one frequency and listen on another at the same time.

THE NUMBER THAT DECIDES YOUR ARCHITECTURE: TURNAROUND TIME

CSMA and acknowledgements both need the radio to go from "heard the end of that frame" to "transmitting a reply" quickly. 802.11 allows **16 μ s**.

Now measure this board's host path. A capture buffer of 262 144 samples at 5 MSPS is **52 milliseconds** of signal — and nothing in it can be examined until the buffer completes, with the network stack and the scheduler still to come. That is more than three orders of magnitude too slow, and no amount of tuning closes a gap that size.

So: a **latency-sensitive MAC cannot live on the host**. Carrier sense, acknowledgement timing and slot boundaries belong in the fabric, where the answer is available the cycle after the samples arrive. The host gets the payloads. This is lesson 29's argument arriving from a completely different direction, and it is the single most important thing in this lesson.

Retries, and what makes them possible

Stop-and-wait	Send a frame, wait for an acknowledgement, send the next. Simple, and its throughput is capped by the round trip: one frame per turnaround, however fast the link.
Sliding window	Keep several frames in flight and acknowledge by sequence number. Needed the moment the round trip is long compared with a frame — which, on this board's host path, it always is.

Sequence numbers Not optional. A lost acknowledgement causes a re-send of a frame that *did* arrive, so without numbering the receiver silently duplicates data.

WHAT THIS BOARD GIVES YOU, AND WHAT IT DOES NOT

The transmit DMA is cyclic (lesson 16), which means the hardware will replay a buffer forever with no help from the host. That is perfect for a **beacon** — a frame repeating on a fixed schedule — and it is why every full-rate measurement in `modulation-and-throughput.md` could be taken at 61.44 MSPS when streaming gave up at 5.

It is useless for a *conversation*, because a cyclic buffer by definition carries no new information. The moment you want to send different bytes each time, you are back to feeding the DMA from software, back under the 31 MB/s plateau, and back to tens of milliseconds of latency. Knowing which of those two worlds your design lives in is the first decision, not the last.

THE SMALLEST USEFUL THING TO BUILD

A one-way beacon: preamble, sync word, a 1-byte length, 32 bytes of payload, a CRC-16. Generate it in Python, shape it (lesson 30), load it once with `iio_writedev -c` and let the hardware repeat it. Then write the receiver: correlate for the sync word (lesson 32), slice, check the CRC, print the payload.

No media access, no retries, no turnaround — and it is a complete radio link, end to end, that you understand every layer of. Everything else in this lesson is what you add when there are two of them.

Check yourself: why can a beacon run at 61.44 MSPS on this board while a two-way link struggles at 5?

Because a beacon's bytes never change, so the transmit DMA's cyclic mode replays one buffer with the host out of the loop entirely — no samples cross the network at all once it is loaded.

A two-way link needs new payload bytes for every frame, and it needs to *react* to what it hears. So every sample crosses the 31 MB/s bottleneck in both directions, and every decision waits for a buffer to complete. The rate limit and the latency limit are the same constraint seen twice, and the way out of both is to move the reacting part into the fabric.

37 Measuring what you built

Six numbers describe an RF system. Knowing which one answers your question — and how to get it honestly from this board — is what separates a measurement from a screenshot.

THE VOCABULARY, ONCE

- SNR** Signal to noise ratio. Wanted power over noise power, in dB. The basic one.
- THD** Total harmonic distortion — how much energy appears at multiples of your test tone. Measures *non-linearity*, not noise.
- SINAD** Signal to noise *and* distortion. SNR and THD combined into one honest number, because in practice both degrade you.
- ENOB** Effective number of bits. SINAD expressed as converter bits, via the same $6.02N + 1.76$ relationship from the fixed-point lesson. Says what your 12-bit converter is *really* delivering.
- SFDR** Spurious-free dynamic range — the gap between your signal and the largest single unwanted spike. What decides whether a weak signal is findable next to a strong one.
- EVM** Error vector magnitude. For modulated signals, the average distance from where symbols landed to where they should be, as a percentage. The summary figure for a link.

Which one answers which question

YOUR QUESTION	MEASURE
Can I hear a weak signal at all?	SNR, and the noise floor
Can I hear it <i>next to a strong one</i> ?	SFDR
Is my amplifier being driven too hard?	THD, or a two-tone IMD3 test
What is my converter actually worth?	ENOB
Will this modulation decode?	EVM
Is my IQ balance any good?	Image rejection

Measured on this board

Through a cable from TX2 to RX2 with a 20 dB attenuator, at 900 MHz:

FIGURE	RESULT	CONDITIONS
Carrier SNR	71–90 dB	CW tone, depending on rate
Image rejection	50–76 dBc	improves at higher rates
IMD3	56–70 dBc	two-tone
EVM, QPSK	1.3–2.2%	5 to 61.44 MSPS
EVM, 16-QAM	1.6–2.3%	same

THREE WAYS TO MEASURE A NUMBER THAT IS NOT TRUE

- **A test tone at a simple fraction of the sample rate.** Quantisation error then correlates with the signal and piles into harmonics, so SFDR reads far worse than the hardware is. Use an awkward, ideally prime, frequency.
- **Too short an FFT.** More samples means more bins, so less noise per bin and a better apparent noise floor. Always say how long the transform was — a figure without it is not comparable to anything.
- **A normalised plot.** Normalising hides absolute level. On this board a *muted* transmitter once looked like "a spray of components" purely because the plot was normalised. Compare against a muted reference in absolute dBFS.

Check yourself: SNR is excellent but EVM is terrible. What is wrong?

Not noise — something structural. Likely candidates: a synchronisation or convention error in your demodulator (see the previous lesson), non-linearity distorting the constellation without raising the broadband floor, or IQ imbalance skewing it.

SNR only measures power ratios. EVM measures whether the symbols are actually *where they should be*, and plenty of impairments move symbols without adding noise.

38 Link budgets: will it actually work over the air?

Everything so far has run down a cable. The moment an antenna is involved there is one calculation that decides whether the link exists at all — and because every term is in decibels, it is a single column of addition.

The whole calculation

the link budget		ALL TERMS IN DB
received power	= TX power + TX antenna gain - cable loss - path loss + RX antenna gain	[dBm]
noise floor	= -174 + 10*log10(bandwidth_Hz) + noise figure	[dBm]
SNR	= received power - noise floor	[dB]
margin	= SNR - SNR the demodulator needs	[dB]

If the margin is positive the link works in the conditions you assumed. How positive it needs to be is the whole art, and we will get to it.

Path loss, and what it really is

In free space, a transmitted wave spreads over an ever-larger sphere, and a receiving antenna catches an ever-smaller share. Nothing is absorbed; the energy is simply somewhere else. That is **free-space path loss**:

free-space path loss	DB
FSPL = 32.44 + 20*log10(distance_km) + 20*log10(frequency_MHz)	

Two consequences fall straight out of the two 20s. **Double the distance and you lose 6 dB. Double the frequency and you lose another 6 dB.** The second one surprises people — the air is not more absorbent at 2.4 GHz than at 900 MHz. A fixed-gain antenna is simply physically smaller at the higher frequency, so it catches less of the sphere.

FREE SPACE IS THE OPTIMISTIC CASE, AND YOU ARE RARELY IN IT

FSPL assumes nothing between the antennas — no ground, no walls, no trees. Real environments obey a steeper law, usually written as distance raised to some exponent n :

ENVIRONMENT	EXPONENT n	LOSS PER DOUBLING OF DISTANCE
Free space	2.0	6 dB
Open outdoor, ground reflection	2.5–3	7.5–9 dB
Indoors, same floor	3–4	9–12 dB
Indoors, through floors	4–6	12–18 dB

At 100 m, an exponent of 3.5 instead of 2 costs you an extra 30 dB. This single term is why link budgets that look comfortable on paper fail in a building.

ANTENNA GAIN IS NOT AMPLIFICATION

What it is	A statement about <i>shape</i> . An antenna with 6 dBi of gain radiates four times the power of an isotropic radiator in its favoured direction — by radiating less everywhere else. Nothing is added; it is redistributed.
dBi vs dBd	dBi is referenced to an ideal isotropic point radiator; dBd to a half-wave dipole. dBd + 2.15 = dBi. Datasheets mix them, deliberately, because dBi is the bigger number.
The consequence	Gain is directivity, so it comes with a beamwidth. A 20 dBi antenna is a wonderful thing until something moves.

How much SNR does the demodulator actually need?

Roughly, for an uncoded link at a bit error rate around 10^{-5} , measured across the occupied bandwidth:

MODULATION	BITS PER SYMBOL	SNR NEEDED (UNCODED)	ROUGH EVM EQUIVALENT
BPSK	1	~10 dB	30 %
QPSK	2	~13 dB	22 %
16-QAM	4	~20 dB	10 %
64-QAM	6	~26 dB	5 %
256-QAM	8	~32 dB	2.5 %

Every extra bit per symbol costs roughly **6 dB** — the same 6 dB per bit as lesson 9's converter formula, and for the same reason: you are halving the distance between decision points. Forward error correction (lesson 33) hands 4–6 dB of that back, which is why every real system uses it.

A worked example on this hardware

900 MHz, 100 m of clear line of sight, a 2 dBi whip at each end, 5 MHz of bandwidth, QPSK:

TERM	VALUE	WHERE IT CAME FROM
TX power	+19 dBm	This board flat out — a capped estimate, never metered
Antenna gains	+4 dB	2 dBi each end
Free-space path loss	-71.5 dB	$32.44 + 20\log_{10}(0.1) + 20\log_{10}(900)$
Received power	-48.5 dBm	the sum of the three above
Thermal noise in 5 MHz	-107.0 dBm	$-174 + 10\log_{10}(5 \times 10^6)$
Noise figure	+5 dB	AD9361, typical
Noise floor	-102.0 dBm	
SNR	53.5 dB	
QPSK needs	13 dB	table above
Margin	+40.5 dB	comfortable — in free space

Forty decibels sounds like the question is settled. Put the same link inside a building with a path exponent of 3.5 and you lose 30 dB of it; add a body standing in the way and another 10; and the margin is gone. **The budget tells you whether a link is plausible, not whether it works.**

FADE MARGIN, AND WHY 10 DB IS THE SMALLEST NUMBER WORTH PLANNING FOR

Multipath means the received level is not steady — it fades as things move, sometimes deeply. A link designed with exactly zero margin is down half the time by construction. Practical designs carry 10–20 dB of fade margin on top of everything above, and mobile systems carry more.

If you cannot afford it, the honest moves are: narrow the bandwidth (lesson 25 — 3 dB per halving), drop to a simpler modulation (6 dB per bit), add coding (4–6 dB), lengthen the preamble (lesson 32), or accept a shorter range. Adding transmit power is usually the one option you do not have.

BEFORE THE ANTENNA GOES ON

Everything in this course so far has been a cable and a 20 dB pad. An antenna makes your signal everybody's problem, and most of this board's tuning range belongs to somebody with a licence — including the FM broadcast band, which it covers happily. The board reaches about +19 dBm, which is not a toy.

Test into a dummy load or a cable first. Then check what you are allowed to transmit, where, and at what power, before anything radiates. The safety rules in the repository's `rf-safety.md` apply to your receiver; the law applies to everybody else's.

Check yourself: the same link at 2.4 GHz instead of 900 MHz. What changes, and by how much?

Path loss rises by $20 \cdot \log_{10}(2400/900) = 8.5 \text{ dB}$, so the margin falls from +40.5 to +32 dB. Nothing else in the budget moves.

The usual counter is that antennas of the same physical size have more gain at higher frequency — a 10 cm patch is a poor antenna at 900 MHz and a good one at 2.4 GHz — so a real comparison often wins back most of that 8.5 dB at both ends. Frequency by itself is not the enemy; *fixed antenna gain* is the assumption that makes it look that way.

39 Antennas and the RF front end: what happens after the SMA

Lesson 38 stopped at "transmit power" and "antenna gain" as if they were settings. They are hardware, and on this board some of that hardware is missing on purpose. This is the part of the signal chain that no amount of Verilog can fix.

Fifty ohms, and why anything has an impedance at all

At low frequency a wire is a wire. Once the wavelength is comparable to the cable, a signal travelling down it meets the far end and asks what impedance it sees. If that does not match the cable's own **characteristic impedance**, part of the wave **reflects** and comes back.

Everything in radio is standardised on $50\ \Omega$ so that this does not happen: the amplifier, the connector, the cable, the antenna all present $50\ \Omega$ and the wave goes out and never comes back. Fifty is a compromise — around $77\ \Omega$ gives the lowest loss in coax and around $30\ \Omega$ the highest power handling — and the industry split the difference sixty years ago.

THREE NAMES FOR ONE MEASUREMENT

VSWR	Voltage standing wave ratio. 1:1 is perfect, 2:1 is ordinary, 3:1 is poor. It is the ratio of the peaks and troughs of the standing wave a reflection creates.
Return loss	The same thing in decibels: how far below the forward wave the reflected one is. Bigger is better. 2:1 VSWR is 9.5 dB return loss.
S11	The same thing again, as a complex number, from a vector network analyser. Its magnitude is the return loss; its phase tells you <i>which way</i> the mismatch is, which is what lets you design the fix.

At 2:1 VSWR about 11 % of your power comes back — only 0.5 dB of loss, which is why 2:1 is widely accepted. At 3:1 it is 25 %, and at 6:1 half your transmit power is heading back into the amplifier.

The antenna, in the three facts that matter

It is a resonant length

A quarter-wave monopole is about a **quarter of the wavelength**, shortened a few percent because the wire is not infinitely thin. At 900 MHz the wavelength is 333 mm, so the whip is about **78 mm**. Get the length wrong and the impedance stops being $50\ \Omega$, the VSWR climbs, and power reflects.

A monopole needs a ground plane

It is half a dipole; the other half is the mirror image in the ground plane. A whip screwed into a small board with no ground plane is a different, worse antenna than the datasheet's, and this surprises people every time.

Small is bad, and unavoidably so

Make an antenna much smaller than a quarter wave and you can still match it — but only over a narrow band, and with increasing loss. Bandwidth, size and efficiency trade against one another by physical law, not by engineering effort. This is why the antenna in a small device is nearly always the weakest part of the link.

POLARISATION: 20 DB THAT IS NOWHERE IN THE LINK BUDGET

Two whips both vertical are aligned. Turn one horizontal and, in a clean line-of-sight path, the coupling drops by **20 dB or more** — a link budget's entire fade margin, gone, for a reason the budget never mentioned.

If either end moves or tumbles, that is a strong argument for circular polarisation at one end (3 dB of loss always, instead of 20 dB sometimes) or for diversity (lesson 42). Indoors, scattering mixes the polarisations back up and the effect is softer — which is exactly why the failure is worse outdoors, where everything else is better.

The front end, and what this board has

From the schematic's parts list in [docs/hardware.md](#), each RF port's path is short:

this board, per channel		THE RF SIDE
receive	RX1A/RX2A --> balun (T1-T4) --> AD9361 differential RF input	
transmit	AD9361 --> balun --> PGA-102+ (U12/U13, ~15.7 dB) --> TX1A/TX2A	

Baluns, a power amplifier per transmit channel, and the AD9361. **No external band filter in either direction.** The chip's own front end is deliberately wideband — it has to cover 70 MHz to 6 GHz — so what protects a narrowband receiver elsewhere simply is not here. Two consequences follow, and both are practical rather than theoretical.

RECEIVING: A STRONG SIGNAL YOU ARE NOT LISTENING TO STILL RUINS YOUR DAY

An FM broadcast transmitter or a nearby LTE base station arrives at the SMA at full strength whatever you tuned to, because nothing filtered it out. The receiver's front end sets its gain for the loudest thing present, so a strong out-of-band signal pushes the gain down and your wanted signal down with it. The effect is called **desensitisation**, or blocking, and it looks exactly like a weak signal — the noise floor appears to rise for no reason.

The fix is a band-pass filter at the antenna, before anything else. It is the first accessory worth owning, and an SDR that "works on the bench and not in the field" is very often this.

TRANSMITTING: YOUR HARMONICS LEAVE THE BUILDING TOO

A power amplifier is not perfectly linear, so it emits copies of your signal at twice and three times the frequency. Measured on this board those sit at **-64 to -80 dBc** (second) and **-71 to -85 dBc** (third). Into a cable and a load, nobody cares. Into an antenna at +19 dBm, the second harmonic of a 900 MHz signal is a real transmission at 1.8 GHz, in somebody else's band, at up to -45 dBm.

A transmit low-pass filter is what removes it, and this board has none. So the rule from lesson 38 is sharper than it first sounds: before anything radiates, know what you are allowed to transmit, where, at what power — *and at what harmonics*.

Where the noise figure is decided

Cascade several stages and the noise figure of the whole chain is dominated by the first one. Friis's formula says why:

noise figure of a cascade

LINEAR, NOT DB

$$F_{\text{total}} = F_1 + \frac{(F_2 - 1)}{G_1} + \frac{(F_3 - 1)}{(G_1 \cdot G_2)} + \dots$$

|
divided by the first stage's gain

Every later stage's noise is divided by everything in front of it, so a low-noise amplifier with 15 dB of gain at the very front makes the rest of the chain nearly irrelevant. It also explains the rule that looks like folklore: **put the LNA at the antenna, not at the radio**. Ten metres of coax at 1 dB of loss in front of the LNA costs you a whole decibel of noise figure; the same coax *behind* it costs almost nothing.

CABLES, CONNECTORS, AND THE REASON EVERY MEASUREMENT HERE SAYS "EXACTLY 20 DB"

- **Use a real attenuator.** This board's receive input is rated to about **+2.5 dBm** and its transmitter reaches about **+19 dBm**. A loopback without a pad destroys the receiver, once, permanently.
- **Exactly 20 dB, not more.** With a bigger pad the board's own internal transmit-to-receive leakage — measured as an equivalent 58 to 77 dB of isolation below 1 GHz, and only 33 to 51 dB at 3 to 6 GHz — starts to compete with the signal through the cable, and you measure the board's crosstalk instead of your link.
- **SMA connectors are a torque spec, not a hand-tight fitting.** A loose connector is an intermittent mismatch, and it looks like a flaky radio.

Check yourself: your 900 MHz link works on the bench through a cable and fails on antennas at 30 m. Name four suspects, in order.

1. **Polarisation.** Free, instant to test, and worth 20 dB. Line the whips up.
2. **Path loss is not what you assumed.** 30 m indoors at an exponent of 3.5 rather than 2 is about 22 dB worse than the free-space figure lesson 38 gives you.
3. **Blocking.** No front-end filter, so anything strong nearby is desensitising the receiver. Look at a wide spectrum capture before assuming your own signal is weak.
4. **The antennas themselves.** Wrong length, no ground plane, or a poor match — and the mismatch costs you at *both* ends.

Notice that none of the four is a bug in your FPGA, your modulation or your code. When a link fails after the connector, it is nearly always after the connector.

40 IQ files, and the metadata that turns them into measurements

A capture is a pile of numbers with no units, no frequency and no date. Six months later that is not a measurement, it is a file you are afraid to delete. This lesson is about the five minutes of work that prevents it.

What is actually in the file

`iio_readdev` writes raw samples to stdout with no header of any kind. The layout is exactly the packer's output from lesson 15, little-endian 16-bit signed integers, interleaved:

one receive channel enabled	4 BYTES PER SAMPLE
I0 Q0 I1 Q1 I2 Q2 ... # int16 LE, I then Q	

both receive channels enabled	8 BYTES PER SAMPLE INSTANT
I0 Q0 I1 Q1 I0 Q0 I1 Q1 ... -- instant 0 -- -- instant 1 -- # ch0 I, ch0 Q, ch1 I, ch1 Q	

There is no marker between channels and no marker between samples. If you enable channels in a different order, or forget that you enabled two, the file still opens and the numbers still look plausible — they are simply somebody else's signal. Nothing in the file will tell you.

THE SAME INT16 MEANS TWO DIFFERENT THINGS ON THIS BOARD

Receive

The AD9361's converter is **12-bit**, delivered sign-extended into an int16. Full scale is ± 2047 , not ± 32767 .

Transmit

The DAC takes the **top 12 bits** of the int16 you hand it, so you fill the whole ± 32767 range and the hardware discards the low nibble — the same nibble the GPIO feature in `docs/tx-gpio-bitmap.md` routes to the header pins.

Divide a receive capture by 32768 instead of 2048 and every absolute level you quote is **24 dB** too low, consistently, so nothing looks obviously wrong. This is the single most common way to publish a confidently incorrect dBFS figure from this board.

How big is it going to be?

Four bytes per sample per channel, and no compression anywhere. At full rate that is a firehose.

What the file does not say, and needs to

Open a bare `.bin` a year later and every one of these is unrecoverable from the data itself:

MISSING	WHY IT MATTERS
Sample rate	Every frequency axis you plot is wrong by the ratio you guessed
Centre frequency	Offsets in the file are relative to an LO you no longer know
Gain, and whether AGC was on	No absolute level can be reconstructed
Which channel	RX1 and RX2 differ by 1.5 dB on this board, and by more at 3–6 GHz
Date and time	You cannot correlate it with anything else that happened
Whether the fabric decimator was engaged	Changes the rate by 8 <i>and</i> , on stock firmware, whether channel 1 is trustworthy at all (lesson 15)

SigMF: a sidecar file, and the end of the problem

SigMF — the Signal Metadata Format — is a convention, not a library. You rename the capture `x.sigmf-data` and write a small JSON file called `x.sigmf-meta` next to it. Nothing about the samples changes, so every tool that read the bare file still reads it, and now the file explains itself.

capture.sigmf-meta	JSON
<pre>{ "global": { "core:datatype": "ci16_le", # complex, int16, little-endian "core:sample_rate": 7680000, "core:version": "1.0.0", "core:hw": "Fishball17020 / PlutoSky, AD9361, RX2A", "core:description": "10 MHz DDS tone, fabric decimator engaged, 20 dB pad" }, "captures": [{ "core:sample_start": 0, "core:frequency": 900000000, "core:datetime": "2026-09-23T04:20:00Z" }], "annotations": [{ "core:sample_start": 0, "core:sample_count": 262144, "core:freq_lower_edge": 2200000, "core:freq_upper_edge": 2450000, "core:label": "alias of the 10 MHz tone" }] }</pre>	

global	Facts true of the whole file: the number format, the sample rate, what recorded it.
captures	One entry per retune or restart, each marking the sample index it begins at. A band sweep is one data file with many capture segments.
annotations	Labelled boxes in time and frequency. This is where "the alias is here" lives — a note you can act on rather than a sentence in a commit message.

```
import json, os, datetime

def sigmf(binpath, fs, lo, desc, hw="Fishball17020 / AD9361"):
    base = os.path.splitext(binpath)[0]
    os.rename(binpath, base + ".sigmf-data")
    meta = {
        "global": {"core:datatype": "ci16_le", "core:sample_rate": fs,
                  "core:version": "1.0.0", "core:hw": hw,
                  "core:description": desc},
        "captures": [{"core:sample_start": 0, "core:frequency": lo,
                     "core:datetime": datetime.datetime.now(
                         datetime.timezone.utc).isoformat()}],
        "annotations": []}
    with open(base + ".sigmf-meta", "w") as f:
        json.dump(meta, f, indent=2)
```

Fourteen lines, once, and no capture you take afterwards is ever ambiguous.

RECORD WHAT YOU READ BACK, NOT WHAT YOU WROTE

The rule that governs transmit attenuation on this board applies to metadata generally: a value you wrote is an intention, and a value you read back is a fact. The AD9361 quantises gain to its own table (see [ad9361-gain-tables.md](#)), `rf_bandwidth` snaps to what the filter design supports, and `sampling_frequency` lands on what the clock tree can actually produce.

So build the metadata from an `iio_attr` read taken *after* configuration, not from the numbers in your script. Otherwise the sidecar is a second place to be wrong.

THE TWO HABITS WORTH ADOPTING TODAY

1. **Always capture a muted reference.** Same settings, transmitter at -89.75 dB, a second or two of samples. It is the only way to tell a real weak signal from something your own board is doing — and a normalised spectrum once made pure silence look like a spray of components in this very repository.
2. **Write the sidecar in the same script that takes the capture.** Metadata added afterwards is metadata remembered, and remembering is the part that fails.

Check yourself: a colleague sends you rx.bin, 400 MB, and says "it's the 900 MHz capture". What can you work out, and what can you not?

Can: the file is 400 MB, so at 4 bytes per sample it holds 100 M samples — but only if one channel was enabled; with two it is 50 M sample instants. Relative frequencies within the file are recoverable as fractions of fs, so you can say "the tone sits at $0.163 \times fs$ ".

Cannot: the sample rate, so no frequency in hertz and no duration; the gain, so no absolute level; which channel; whether the decimator was engaged, which changes the rate by eight; and whether "900 MHz" was the LO or where the interesting signal happened to be.

Everything you cannot work out is three lines of JSON. That is the lesson.

41 Two coherent receivers: what this board can do that most cannot

The shared local oscillator looked like a limitation in lesson 14. It is also this board's most interesting capability.

Both receivers run from **one** synthesiser, sample on **one** clock, and are captured on **one** strobe. They are therefore **coherent**: the phase relationship between them is stable and meaningful, not accidental.

That is a much stronger property than "two receivers". Two independent radios tuned to the same frequency drift against each other and give you two recordings. Two coherent receivers give you one measurement with *two viewpoints*, and the difference between those viewpoints carries information.

What the phase difference tells you

A wave arriving at an angle reaches one antenna slightly before the other. That delay appears as a **phase difference** between the channels — and from it you can compute the direction the signal came from.

direction from phase

PYTHON

```
# rx1, rx2: simultaneous captures from the two receivers
phase_diff = np.angle(np.mean(rx2 * np.conj(rx1)))
# d = antenna spacing, lam = wavelength
angle_of_arrival = np.arcsin(phase_diff * lam / (2 * np.pi * d))
```

With antennas half a wavelength apart this is unambiguous across a useful arc — the basis of direction finding, and the first step toward beamforming, where you deliberately add the two channels with a phase shift to steer sensitivity.

COHERENT DOES NOT MEAN CALIBRATED

The two chains have their own amplifiers, their own cables and their own gain settings, so there is a *fixed* phase and amplitude offset between them that has nothing to do with the incoming signal. Measured on this board, the two receivers differ by about **1.5 dB** in sensitivity before any correction.

So calibrate first: feed both inputs the same signal through a splitter, measure the offset, and subtract it. What remains is the part that carries direction.

AND DO NOT ENGAGE THE FABRIC DECIMATOR ON STOCK FIRMWARE

Everything here depends on the two channels being sample-aligned. On the stock design, engaging the $\div 8$ decimator filters channel 0 and not channel 1, which offsets them by the filter's group delay and destroys exactly the relationship you are trying to measure.

Either leave the decimator bypassed, or apply the optional patch that filters both channels in lockstep (lesson 14).

TRY IT

Split one signal into both receive ports with equal-length cables. Capture both channels, compute `angle(mean(rx2 * conj(rx1)))`, and confirm it is stable over time — that stability *is* the coherence. Then lengthen one cable by a known amount and watch the phase shift by the amount the extra delay predicts.

That single experiment is the foundation of every direction-finding and beamforming project you might build on this board.

42 MIMO and beamforming: what more than one antenna buys

Lesson 41 showed that this board's two receivers share a local oscillator, and used the phase between them to find a direction. That is one of three quite different things people mean by "multiple antennas", and it is worth knowing which one you are asking for.

Three things, one name

	WHAT IT DOES	WHAT IT NEEDS	WHAT IT GIVES
Diversity	Two antennas see independent fades; use whichever is better, or combine them	Antennas far enough apart to fade independently	Up to 3 dB of gain, and — far more valuable — most of the fade margin back
Beamforming	Combine with deliberate phase shifts so the array listens or shouts in one direction	Known geometry and calibrated phase	3 dB per doubling of elements, plus rejection of interference from other directions
Spatial multiplexing	Send <i>different</i> data from each antenna at the same time on the same frequency	A rich scattering channel, and $N \times N$ antennas	N times the throughput, with no extra bandwidth or power

Only the third one is "MIMO" in the strict sense. All three are available on this board in some form, and they want opposite things from your antenna spacing — which is the first practical thing to understand.

WHY THE SPACING IS THE DESIGN DECISION

Diversity wants the antennas far apart — several wavelengths — so their fades are uncorrelated. Two antennas 5 mm apart fade together and give you nothing.

Beamforming wants them close and exact — half a wavelength — so that the phase difference maps unambiguously to an angle. At 900 MHz that is **167 mm**.

Go wider than half a wavelength and several angles produce the same phase difference: the array develops **grating lobes** and can no longer tell them apart. Go much narrower and the phase difference shrinks into the noise. You cannot have one spacing that is good at both jobs, so decide which you are building.

Diversity, and the thing that actually rescues links

A fade is deep and local: move half a wavelength and it is gone. So two receivers, separated, rarely fade at the same instant.

Selection combining

Take whichever branch is stronger. Crude, nearly free, and already most of the benefit.

Maximal ratio combining (MRC)

Co-phase the two branches and add them, weighted by their own signal-to-noise. Optimal. With two equal branches it is worth 3 dB of plain gain

— but against *fading* it is worth far more, because the probability that both branches are deeply faded at once is the square of the probability that one is.

That second sentence is the point. In a link-budget sense diversity buys 3 dB; in a *reliability* sense it can be worth 10 to 20 dB of fade margin, which is normally the most expensive part of lesson 38's budget.

Beamforming, in the only maths you need

Two elements a distance d apart, a wave arriving at angle θ from broadside. The far element is $d \cdot \sin \theta$ further away, so its signal is late by that much — which, in phase, is:

the whole of array processing, twice		TWO ELEMENTS
receive (direction finding)	phase difference = $2\pi \cdot d \cdot \sin(\theta) / \lambda$	
transmit (steering a beam)	apply that same phase difference deliberately, and the two waves add in phase toward θ and cancel elsewhere	

At $d = \lambda/2$ that simplifies to $\phi = \pi \cdot \sin \theta$: broadside gives 0° , endfire gives 180° , and everything in between maps one-to-one. Measure ϕ , take the arcsine, and you have the direction. Impose ϕ , and you have a beam.

Two elements give a broad beam and one unavoidable ambiguity. Signals at $+30^\circ$ and -30° produce opposite phase differences and are easily told apart — but a signal 30° in *front* of the array and one 30° *behind* it produce the same phase difference, because $\sin \theta$ is the same for both. A linear array of two isotropic elements simply cannot distinguish them. A third element, a directional element, or moving the array resolves it. Scanning ϕ across all angles and plotting the output is called a **beamscan**; the sharper estimators (MUSIC, ESPRIT) do better by exploiting the fact that noise and signal live in different subspaces, and PySDR chapters 19 to 21 work through them properly.

Spatial multiplexing, and why it is not free here

If the channel between two transmit and two receive antennas is a 2×2 matrix $\mathbf{H}$ that happens to be invertible, the receiver can solve for two independent streams sent simultaneously on the same frequency. Throughput doubles with no extra bandwidth and no extra power. That is 802.11n and everything after it.

The catch is that $\mathbf{H}$ must be well-conditioned, which means the two paths have to be genuinely different — lots of scattering, or antennas far apart with different views. In free space with two whips side by side the two rows of $\mathbf{H}$ are nearly identical, the matrix is close to singular, and inverting it amplifies noise enormously. *Multipath, which every other lesson has treated as the enemy, is the resource that makes spatial multiplexing work at all.*

WHAT THIS BOARD GENUINELY HAS, AND WHAT IT DOES NOT

- **Two receivers on one RX_LO and two transmitters on one TX_LO.** The chains are therefore coherent: their phase relationship is stable, and they share phase noise, so it cancels in the *difference*. That is the expensive property, and at this price it is unusual.
- **Two is not many.** Two elements give a broad beam and a coarse angle. Serious direction finding wants four or eight, and that means several boards locked to one clock.
- **Nothing is calibrated.** The chains differ by about 1.5 dB in receive and 0.1 to 0.25 dB in transmit on the measured unit, and each has its own fixed phase offset through its own balun and traces. Lesson 41 says it and it bears repeating: *coherent does not mean calibrated*.
- **Transmit calibration drifts.** The AD9361's transmit quadrature calibration on this board varies by up to 10 dB run to run, so a transmit beamformer needs re-calibrating far more often than a receive one.

CALIBRATE THE TWO RECEIVERS BEFORE YOU BELIEVE ANY ANGLE

Feed *the same* signal to both receive ports through a splitter and two cables of the same length. Whatever phase difference you measure is the board's, not the world's — it is the sum of the two baluns, the two traces and the two chains. Record it, subtract it from every subsequent measurement, and re-measure it after every retune, because it changes with frequency.

Do this before your first direction-finding experiment rather than after it. The uncalibrated offset is easily tens of degrees, which at $\lambda/2$ spacing is tens of degrees of angle error — enough to make a working algorithm look broken.

Check yourself: you want diversity *and* direction finding from one pair of antennas. Can you?

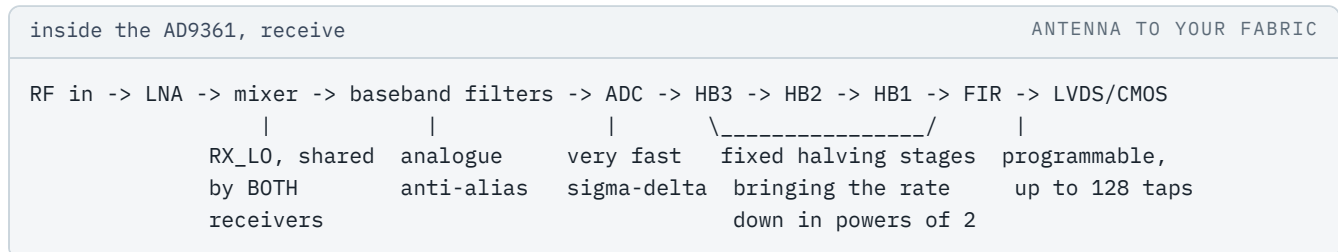
Not well, because they want opposite spacings. Diversity needs the antennas far enough apart to fade independently — several wavelengths — and direction finding needs them at half a wavelength or closer to keep the angle unambiguous.

At a wide spacing you still get a phase difference; it just maps to several possible angles at once, so you would need another constraint — a third antenna, a rough prior, or movement — to resolve which. The usual answer with only two chains is to pick the job, and if you truly need both, to add elements rather than compromise the spacing.

43 The AD9361 itself: the chip in front of your fabric

Every sample your logic has ever seen came out of this chip, already filtered, already decimated, already gain-controlled by something you did not write. Eleven lessons have quietly depended on how it behaves. Time to open it.

The receive path, stage by stage



Two things in that diagram explain most of this course. The mixer uses **one RX_LO for both receivers**, which is why lessons 41 and 42 work and why you cannot tune the two channels separately. And the converter runs far faster than your sample rate, with a chain of halving filters behind it — which is why `sampling_frequency` will not accept arbitrary values.

ZERO-IF, AND THE TWO ARTEFACTS IT GIVES YOU FOR FREE

What it is

The mixer brings your signal straight down to **zero** — the wanted band ends up centred on DC, as I and Q. No intermediate frequency, no image-reject filter, very few parts. It is why an SDR this small is possible at all.

Artefact one: the DC spike

The LO leaks into its own mixer and produces a constant offset, which lands exactly at 0 Hz — in the middle of your signal. Every spectrum in this course has a bump at DC, and it is not a signal.

Artefact two: the image

If the I and Q paths differ at all in gain or phase, a tone at +2 MHz produces a ghost at -2 MHz. How far down that ghost is, is the **image rejection** lesson 37 measures. On this board it is 44 to 60 dBc after a fresh calibration, and it varies by up to 10 dB run to run.

The rate chain, and why the driver says no

Ask for 5 MSPS and the driver does not set anything to 5 MSPS. It picks a converter rate, a path through HB3/HB2/HB1, and a set of FIR coefficients whose product lands on 5 MSPS. Ask for a rate it cannot reach that way and it gives you the nearest one it can.

Below 2.083 MSPS

The half-band chain runs out, so the driver enables the chip's own programmable **128-tap FIR** to get the rest of the way. `pyadi-iio` and the MCP both rely on this and neither touches it — which is the right default, because a hand-written coefficient set that does not match the rate produces a receiver that is quietly wrong rather than obviously broken.

Above that

The FIR is bypassed and the half-bands do the work alone.

With the fabric decimator engaged `sampling_frequency_available` reports exactly two values — `{converter rate, converter rate / 8}` — because that is the whole of what lesson 27's filter offers. The chip's own chain and the fabric's are two separate decimators in series, and only the fabric one is yours.

Gain is not in decibels, whatever the units say

The receive gain attribute reads in dB and behaves like a number of dB — its slope is within 1.7 % of 1.000 dB/dB across 56 measured slopes on this board, which is excellent. But underneath, gain is a **table index**: each step switches a specific combination of LNA, mixer and baseband amplifier settings, chosen by ADI per frequency band.

Two consequences. There are **discontinuities** — places where one step changes noise figure or linearity much more than it changes gain, because the chip switched which amplifier is doing the work. And the table *differs by band*, so a calibration taken at 900 MHz does not transfer to 2.4 GHz. `ad9361-gain-tables.md` in this repository has the measured detail; the short version is to measure gain where you intend to use it.

The calibrations, and the loops that never stop

CALIBRATION	WHAT IT REMOVES	WHEN IT RUNS
Baseband DC	Offset in the analogue baseband	On initialisation
RF DC	The LO-leakage spike at 0 Hz	On initialisation and on retune
RX quadrature	I/Q imbalance on receive — the image	On retune, then tracked
TX quadrature	I/Q imbalance on transmit	On request; this is the one that varies

The DC and quadrature **tracking** loops then keep running while you receive, continuously nudging the corrections. It is tempting to switch them off for a clean measurement; lesson 33 records what happened when that was tried on the OFDM link — **it got worse**. Leave them on unless you have measured that they are hurting.

BIST: MEASURING THE RADIO WITHOUT ANY RADIO

The chip can inject a tone into its own receive path, and can loop its transmit digital data straight back to receive, with no RF involved at all:

```
a known tone, entirely inside the chip                                RUN ON YOUR HOST

iio_attr -u ip:192.168.2.1 -D ad9361-phy bist_tone "2 7680000 0 0"  # on
# ... capture ...
iio_attr -u ip:192.168.2.1 -D ad9361-phy bist_tone "0 0 0 0"        # off
```

This is how the sample-drop measurement in `modulation-and-throughput.md` proved the drops were in the capture and transport path rather than in the radio: the same drops appeared with no RF in the experiment at all. Whenever you cannot tell whether a fault is the radio or everything after it, BIST is the knife that separates them.

How it is configured, and by whom

Not by your fabric. The AD9361's control bus is **the processor's own SPI0, routed out through EMIO** (lesson 23's mechanism) — not the `axi_spi` block that also appears in the block design, which is there for other purposes. The Linux driver owns every register, and the IIO attributes you write are the driver's interpretation of what you asked for.

That is why lesson 13's map matters: your logic sits in the *datapath*, downstream of a chip whose configuration is somebody else's. Nothing you put in the fabric can change the gain, the bandwidth or the LO. You have to ask.

THE ONE OPERATIONAL RULE THAT HAS COST THE MOST TIME HERE

The board mutes its transmitters when no DMA stream is running, and restores a *cached* attenuation when a stream starts. So an attenuation you write **before** opening a buffer guarantees nothing about what happens during it.

Always: open the buffer, *then* set the attenuation, *then* read it back and assert it. Every measurement in this repository does exactly that, and asserts both channels return to the -89.75 dB floor afterwards. The one exception is a single one-shot buffer, which has already finished playing by the time you could write anything.

There is a second edge here worth knowing: on this board the transmit side comes up **energised at power-on**, before any software has run. See `docs/transmitter-safety.md` — and do not attach an antenna to a board you are about to power up.

READ THE CHIP'S OWN OPINION OF ITSELF

what the chip will tell you

RUN ON YOUR HOST

```
iio_attr -u ip:192.168.2.1 -d ad9361-phy          # every device attribute
iio_attr -u ip:192.168.2.1 -c ad9361-phy voltage0 # the receive channel's
iio_attr -u ip:192.168.2.1 -c ad9361-phy temp0 input # die temperature, millidegrees
```

Then run `sdr_selftest.py --ssh`, which reads the supply rails, the die temperatures, the digital interface eye (157 to 181 of 256 delay positions pass on a healthy board) and the internal loopback — all without transmitting anything. It is the fastest way to find out whether a strange measurement is your code or your hardware.

Check yourself: you want RX1 at 900 MHz and RX2 at 2.4 GHz, simultaneously. What happens?

You cannot. There is one `RX_LO` and both receivers hang off it, so both are always tuned to the same frequency. The same is true of `rf_bandwidth`; only **gain** is genuinely per-channel.

What you can do is tune to a centre frequency with both signals inside one receive bandwidth — up to 56 MHz — and separate them in the fabric with two digital down-converters (lesson 28). What you cannot do is cover 900 MHz and 2.4 GHz at once with one board. That takes two boards, and then a shared clock if you want them coherent.

And a corollary worth carrying: because both receivers share the LO, the fix in `both-receive-channels.md` is correct by construction — one anti-alias filter design serves both channels, because both channels are always looking at the same band.

44 Projects, in order of difficulty

Each one uses the board for something software cannot do as well.

1. **A sample-locked trigger.** Divide the sample stream and pulse a header pin. You now have a scope trigger with a fixed, known relationship to the transmitted RF. Uses lessons 13–16.
2. **A power meter.** Accumulate $I^2 + Q^2$ over a window in the fabric and expose the total in a register. The processor reads one number instead of a megabyte of samples. Your first real use of a DSP slice.
3. **A digital downconverter.** NCO plus mixer plus decimating filter: tune to a signal *within* the captured band and deliver only that, at a low rate. Lessons 26–28 combined, and genuinely useful.
4. **A matched filter / correlator.** Detect a known preamble in hardware and raise a flag. The gateway to radar, ranging and packet detection — and the first design where latency, not throughput, is the point.
5. **A modulator.** The exercise from lesson 29.
6. **Hardware timestamping.** Count `1_clk` edges and tag each buffer. This is the missing piece that makes cellular-class stacks work on AD936x boards. Hard, and genuinely valuable.

45 The rules worth taping to the wall

Every one of these was learned the expensive way on this exact board.

- **Delete the Vivado project** before any HDL or coefficient change, or your change is silently ignored.
- **Simulate before you synthesise.** One second against twenty minutes.
- **Gate on the valid strobe**, never on `l_clk` alone — this board is 2R2T, so a sample arrives every second edge.
- **Capture on `fifo_rd_valid | fifo_rd_underflow`**, never on `fifo_rd_en`, which is the request and arrives a cycle early.
- **Synchronise every control bit** that crosses a clock domain, and constrain the crossing with an explicit `-from`.
- **Verify before you flash**, and `./devkit verify --board` after — it is the only thing that proves the board runs what you built.
- **Set TX attenuation after the buffer opens**, then read it back. Writing it before a stream starts guarantees nothing.
- **Never loop transmit back to receive** without at least 20 dB of attenuation. The receive port survives about +2.5 dBm; the transmitter reaches about +19 dBm.
- **When the radio misbehaves, check `/mnt/jffs2` first.** It is writable, it survives reflashing, and a script there can rewrite settings underneath your application.

Measured figures quoted throughout — DSP and LUT counts, timing slack, mute depth, interpolator behaviour — come from one physical board. Treat them as indicative rather than specification.

A Where to go for more, and what this course still leaves out

This course teaches the fabric, the signal processing that feeds it, and enough of the layers either side to build something that works. It does not replace a communications textbook. Here is which book to open for which lesson, what those books taught this one, and — plainly — the ground none of it covers.

The two books worth having open

- SDR4E **Software-Defined Radio for Engineers** — Collins, Getz, Pu and Wyglinski, Artech House 2018. Free from Analog Devices. A communications-theory textbook that uses the Pluto — the same AD9361, the same Zynq — as its instrument. Mathematical, MATLAB-based, and the authoritative source for most of what this course states without deriving.
- PySDR **PySDR: A Guide to SDR and DSP using Python** — Marc Lichtman, pysdr.org. Free online. Lighter, practical, Python throughout, and far more approachable as a first read. If you finished lesson 24 wanting to *play* rather than prove, start here.

Neither has an FPGA chapter. That gap is the reason this course exists — and, equally, why the signal-processing lessons here stop where a textbook would start doing algebra.

Lesson by lesson

Every row is something this course teaches. The two right-hand columns say where to go once the lesson has taken you as far as it intends to.

TOPIC	TAUGHT IN	SDR4E	PYSDR
What an SDR is at all	lesson 0	Ch 1	Ch 1
Fixed point and quantisation	lesson 9	§2.5.2–2.5.3	—
What an IQ sample is	lesson 11	§2.3	Ch 3
Sampling, Nyquist, aliasing	lesson 12	§2.2	Ch 2, 3
Packaging logic as IP, and splicing the datapath	lesson 19	—	—
The frequency domain, FFTs, windows	lesson 24	§2.1	Ch 7
Noise, dB, and where the floor comes from	lesson 25	§3.5–3.7	Ch 10
Filters and convolution	lesson 26	§2.6	Ch 11
Decimation and rate conversion	lesson 27	§2.2.5	Ch 11
Mixers, NCOs, CORDIC	lesson 28	§2.3	Ch 16
Modulation schemes	lesson 29	Ch 4	Ch 4
Pulse shaping, and why RRC	lesson 30	§2.7.3–2.7.5	Ch 16
Timing and carrier recovery	lesson 31	Ch 6, 7	Ch 17
Correlation, matched filters, detection	lesson 32	§2.6.2	Ch 24
OFDM and the cyclic prefix	lesson 33	Ch 10	Ch 13
Equalisation: ZF, MMSE, LMS, DFE	lesson 34	Ch 9	—
Channel coding and interleaving	lesson 35	Ch 8	Ch 13
Framing, media access, retries	lesson 36	—	—
EVM, SNR, SINAD, ENOB, SFDR	lesson 37	§2.4.1, §7.2.4	Ch 10
Link budgets	lesson 38	—	Ch 12
Antennas, matching, the RF front end	lesson 39	Ch 3	Ch 12
Storing IQ with metadata (SigMF)	lesson 40	—	Ch 14
Coherent receivers, diversity, beamforming, MIMO	lessons 41–42	—	Ch 19, 20, 21
The AD9361 as a system	lesson 43	Ch 2	—

What this course still does not teach

Shorter than it used to be, and each of these is a real boundary rather than an oversight.

How modern codes actually decode

Lesson 35 says what turbo and LDPC codes buy and roughly how they work. The iterative message-passing that gets them within 1 dB of Shannon is a field of its own. SDR4E Ch 8 is the next step; nothing in this repository implements one.

Everything above the MAC

Lesson 36 ends at framing, addressing and retries. Routing, congestion control, and the protocols people actually deploy are a different book entirely.

Security

Not touched anywhere. Encryption, authentication, replay protection, and resistance to jamming and spoofing are all real requirements of a real radio link, and a physical layer that works says nothing about any of them.

RF design, as opposed to RF understanding

Lesson 39 explains impedance, matching, antennas and noise figure well enough to use them. *Designing* a matching network, laying out a board, or simulating an antenna needs a Smith chart, a network analyser and an electromagnetic solver.

The AD9361 register by register

Lesson 43 gives the mental model — the rate chain, the gain tables, the calibrations, the shared LOs. The registers themselves, and ADI's own filter design tool, are UG-570 and the driver source.

The underlying theory

Detection and estimation theory, information theory, and the proofs behind "this estimator is optimal". This course tells you which estimator to use and why it behaves as it does; it does not derive it.

Two things the books taught this course

- **"Process gain" is the standard term** for what lesson 27 calls the decimation bonus, and SDR4E §2.5.2 derives it from the ideal-converter formula $6.02N + 1.76$ — which is why decibels convert to bits at all, and why lessons 24, 25, 27 and 32 keep turning out to be the same fact seen from different directions.
- **Choose an awkward test frequency.** SDR4E demonstrates spur-free dynamic range dropping by 11.4 dB purely because the test tone was harmonically related to the sample clock. The full-rate sweep in this repository used a tone at exactly $f_s/8$ — the worst case — so those SFDR figures are pessimistic. That correction came straight out of §2.5.2.

Also worth knowing

- **ADI's Pluto wiki** — wiki.analog.com/university/tools/pluto. The hardware's own documentation, including the parts of the AD9361 this course skips.
- **AD9361 UG-570** — the reference manual behind lesson 43. Long, and the only place some of these answers exist.
- **The SigMF specification** — github.com/sigmf/SigMF. Short, readable, and the whole of lesson 40 in normative form.
- **This repository's own docs/** — [block-design.md](#) is the block-by-block reference this course summarises; [hardware.md](#) is the parts list behind lesson 39; and [measured-performance.md](#), [modulation-and-throughput.md](#) and [both-receive-channels.md](#) hold every number quoted in these pages, with the measurement conditions attached.

Fabric School — written against the Fishball7020 / PlutoSky devkit (Xilinx XC7Z020-CLG400 + Analog Devices AD9361), Vivado/Vitis 2022.2 on Ubuntu 22.04.

Every trap in these pages is one that actually cost someone a build, a measurement, or an afternoon.